

ForeSyte

Project Team

Inam Ullah Shaikh	22i-0857
Emaan Fatima	22i-0869
Aden Sial	22i-1313

Session 2022-2026

Supervised by

Ms. Saira Qamar



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2026

Contents

1	Introduction [AS PER SCOPE DOCUMENT]	1
1.1	Existing Solutions	1
1.1.1	How ForeSyte is Different	2
1.2	Problem Statement	3
1.3	Scope	4
1.4	Modules	5
1.4.1	ForeSyte Web Portal	6
1.4.2	Backend Service	6
1.5	Work Division	7
2	Project Requirements [AS PER FYP1 MID REPORT]	8
2.1	Use-cases	8
2.2	Functional Requirements	16
2.3	Non-Functional Requirements	22
2.3.1	Reliability	22
2.3.2	Usability	22
2.3.3	Performance	22
2.3.4	Security	23
2.3.5	Maintainability	23
3	System Overview [AS PER FYP1 MID REPORT]	24
3.1	Architectural Design	24
3.1.1	Box and Line Diagram (Initial Design Stage)	25
3.1.2	High-Level Architectural Style	25
3.1.3	Mapping of Modules/Components	26
3.2	Data Design	26
3.2.1	Data Structures and Organization	26
3.2.2	Major Data Entities	27
3.2.3	Data Flow Diagrams (DFDs)	27
3.2.4	Data Flow and Processing	30
3.3	Domain Model	30
3.4	Design Models [UPTO THE CURRENT ITERATION ONLY]	30
3.4.1	Class Diagram	30

3.4.2	Activity Diagram	33
3.4.3	System Sequence Diagram	33
3.4.4	Sequence Diagrams	35
3.4.5	State Transition Diagram	42
4	Implementation and Testing	44
4.1	Current Progress	44
4.1.1	Front-End	44
4.1.2	Back-End	45
4.2	External APIs/SDKs	45
4.2.1	External APIs and SDKs Table	46
4.2.2	Detailed Usage	46
4.3	Algorithm Design	49
4.3.1	Admin Dashboard Aggregation	49
4.3.2	Report Generation Workflow (Planned Design)	50
4.3.3	Investigator Violation Review Flow	51
4.3.4	Seating Plan Upload and Processing	51
4.4	Testing Details	55
4.4.1	Unit Testing	55
	References	63
A	Appendices	64
A.1	Appendix A	65
A.1.1	Admin Portal Screenshots	65
A.1.1.1	Login Page	65
A.1.1.2	Sign Up Page	66
A.1.1.3	Dashboard	67
A.1.1.4	Analytics Page	68
A.1.1.5	Exams Page	69
A.1.1.6	Users Page	70
A.1.1.7	Incidents Page	71
A.1.1.8	Monitoring Page	72
A.1.1.9	Reports Page	73
A.1.1.10	Seating Plan Page	74
A.1.1.11	Upload Seating Plan Page	75
A.1.1.12	Settings Page	76
A.1.1.13	Test Results	77
A.1.2	Investigator Portal Screenshots	78
A.1.2.1	Investigator Dashboard	78
A.1.2.2	Violations Page	79

A.1.2.3	Invigilator Activities Page	80
A.1.2.4	Student Activities Page	81
A.1.2.5	Reports Page	82
A.1.2.6	Live Monitoring Page	83
A.1.2.7	Video Processing Page	84
A.1.2.8	Settings Page	85
A.1.2.9	Test Results	86
A.1.3	Student Portal Screenshots	87
A.1.3.1	Student Dashboard	87
A.1.3.2	Disciplinary Records Page	88
A.1.3.3	Profile Page	89
A.1.3.4	Settings Page	90
A.1.3.5	Test Results	91

List of Figures

1.1	Estimated reduction in cheating rates using ForeSyte AI-based surveillance system	4
1.2	Frequency of students who reported cheating during written examinations [3]	4
1.3	Scope workflow of the ForeSyte system	5
2.1	ForeSyte Use Case Diagram	16
3.1	Box and Line Diagram for ForeSyte AI Exam Surveillance System	25
3.2	High-Level Architectural Design of the ForeSyte AI Exam Surveillance System	26
3.3	DFD Level 0 – ForeSyte System Context Diagram	28
3.4	DFD Level 1 – Detailed ForeSyte System Data Flow	29
3.5	ForeSyte Domain Model Diagram	31
3.6	ForeSyte Class Diagram	32
3.7	ForeSyte Activity Diagram	33
3.8	System Sequence Diagram for ForeSyte	34
3.9	Sequence Diagram for UC-01: View and Manage Reports/Dashboard	35
3.10	Sequence Diagram for UC-02: Manage User Profiles	36
3.11	Sequence Diagram for UC-03: Generate Detection Reports	37
3.12	Sequence Diagram for UC-04: Verify Student Identity & Seat Mapping	38
3.13	Sequence Diagram for UC-05: Monitor Invigilator Activity	39
3.14	Sequence Diagram for UC-06: Monitor Student Behaviour	40
3.15	Sequence Diagram for UC-07: Process Exam Footage (Live/Recorded)	41
3.16	State Transition Diagram	42
A.1	Admin Portal Login Page	65
A.2	Admin Portal Sign Up Page	66
A.3	Admin Portal Dashboard	67
A.4	Admin Portal Analytics Page	68
A.5	Admin Portal Exams Management Page	69
A.6	Admin Portal Users Management Page	70
A.7	Admin Portal Incidents Page	71
A.8	Admin Portal Monitoring Page	72
A.9	Admin Portal Reports Page	73
A.10	Admin Portal Seating Plan Management Page	74

LIST OF FIGURES

A.11 Admin Portal Upload Seating Plan Page	75
A.12 Admin Portal Settings Page	76
A.13 Admin Portal Unit Test Results	77
A.14 Investigator Portal Dashboard	78
A.15 Investigator Portal Violations Management Page	79
A.16 Investigator Portal Invigilator Activities Page	80
A.17 Investigator Portal Student Activities Page	81
A.18 Investigator Portal Reports Page	82
A.19 Investigator Portal Live Monitoring Page	83
A.20 Investigator Portal Video Processing Page	84
A.21 Investigator Portal Settings Page	85
A.22 Investigator Portal Unit Test Results	86
A.23 Student Portal Dashboard	87
A.24 Student Portal Disciplinary Records Page	88
A.25 Student Portal Profile Page	89
A.26 Student Portal Settings Page	90
A.27 Student Portal Unit Test Results	91

List of Tables

1.1	Comparison of Existing Solutions	2
1.2	Work Division among Team Members	7
2.1	Summary of Use Cases for ForeSyte	8
2.2	Use Case 1: View & Manage Reports/Dashboard	9
2.3	Use Case 2: Manage User Profiles	10
2.4	Use Case 3: Generate Detection Reports	11
2.5	Use Case 4: Seat Mapping	12
2.6	Use Case 5: Monitor Invigilator Activity	13
2.7	Use Case 6: Monitor Student Behaviour	14
2.8	Use Case 7: Process Exam Footage (Live/Recorded)	15
2.9	Functional Requirements for UC-01	17
2.10	Functional Requirements for UC-02	17
2.11	Functional Requirements for UC-03	17
2.12	Functional Requirements for UC-04	18
2.13	Functional Requirements for UC-05	18
2.14	Functional Requirements for UC-06	18
2.15	Functional Requirements for UC-07	19
2.16	General Functional Requirements	19
4.1	External APIs and SDKs for ForeSyte	46
4.2	Unit Testing for Investigator Portal	56
4.3	Unit Testing for Investigator Portal (continued)	57
4.4	Unit Testing for Student Portal	58
4.5	Unit Testing for Student Portal (continued)	59
4.6	Unit Testing for Admin Portal	60
4.7	Unit Testing for Admin Portal (continued)	61
4.8	Unit Testing for Admin Portal (continued)	62

Chapter 1

Introduction [AS PER SCOPE DOCUMENT]

This project proposal presents **ForeSyte**, an AI-powered CCTV/camera monitoring system designed to detect and log cheating attempts and monitor invigilator activity during examinations. The purpose of this document is to outline the problem, motivation, proposed solution, and stakeholders while providing a detailed background of the system.

ForeSyte integrates computer vision and deep learning techniques to analyze both real-time CCTV/camera feeds and uploaded exam footage. By focusing on seat-specific mapping, the system ensures precise tracking of students and their activities while simultaneously monitoring invigilators' behavior to maintain a fair and transparent examination environment [3].

Furthermore, several studies have highlighted the growing prevalence of academic dishonesty and the limitations of traditional manual invigilation methods. By leveraging AI-driven video analysis, **ForeSyte** aims to improve the accuracy of cheating detection, automate timestamped reporting, and enhance overall exam integrity. The system also acts as a deterrent; as students become aware that their activities are continuously monitored, the likelihood of cheating is expected to decrease significantly over time [4].

1.1 Existing Solutions

Several solutions have been developed to address the problem of cheating detection and exam monitoring, each with distinct approaches and limitations. Below is a critical overview and comparison of the most relevant systems:

- **IndoAI:** Employs proprietary CNN-based gesture and face models running on Edge AI cameras. While it offers AI-driven gesture and face analysis, it does not support

seat-based activity detection or invigilator monitoring. Additionally, it lacks the capability to generate structured, timestamped reports. Its reliance on specialized hardware (Edge AI cameras) limits scalability and integration with existing CCTV systems.

- **IPB University System:** Uses dual webcams (front and rear) for each student, combined with AI software for face and gaze monitoring. This approach is not based on CCTV and does not provide seat-based activity detection or invigilator monitoring. While it offers some level of timestamped reporting, it lacks structured, per-seat violation logs and is less scalable for large exam halls due to the need for individual webcams per student.

Table 1.1: Comparison of Existing Solutions

System Name	Technology	Limitations
IndoAI	Proprietary CNN-based gesture + face models on Edge AI cameras.	No seat-based activity detection; no invigilator monitoring; no structured timestamped reports; requires specialized hardware.
IPB University System	Dual webcam (front + rear) per student; AI software (face/gaze monitoring models); not CCTV.	No seat-based activity detection; no invigilator monitoring; limited but no structured per-seat timestamped reports; not scalable for large halls.

1.1.1 How ForeSyte is Different

ForeSyte stands out from existing solutions by addressing their key limitations:

- **Seat-based Activity Detection:** ForeSyte uniquely maps detected activities to specific seats, ensuring identification of students involved in suspicious behavior.
- **Integration with Existing CCTV:** Unlike other solutions that require specialized hardware or individual webcams, ForeSyte works seamlessly with standard CCTV infrastructure, making it scalable and cost-effective for large exam halls.
- **Invigilator Monitoring:** ForeSyte not only monitors students but also tracks invigilator presence and activity, identifying negligence or prolonged absences.

- **Structured, Timestamped Reports:** The system generates detailed, per-seat violation logs with timestamps, enabling efficient review and transparent reporting.
- **Comprehensive Automation:** By combining real-time and recorded video analysis, ForeSyte automates the entire monitoring and reporting process, reducing manual effort and enhancing exam integrity.

This approach ensures that ForeSyte provides a comprehensive, scalable, and automated exam monitoring solution that overcomes the major shortcomings of existing systems.

1.2 Problem Statement

Academic dishonesty has become a significant concern in educational institutions worldwide. Studies indicate that approximately 61% of university students have admitted to cheating during written examinations, while nearly 68% confess to cheating at least once during their academic career [3]. Traditional manual invigilation methods are often insufficient, as invigilators may unintentionally overlook suspicious activities, and reviewing long hours of CCTV/camera footage after exams is both time-consuming and ineffective.

To address these challenges, **ForeSyte** provides an AI-driven solution for automated examination monitoring. The system processes both real-time CCTV/camera streams and uploaded video footage, detecting suspicious activities such as phone usage, hand waving, bending over the desk, looking around, and standing up using advanced computer vision techniques.

The system focuses on seat-specific mapping, ensuring that every detected activity is linked to the correct student. In addition, **ForeSyte** incorporates an Invigilator Activity Monitoring module to:

- Track invigilators' movements
- Detect prolonged idle periods
- Identify potential negligence

An admin dashboard provides real-time monitoring and automated report generation, enabling discipline committees to directly access timestamped violations instead of manually searching through hours of footage. By combining AI-powered surveillance with intelligent analytics, **ForeSyte** enhances exam security and aims to reduce academic dishonesty over time by providing actionable insights. Furthermore, as students become aware that their activities are continuously monitored and every suspicious action is captured with timestamped reports, the likelihood of cheating is expected to decrease significantly over repeated examination sessions.

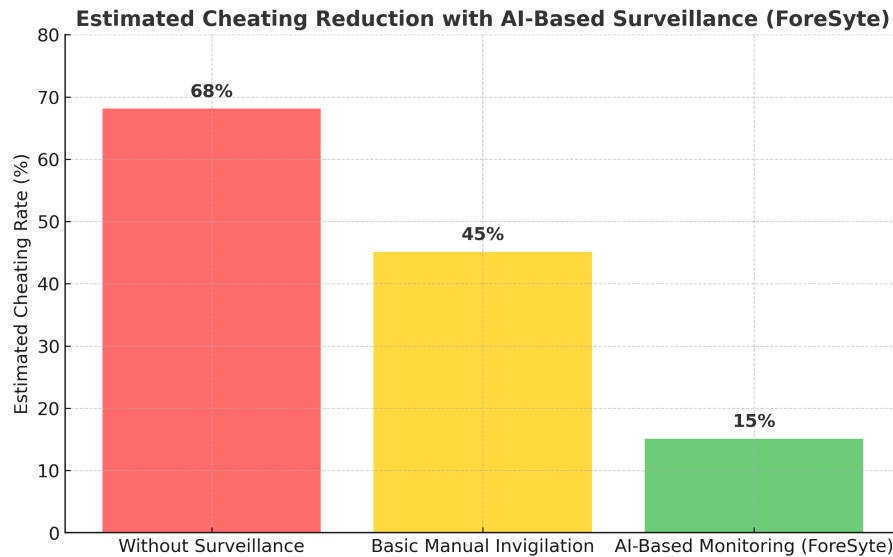


Figure 1.1: Estimated reduction in cheating rates using ForeSyte AI-based surveillance system

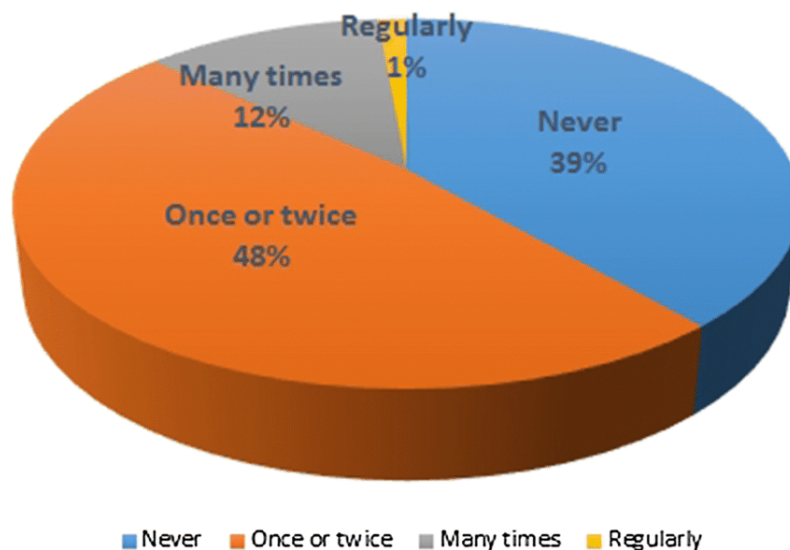


Figure 1.2: Frequency of students who reported cheating during written examinations [3]

1.3 Scope

The scope of the ForeSyte project includes developing an AI-powered exam surveillance system that enhances the integrity of examinations conducted in physical exam halls. Recent research highlights the importance of automated invigilation for maintaining academic integrity [3, 4].

The system will process CCTV/camera footage, both live streams and uploaded recordings, to automatically detect and log suspicious activities such as unauthorized object

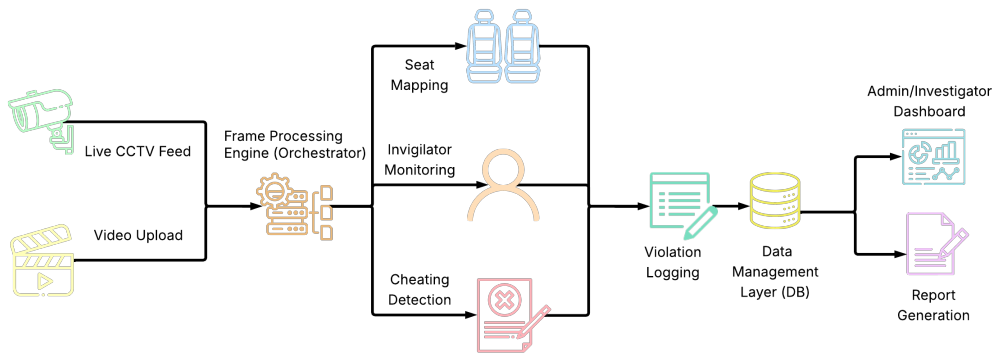


Figure 1.3: Scope workflow of the ForeSyte system

usage and irregular gestures. It will also incorporate seating arrangement verification, ensuring that each detected activity is accurately linked to the correct student through seat mapping, similar to approaches in modern proctoring systems [1, 5].

The project further extends to monitoring invigilator activity within the exam hall, identifying negligence or prolonged absence from duty. Similar works on offline and intelligent monitoring systems support this approach [6, 7]. All detected events will be recorded with precise timestamps, seat numbers and room identifiers, enabling a structured and transparent record of violations.

An administrative dashboard will be developed to centralize monitoring and provide clear violation reports supported with visual evidence. This will reduce the need for institutions to manually review extensive CCTV/camera footage, offering instead a concise and accurate summary of flagged incidents.

The system will be designed for real-time performance using optimized frame sampling and AI models capable of efficient video analysis. The project will be self-funded and developed entirely by our team. Related research also discusses the ethical implications of AI in exam supervision, highlighting fairness and privacy issues.

The boundaries of this project are confined to creating a robust and intelligent surveillance solution that automates detection, verification, and reporting within physical exam environments, thereby ensuring greater fairness, efficiency, and accountability in academic assessments.

1.4 Modules

The **ForeSyte** project is organized into several core modules, each responsible for specific functionalities required to ensure seamless, automated exam monitoring. The modules are divided into two main components: the *ForeSyte Web Portal* for stakeholders and the

Backend Service that powers the AI-driven monitoring capabilities.

1.4.1 ForeSyte Web Portal

The **ForeSyte Web Portal** is a multi-user interface designed for administrators, discipline committee members, and university management. It provides centralized access to seating plans and violation reports, ensuring transparent and efficient management of examination activities.

1. **Dashboard Visualization:** Displays real-time updates of suspicious activities, flagged violations, and invigilator presence across multiple exam halls using data from connected CCTV/camera systems.
2. **Report Access:** Enables authorized users to view, manage, and download detailed violation reports containing timestamps, seat details and room information.
3. **User Roles and Permissions:** Implements secure role-based access, where administrators manage seating plans, camera configurations, and exam schedules, while discipline committee members focus on reviewing reports and initiating actions.
4. **Seating Plan Integration:** Links violations to student seating layouts to ensure accurate identification of candidates involved in flagged incidents.

1.4.2 Backend Service

The **Backend Service** powers the core intelligence of the ForeSyte system, managing video analysis, behavioral detection, invigilator monitoring, and reporting. This module processes video streams from CCTV/camera systems and uploaded exam footage to ensure accurate real-time and recorded exam monitoring.

1. **Video Stream Processing:** Integrates with CCTV/camera systems and uploaded recordings, extracting frames for real-time detection of activities and events.
2. **Cheating Detection Engine:** Uses advanced computer vision techniques to identify suspicious behaviors such as unauthorized object usage, irregular gestures, communication, and frequent seat movements [1, 2].
3. **Seat Mapping:** Maps detected activities to seating plans for accurate violation reporting, ensuring each activity is linked to the correct student seat.
4. **Invigilator Monitoring:** Tracks invigilator's prolonged absences from assigned zones, and identifies potential negligence or unauthorized phone usage[6].

5. **Report Generation:** Compiles timestamped violation logs, including room details, seat numbers, and integrates them directly into the ForeSyte Web Portal for automated PDF/Excel report exports.

1.5 Work Division

For each module and respective feature, responsibility is assigned to a team member.

Name	Responsibility
Inam Ullah Shaikh	• Frontend Development • Model Training • Documentation • Deployment • Backend Integration
Emaan Fatima	• Backend Development • Model Research • Model Training • Documentation • Deployment
Aden Sial	• Data Collection • Model Training • Model Research • Documentation • Deployment

Table 1.2: Work Division among Team Members

Chapter 2

Project Requirements [AS PER FYP1 MID REPORT]

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-cases

The Table 2.1 lists down all the possible use cases for our project. Note that login, sign up and feedback pages are not included in these use cases.

Table 2.1: Summary of Use Cases for ForeSyte

Usecase ID	Use Case Name	Description
UC-01	View & Manage Reports/Dashboard	Authorized users view and manage reports of student and invigilator activity.
UC-02	Manage User Profiles	System administrator manages accounts and roles of all users.
UC-03	Generate Detection Reports	System generates timestamped reports of detected cheating or negligence.
UC-04	Seat Mapping	System maps detected activities to assigned student seats.
UC-05	Monitor Invigilator Activity	System monitors invigilators for negligence, absence, or unauthorized actions.
UC-06	Monitor Student Behaviour	System monitors student actions to detect cheating attempts.
UC-07	Process Exam Footage (Live/Recorded)	System processes live and recorded video streams to detect activities.

Below, you can find the detailed use case description for each use case.

2.1.1 UC-01

Table 2.2: Use Case 1: View & Manage Reports/Dashboard

Use Case Detail	Description
Use Case ID	UC-01
Use Case Name	View & Manage Reports/Dashboard
Scope	ForeSyte Web Portal
Level	User Goal
Brief Description	Users view, filter, and manage real-time dashboards and past reports.
Actor(s) Involved	System Administrator, Investigator, Discipline Committee
Stakeholders	• Discipline Committee: Reviews violation data • Admins: Configure reports
Pre-conditions	Reports and logs already generated by detection system.
Post-conditions	Users access detailed dashboards with downloadable reports.
Main Success Scenario	1. Actor logs into the ForeSyte Web Portal successfully. 2. Actor navigates to the Reports/Dashboard section from the main menu. 3. System retrieves all available violation reports from the database. 4. System displays a dashboard with filters (e.g., by student ID, seat number, violation type, or timestamp). 5. Actor applies filters to refine the report view. 6. System dynamically updates the dashboard according to the selected filters. 7. Actor selects a specific report for detailed view. 8. System displays complete details of the selected report (student, invigilator, time, violation evidence). 9. Actor chooses the option to download or export the report. 10. System generates the report in the selected format (PDF/Excel) and confirms successful export.
Special Requirements	Role-based access, secure data visualization

2.1.2 UC-02

Table 2.3: Use Case 2: Manage User Profiles

Use Case Detail	Description
Use Case ID	UC-02
Use Case Name	Manage User Profiles
Scope	ForeSyte Web Portal
Level	User Goal
Brief Description	System administrator manages user accounts, roles, and permissions.
Actor(s) Involved	System Administrator
Stakeholders	• Admins: Maintain structured access
Pre-conditions	Admin is authenticated.
Post-conditions	User accounts created, modified, or deleted.
Main Success Scenario	1. Admin logs into the ForeSyte Web Portal. 2. Admin navigates to the User Management module. 3. System displays the list of existing users with their roles. 4. Admin selects an action (add, edit, or delete a user). 5. If adding: Admin enters new user details including user-name, role, and credentials. 6. If editing: Admin updates user details or permissions. 7. If deleting: Admin selects a user and confirms deletion. 8. System validates the request and updates the database accordingly. 9. System confirms success with a message (user added/updated/deleted). 10. Admin logs out or continues with further management tasks.
Special Requirements	Secure password policies, role-based restrictions

2.1.3 UC-03

Table 2.4: Use Case 3: Generate Detection Reports

Use Case Detail	Description
Use Case ID	UC-03
Use Case Name	Generate Detection Reports
Scope	Backend Service + Web Portal
Level	User Goal
Brief Description	System generates timestamped detection reports of suspicious activity.
Actor(s) Involved	Investigator, Discipline Committee
Stakeholders	• Discipline Committee: Reviews reports • University Management
Pre-conditions	Detection system has analyzed video footage.
Post-conditions	Report generated and stored in database.
Main Success Scenario	1. AI detection engine processes exam footage. 2. System identifies suspicious activity (e.g., phone usage, looking around, invigilator negligence). 3. System logs details including timestamp, seat number, room, and involved actor. 4. System compiles the violation data into a structured report format. 5. Investigator accesses the Web Portal and navigates to the Reports section. 6. System displays the list of generated reports. 7. Investigator selects a specific report for review. 8. System displays detailed evidence including video snippets or screenshots. 9. Investigator verifies and approves the report for further review. 10. Report is stored in the system and available for export (PDF/Excel).
Special Requirements	Reports exportable to PDF/Excel

2.1.4 UC-04

Table 2.5: Use Case 4: Seat Mapping

Use Case Detail	Description
Use Case ID	UC-04
Use Case Name	Seat Mapping
Scope	Backend Service
Level	User Goal
Brief Description	System maps detected activities to assigned student seats.
Actor(s) Involved	System (Automated)
Stakeholders	• University Management
Pre-conditions	Seating plan and student ID data available.
Post-conditions	Detected activities are mapped to correct student seats.
Main Success Scenario	1. Exam seating plan is uploaded into the system. 2. System processes video feeds from CCTV or camera sources. 3. System detects suspicious activities in the video frames. 4. System maps detected activities to specific seat locations using seating plan data. 5. System links each detected activity to the student assigned to that seat. 6. System logs the activity along with seat ID and student ID. 7. Mapped data is linked to subsequent activity monitoring and reports.
Special Requirements	Accurate seat mapping with reliable coordinate detection

2.1.5 UC-05

Table 2.6: Use Case 5: Monitor Invigilator Activity

Use Case Detail	Description
Use Case ID	UC-05
Use Case Name	Monitor Invigilator Activity
Scope	Backend Service
Level	User Goal
Brief Description	System tracks invigilators for negligence, absence, or unauthorized actions.
Actor(s) Involved	Invigilator, Discipline Committee
Stakeholders	• University Management
Pre-conditions	Cameras active and invigilator zones defined.
Post-conditions	Invigilator activity log generated.
Main Success Scenario	1. Surveillance cameras start recording invigilator activity. 2. System tracks invigilator's presence across different zones of the exam hall. 3. AI analyzes the video feed to detect movement or prolonged inactivity. 4. If invigilator leaves their zone for too long, the system flags negligence. 5. If invigilator is using unauthorized devices (e.g., phone), system captures it. 6. System logs the timestamp, location, and detected action. 7. Discipline Committee accesses the dashboard to view invigilator logs. 8. System displays flagged negligence cases in the report. 9. Committee reviews and validates the report. 10. Finalized invigilator activity log is stored and exported if needed.
Special Requirements	Accurate zone mapping

2.1.6 UC-06

Table 2.7: Use Case 6: Monitor Student Behaviour

Use Case Detail	Description
Use Case ID	UC-06
Use Case Name	Monitor Student Behaviour
Scope	Backend Service
Level	User Goal
Brief Description	Detects cheating behaviors (phone use, gestures, looking around).
Actor(s) Involved	Student, Investigator
Stakeholders	• Discipline Committee • University Management
Pre-conditions	Camera feed and detection models active.
Post-conditions	Violations logged with timestamps.
Main Success Scenario	1. Exam begins and surveillance cameras monitor all students. 2. AI detection model continuously analyzes student actions. 3. System identifies suspicious behavior such as phone use, hand gestures, or looking around repeatedly. 4. When detected, system flags the behavior as a potential violation. 5. System records timestamp, seat ID, and video evidence. 6. Investigator accesses the dashboard to review logged violations. 7. System displays detailed violation logs with associated video frames. 8. Investigator examines the evidence for each flagged action. 9. Investigator confirms or dismisses the violation. 10. Validated violations are added to the final exam report.
Special Requirements	Must differentiate normal vs cheating actions

2.1.7 UC-07

Table 2.8: Use Case 7: Process Exam Footage (Live/Recorded)

Use Case Detail	Description
Use Case ID	UC-07
Use Case Name	Process Exam Footage (Live/Recorded)
Scope	Backend Service
Level	User Goal
Brief Description	System processes live and recorded exam video footage for analysis.
Actor(s) Involved	System Administrator, Investigator
Stakeholders	• University Management
Pre-conditions	Camera streams or uploaded recordings available.
Post-conditions	Footage analyzed and results stored.
Main Success Scenario	1. System connects to live CCTV cameras or receives uploaded exam recordings. 2. System validates video input and prepares it for analysis. 3. AI engine processes video frames in real time or batch mode. 4. System identifies and logs suspicious student or invigilator behavior. 5. System maps detected activity to the correct student seat. 6. All activities are logged with timestamps and stored in the database. 7. Investigator accesses the Reports module. 8. System displays results of processed footage (violations, seat mapping). 9. Investigator reviews the logged activities and evidence. 10. Reports are finalized and exported in required format.
Special Requirements	Efficient processing, storage optimization

2.1.8 Use Case Diagram of ForeSyte

The Figure 2.1 highlights the use case diagram of the entire ForeSyte Web Application.

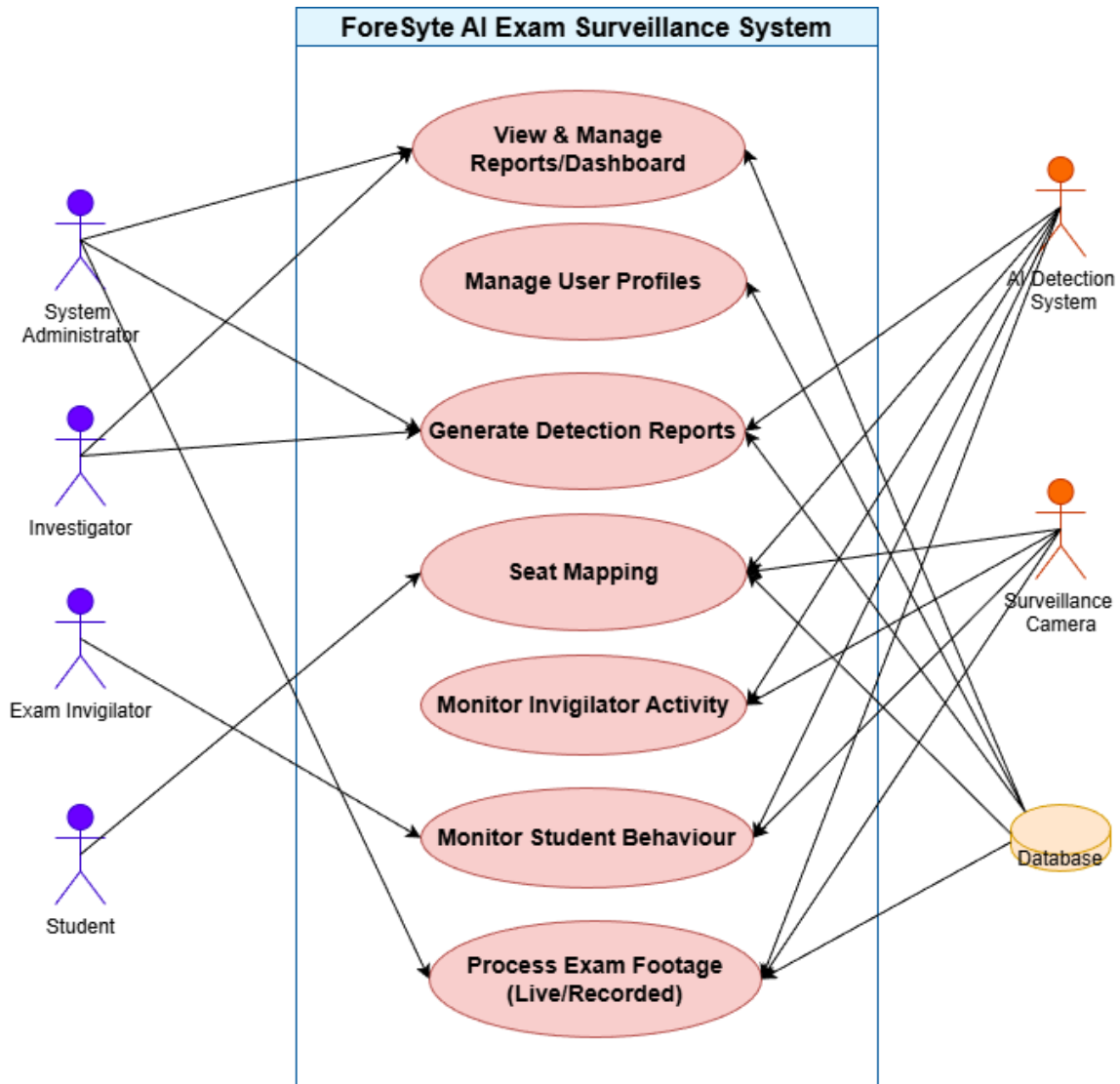


Figure 2.1: ForeSyte Use Case Diagram

2.2 Functional Requirements

This section describes the functional requirements of ForeSyte arranged in a use case-based organizational method.

2.2.1 UC-01 Functional Requirement

Table 2.9: Functional Requirements for UC-01

Functional Requirement ID	Description
FR-01	The system shall allow authorized users to log in and access the Reports/Dashboard module.
FR-02	The system shall display real-time and historical reports of student and invigilator activity.
FR-03	The system shall provide filtering options (by student ID, seat number, violation type, timestamp).
FR-04	The system shall allow exporting reports in PDF and Excel formats.
FR-05	The system shall enforce role-based access to reports.

2.2.2 UC-02 Functional Requirement

Table 2.10: Functional Requirements for UC-02

Functional Requirement ID	Description
FR-06	The system shall allow the administrator to add new user accounts with role assignments.
FR-07	The system shall allow editing of existing user details and permissions.
FR-08	The system shall allow deleting user accounts securely.
FR-09	The system shall validate inputs (username, password, role) before saving.
FR-10	The system shall enforce strong password and security policies.

2.2.3 UC-03 Functional Requirement

Table 2.11: Functional Requirements for UC-03

Functional Requirement ID	Description
FR-11	The system shall automatically generate detection reports when violations are flagged.
FR-12	The system shall include timestamps, seat numbers, and violation type in reports.
FR-13	The system shall store generated reports in a database.
FR-14	The system shall allow investigators to review and approve reports.
FR-15	The system shall allow reports to be exported in PDF/Excel format.

2.2.4 UC-04 Functional Requirement

Table 2.12: Functional Requirements for UC-04

Functional Requirement ID	Description
FR-16	The system shall map detected activities to assigned seat numbers.
FR-17	The system shall link activities to students based on seating plan data.
FR-18	The system shall log activity mapping results (student ID, seat ID, activity type).
FR-19	The system shall ensure accurate coordinate-based seat detection.
FR-20	The system shall handle cases where seat mapping cannot be determined.

2.2.5 UC-05 Functional Requirement

Table 2.13: Functional Requirements for UC-05

Functional Requirement ID	Description
FR-21	The system shall monitor invigilators during exam sessions.
FR-22	The system shall track invigilator presence in defined zones.
FR-23	The system shall detect unauthorized activities (e.g., phone use).
FR-24	The system shall log negligence events with timestamp and location.
FR-25	The system shall provide invigilator activity logs to the Discipline Committee.

2.2.6 UC-06 Functional Requirement

Table 2.14: Functional Requirements for UC-06

Functional Requirement ID	Description
FR-26	The system shall analyze student actions in real time.
FR-27	The system shall detect suspicious behaviors (phone use, hand gestures, looking around).
FR-28	The system shall differentiate between normal and abnormal behaviors.
FR-29	The system shall log violations with seat ID, timestamp, and video snippet.
FR-30	The system shall allow investigators to review and confirm logged violations.

2.2.7 UC-07 Functional Requirement

Table 2.15: Functional Requirements for UC-07

Functional Requirement ID	Description
FR-31	The system shall process both live CCTV feeds and uploaded recordings.
FR-32	The system shall analyze frames using AI detection models.
FR-33	The system shall log activities with timestamps and seat IDs.
FR-34	The system shall store processed results in the database.
FR-35	The system shall make analyzed results available in the Reports module.

2.2.8 General Functional Requirements

Table 2.16: General Functional Requirements

Functional Requirement ID	Description
FR-36	The system shall ensure processing of violations and reports within 20 seconds.
FR-37	The system shall implement secure storage and encryption of student/exam data.
FR-38	The system shall allow a responsive web interface for investigators and administrators.
FR-39	The system shall maintain audit logs for all activities.
FR-40	The system shall support scalability for multiple exam halls simultaneously.

2.2.9 Module 1: ForeSyte Web Portal

Following are the requirements for Module 1 (ForeSyte Web Portal):

1. Dashboard Visualization

- (a) FR-01: The system shall allow authorized users to log in and access the Reports/Dashboard module.
- (b) FR-02: The system shall display real-time and historical reports of student and invigilator activity.
- (c) FR-03: The system shall provide filtering options (by student ID, seat number, violation type, timestamp).
- (d) FR-04: The system shall allow exporting reports in PDF and Excel formats.

- (e) FR-05: The system shall enforce role-based access to reports.

2. Report Access

- (a) FR-11: The system shall automatically generate detection reports when violations are flagged.
- (b) FR-12: The system shall include timestamps, seat numbers, and violation type in reports.
- (c) FR-13: The system shall store generated reports in a database.
- (d) FR-14: The system shall allow investigators to review and approve reports.
- (e) FR-15: The system shall allow reports to be exported in PDF/Excel format.

3. User Roles and Permissions

- (a) FR-06: The system shall allow the administrator to add new user accounts with role assignments.
- (b) FR-07: The system shall allow editing of existing user details and permissions.
- (c) FR-08: The system shall allow deleting user accounts securely.
- (d) FR-09: The system shall validate inputs (username, password, role) before saving.
- (e) FR-10: The system shall enforce strong password and security policies.

2.2.10 Module 2: Backend Service

Following are the requirements for Module 2 (Backend Service):

1. Video Stream Processing

- (a) FR-31: The system shall process both live CCTV feeds and uploaded recordings.
- (b) FR-32: The system shall analyze frames using AI detection models.
- (c) FR-33: The system shall log activities with timestamps and seat IDs.
- (d) FR-34: The system shall store processed results in the database.
- (e) FR-35: The system shall make analyzed results available in the Reports module.

2. Cheating Detection Engine

- (a) FR-26: The system shall analyze student actions in real time.

- (b) FR-27: The system shall detect suspicious behaviors (phone use, hand gestures, looking around).
- (c) FR-28: The system shall differentiate between normal and abnormal behaviors.
- (d) FR-29: The system shall log violations with seat ID, timestamp, and video snippet.
- (e) FR-30: The system shall allow investigators to review and confirm logged violations.

3. Seat Mapping

- (a) FR-16: The system shall map detected activities to assigned seat numbers.
- (b) FR-17: The system shall link activities to students based on seating plan data.
- (c) FR-18: The system shall log activity mapping results (student ID, seat ID, activity type).
- (d) FR-19: The system shall ensure accurate coordinate-based seat detection.
- (e) FR-20: The system shall handle cases where seat mapping cannot be determined.

4. Invigilator Monitoring

- (a) FR-21: The system shall monitor invigilators during exam sessions.
- (b) FR-22: The system shall track invigilator presence in defined zones.
- (c) FR-23: The system shall detect unauthorized activities (e.g., phone use).
- (d) FR-24: The system shall log negligence events with timestamp and location.
- (e) FR-25: The system shall provide invigilator activity logs to the Discipline Committee.

5. Report Generation and Security

- (a) FR-36: The system shall ensure processing of violations and reports within 20 seconds.
- (b) FR-37: The system shall implement secure storage and encryption of student/exam data.
- (c) FR-38: The system shall allow a responsive web interface for investigators and administrators.
- (d) FR-39: The system shall maintain audit logs for all activities.
- (e) FR-40: The system shall support scalability for multiple exam halls simultaneously.

2.3 Non-Functional Requirements

This section specifies the non-functional requirements of the ForeSyte system. These requirements are practical, measurable, and verifiable to ensure the system can be realistically implemented in an academic setting.

2.3.1 Reliability

- REL-1: The system shall achieve at least 95% uptime during scheduled exam sessions.
- REL-2: In case of camera or feed failure, the system shall log the event and notify the administrator within 1 minute.
- REL-3: The system shall store video footage and violation data redundantly to prevent data loss.

2.3.2 Usability

- USE-1: An administrator shall be able to generate a violation report in no more than 3 interactions.
- USE-2: New users (administrators or investigators) shall be able to learn the core functions of the system with less than 2 hours of training.
- USE-3: The dashboard shall present violation summaries in a clear tabular format with filters.

2.3.3 Performance

- PER-1: The system shall process live camera feeds with a maximum latency of 10 seconds.
- PER-2: The cheating detection engine shall analyze at least 20 frames per second per camera feed.
- PER-3: Violation reports shall be generated within 20 minutes of the end of an exam session.

2.3.4 Security

- SEC-1: Only authenticated users shall access the system dashboard.
- SEC-2: User access shall be role-based (administrator, investigator).
- SEC-3: All access attempts shall be logged for audit purposes.

2.3.5 Maintainability

- MAI-1: The system shall follow a modular architecture so that detection models can be updated independently.
- MAI-2: Minor bugs shall be diagnosable and fixable within 48 hours.
- MAI-3: The documentation shall describe system modules, and troubleshooting in detail for developers.

Chapter 3

System Overview [AS PER FYP1 MID REPORT]

ForeSyte AI Exam Surveillance System is an intelligent monitoring solution designed to enhance the integrity of in-person examinations by automating student and invigilator monitoring. The system integrates CCTV cameras, computer vision, and deep learning models to detect suspicious student behaviors such as phone usage, frequent seat movements, or unauthorized communication, while also tracking invigilator activities to ensure accountability. Each activity is mapped to seating plans to ensure accurate identification of students. Detected events are logged as timestamped violations, which are compiled into structured reports accessible through an administrative dashboard. Administrators manage user profiles, seating plans, and exam sessions, while disciplinary investigators utilize the violation reports to review cases and make informed decisions. The system follows a modular architecture consisting of two primary components: the ForeSyte Web Portal, a multi-user web interface for dashboard visualization and report management, and the Backend Service, the core AI engine handling video stream processing, cheating detection, seat mapping, and automated report generation. This architectural design ensures separation of concerns, scalability, and maintainability while providing seamless integration between the user interface and the underlying AI processing capabilities.

3.1 Architectural Design

The ForeSyte AI Exam Surveillance System is designed using a modular architecture to ensure scalability, maintainability, and clear separation of concerns. At a high level, the system is divided into front-end modules, back-end modules, and supporting external systems. Each module plays a distinct role but collaborates with others to achieve the overall goal of automated, AI-powered exam monitoring.

The architecture follows a **two-tier client-server architecture** with **layered components**, implementing a **RESTful API** design pattern. The front-end (presentation tier) handles user input, report visualization, and dashboard interactions through a React-based single-page application. The back-end (application tier) processes video streams, performs AI-based detection and analysis, and manages business logic through FastAPI routes. The data management layer (data tier) ensures persistence and retrieval of information using PostgreSQL for relational data and MongoDB for high-velocity event logs. This decomposition ensures that the AI detection logic, data storage, and user interfaces remain loosely coupled and independently upgradable.

3.1.1 Box and Line Diagram (Initial Design Stage)

In the initial design stage, a simple box and line diagram is used to represent the main system components and their interactions. This diagram provides a high-level overview of how the core modules connect, without detailing the internal architecture or data flow.

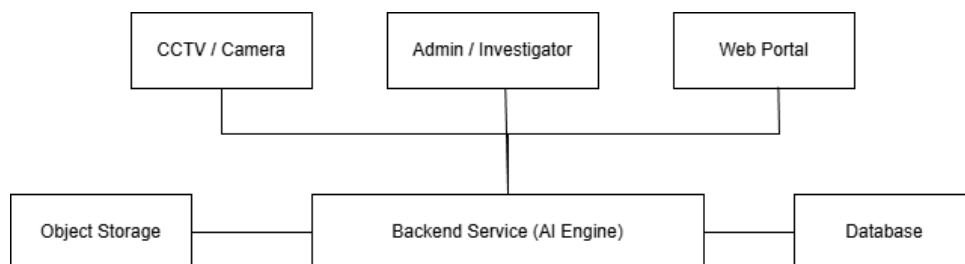
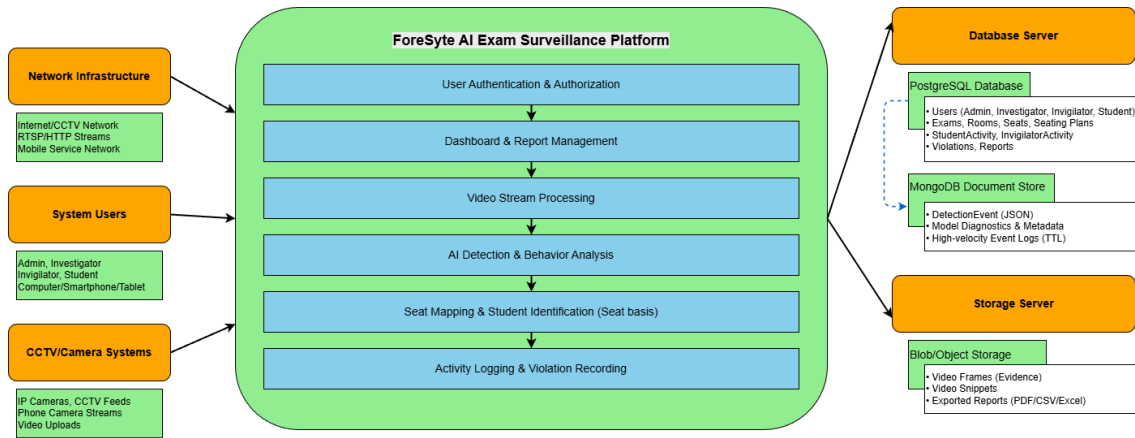


Figure 3.1: Box and Line Diagram for ForeSyte AI Exam Surveillance System

3.1.2 High-Level Architectural Style

After the initial overview, the finalized architecture style/pattern diagram is presented. The system uses a two-tier client-server architecture with layered components, providing clear separation between the presentation layer (React frontend), application layer (FastAPI backend with embedded business logic), and data management layer (PostgreSQL and MongoDB). Figure 3.2 illustrates the high-level architecture of the system, showing the main subsystems and their interactions.

Figure 3.2: High-Level Architectural Design of the ForeSyte AI Exam Surveillance System



3.1.3 Mapping of Modules/Components

Each module/component is mapped to its respective part of the architecture, ensuring modularity and scalability. The architecture supports integration with external systems (CCTV, storage, databases) and provides a robust foundation for AI-powered exam monitoring.

3.2 Data Design

The data design for **ForeSyte** focuses on how the system's information domain is represented and transformed into data structures across relational, document, and object stores. In this section, we outline the major system entities, how they are stored, processed, and managed to ensure robust operation and secure handling of user information and evidence.

3.2.1 Data Structures and Organization

ForeSyte deals with multiple data classes: user identities, seating plans, live/recorded video evidence, AI detections, activities, violations, and reports. The core data is organized so that:

- **Relational (PostgreSQL)** holds normalized, queryable facts: Users, Exams, Rooms, Seats, StudentActivity, InvigilatorActivity, Violations, Reports, and seat assignments; enforced with foreign keys.
- **Document (MongoDB)** holds high-velocity *DetectionEvent* JSON and model diagnostics with TTL, enabling fast ingest and optional replay/debug.

- **Blob/Object Storage** (evidence bucket) stores immutable artifacts: frame crops, video snippets, and exported PDF reports.

Data lifecycle. Streams or uploads produce detection events; an aggregation service converts them into activities and violations with evidence links; dashboards and investigators read relational views and open the referenced evidence in object storage.

3.2.2 Major Data Entities

Key logical entities:

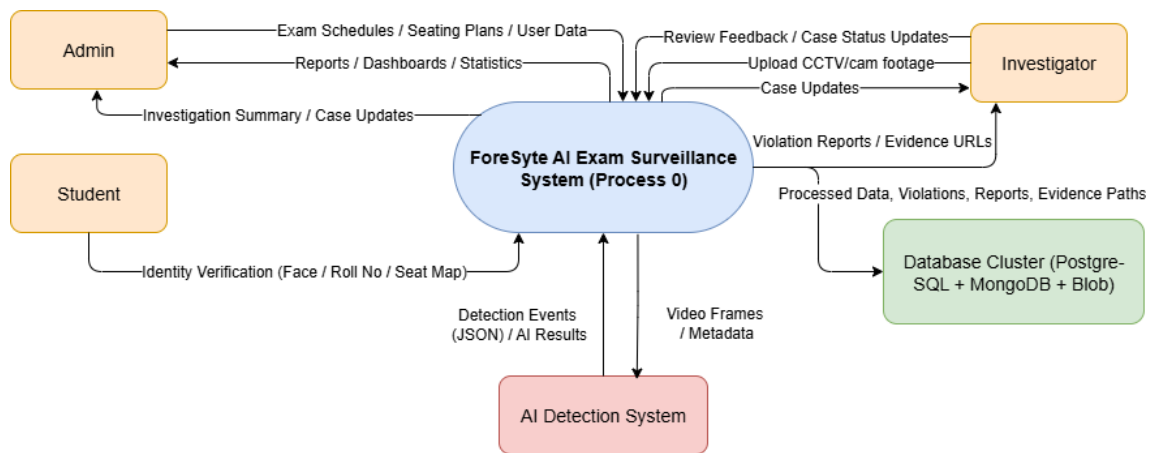
- **User Information:** Admin, Invigilator, Investigator, and Student profiles with credentials and roles.
- **Exam/Room/Seat:** Exam sessions; rooms with cameras; seats mapped to students and exams.
- **DetectionEvent** (document): Raw detector output with timestamps, camera/room source, and model metadata.
- **StudentActivity/InvigilatorActivity:** Curated activities derived from detections; include severity, confidence, and optional evidence URL.
- **Violation:** Confirmed cheating events; include type, timestamp, severity, status, and optional evidence URL.
- **Report:** Generated summaries (PDF) of violations and activities for investigators.

3.2.3 Data Flow Diagrams (DFDs)

This subsection presents the logical data flow within the ForeSyte system at two abstraction levels. Level 0 provides an overview of the main system interactions, while Level 1 elaborates on internal data processing — especially how seat mapping, activity detection, and behavior monitoring are interconnected.

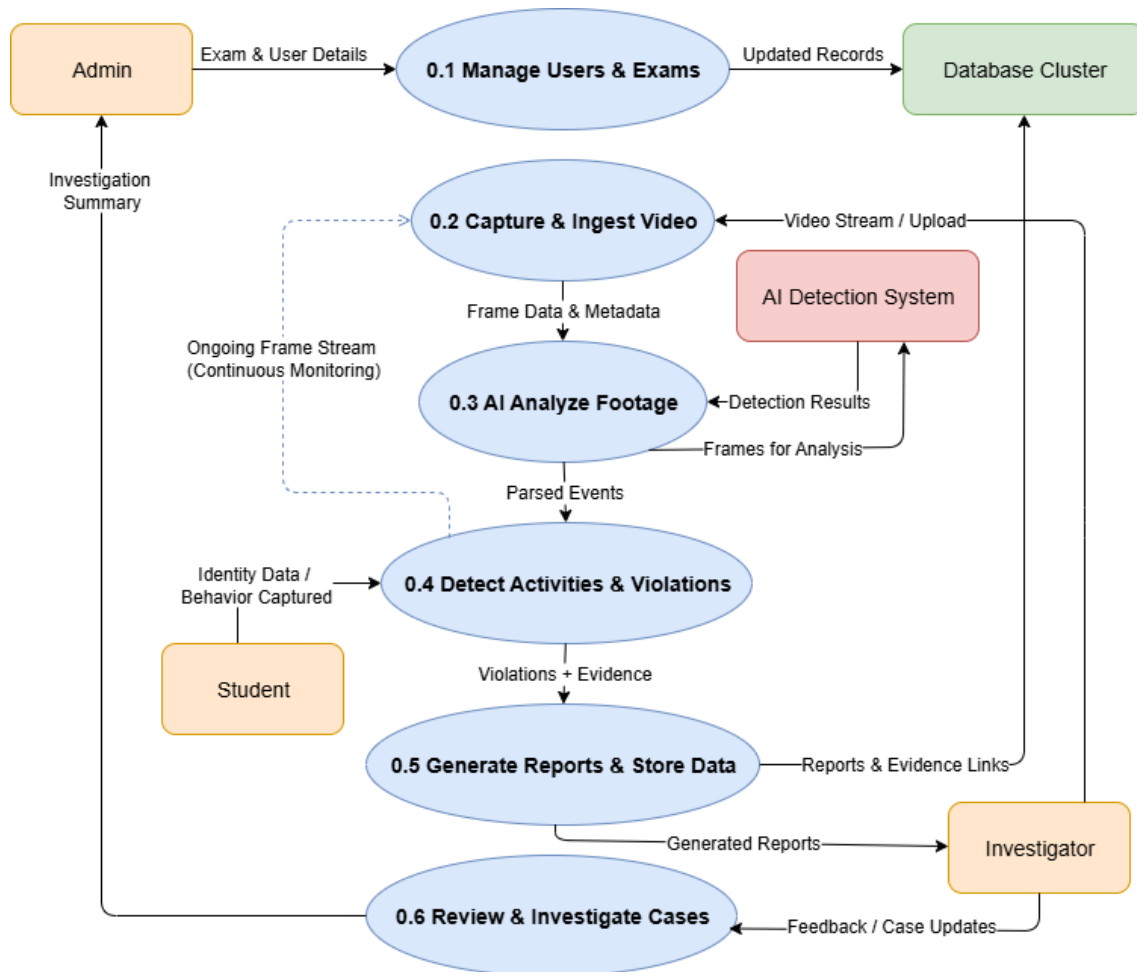
Level 0 – Context Diagram Figure 3.3 illustrates the high-level data flow between external entities (such as Students, Invigilators, and Administrators) and the ForeSyte System. It highlights how examination footage, attendance data, and violation reports circulate through the system.

Figure 3.3: DFD Level 0 – ForeSyte System Context Diagram



Level 1 – System Decomposition Figure 3.4 expands the ForeSyte process into its major internal modules: Seat Mapping, Cheating Detection Engine, Invigilator Monitoring, and Report Generation. Data from entry-point cameras are first processed to map activities to specific seats; this mapping is then propagated through subsequent modules for activity analysis and reporting.

Figure 3.4: DFD Level 1 – Detailed ForeSyte System Data Flow



3.2.4 Data Flow and Processing

The system processes exam footage via two paths that converge into a unified pipeline:

1. **Real-time (Live) Monitoring:** Camera streams feed the AI service, which emits per-frame *DetectionEvents* (JSON).
2. **Uploaded Footage:** After the exam, recorded video is ingested; the AI service produces the same *DetectionEvents* offline.
3. **Aggregation/Rules:** Business rules transform events into *StudentActivity/InvigilatorActivity*; suspicious patterns yield *Violation* rows with evidence links.
4. **Persistence:** Normalized facts land in PostgreSQL; evidence media is stored in Blob/Object storage; optionally, transient raw events remain in MongoDB (TTL).
5. **Consumption:** Admin dashboards query relational views; investigators open pre-signed evidence URLs and download reports.

3.3 Domain Model

Figure 3.5 represents the domain model for ForeSyte. It outlines the key entities, their attributes, and the relationships that define data interactions within the ForeSyte system.

3.4 Design Models [UPTO THE CURRENT ITERATION ONLY]

The ForeSyte system employs an object-oriented development approach, utilizing several key design models to represent the system's structure, behavior, and interactions. These models provide comprehensive documentation of the system's architecture and facilitate understanding of complex relationships between system components.

3.4.1 Class Diagram

The system's class diagram represents the core entities and their relationships within the ForeSyte domain model.

Figure 3.5: ForeSys Domain Model Diagram

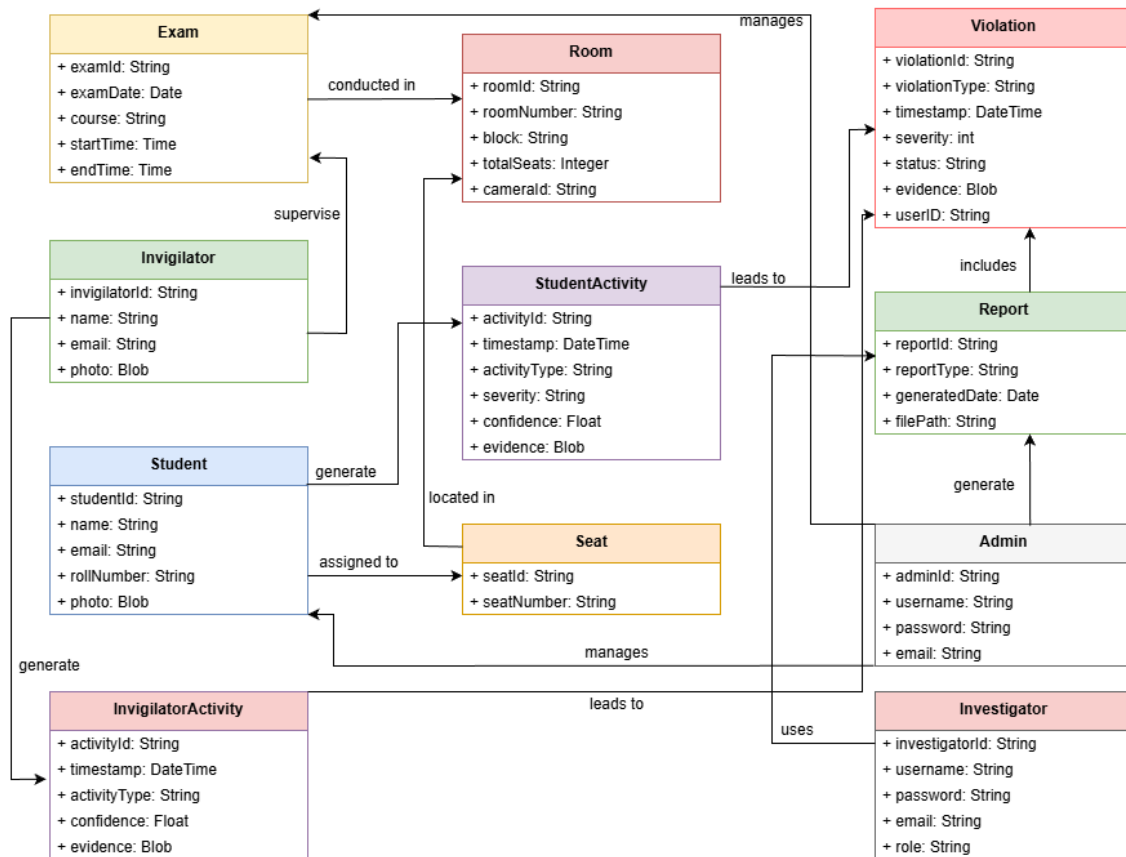
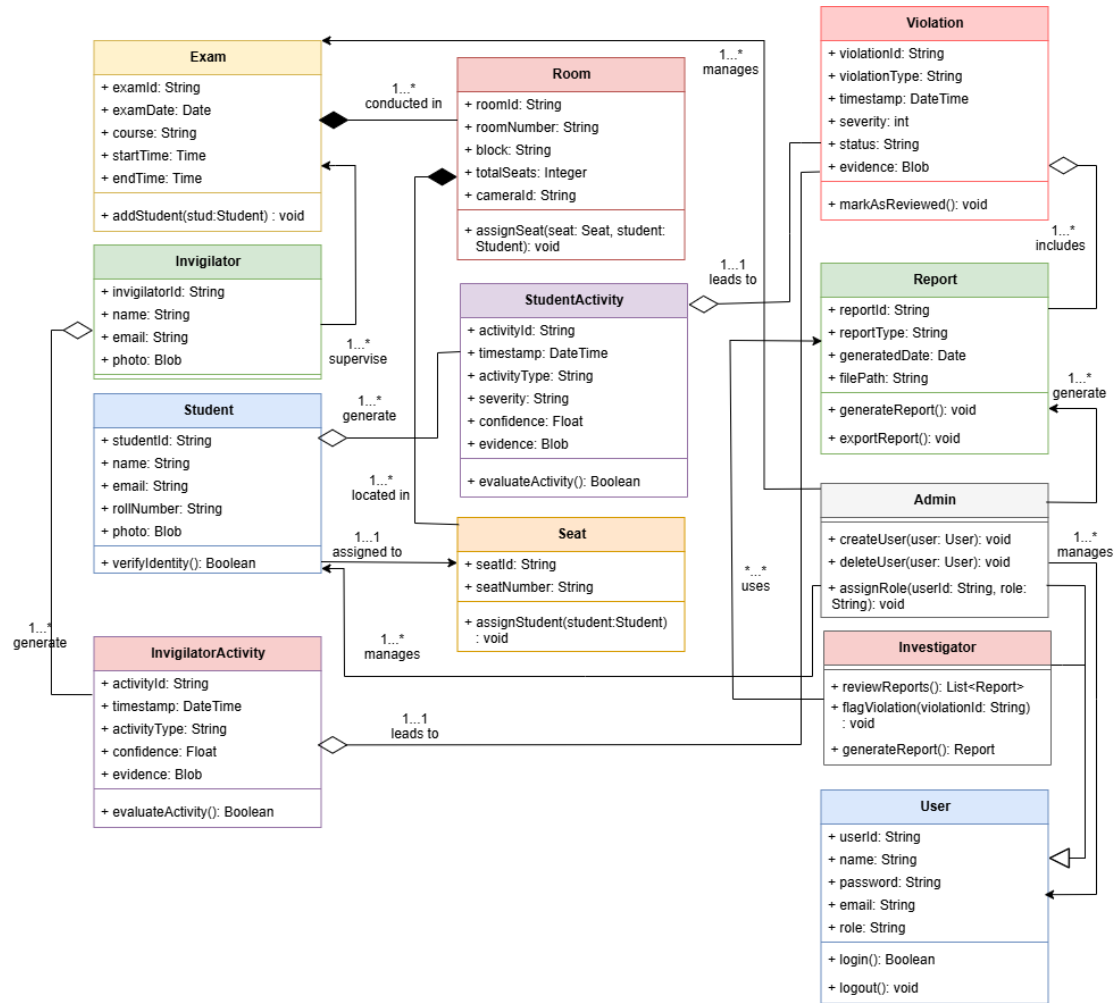


Figure 3.6: ForeSysTe Class Diagram



3.4.2 Activity Diagram

The system's activity diagram illustrates the main processes and workflows:

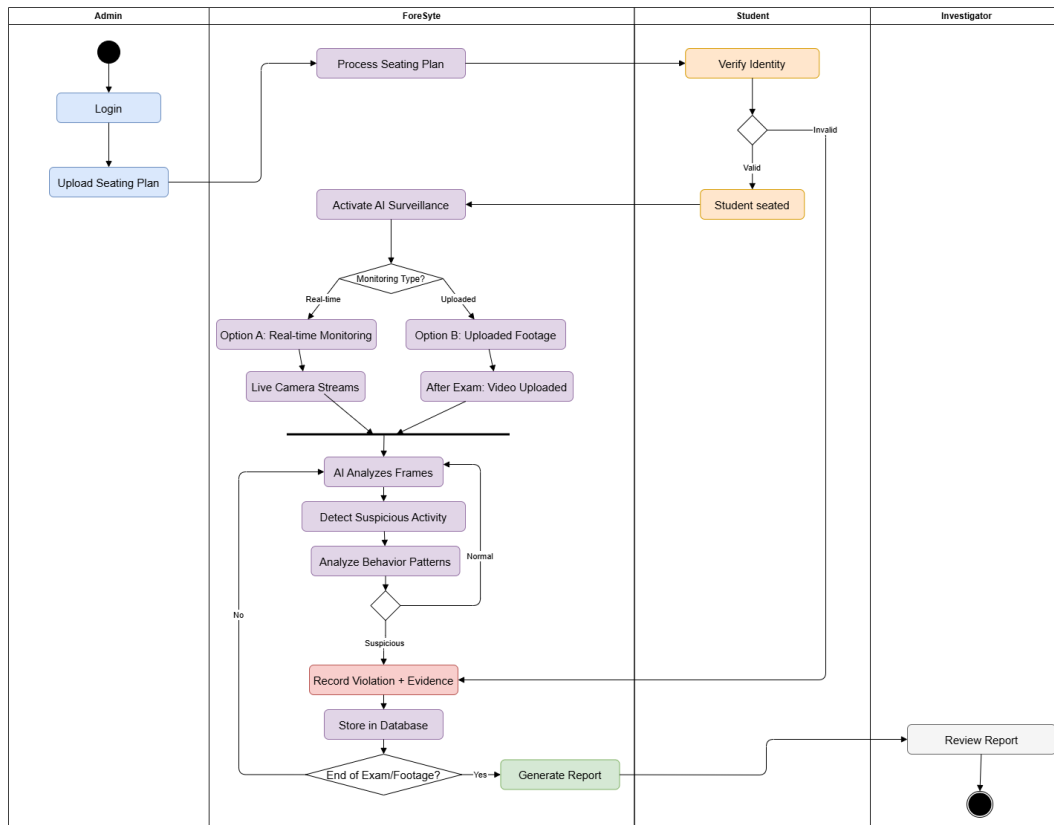


Figure 3.7: ForeSysy Activity Diagram

3.4.3 System Sequence Diagram

Figure 3.8 illustrates the high-level System Sequence Diagram (SSD) for ForeSysy. It depicts the main interactions between external actors (Students, Invigilators, Administrators, and the Discipline Committee) and the ForeSysy AI System across the examination workflow. This diagram provides an overview of how user actions trigger system processes such as attendance tagging, cheating detection, invigilator monitoring, and report generation.

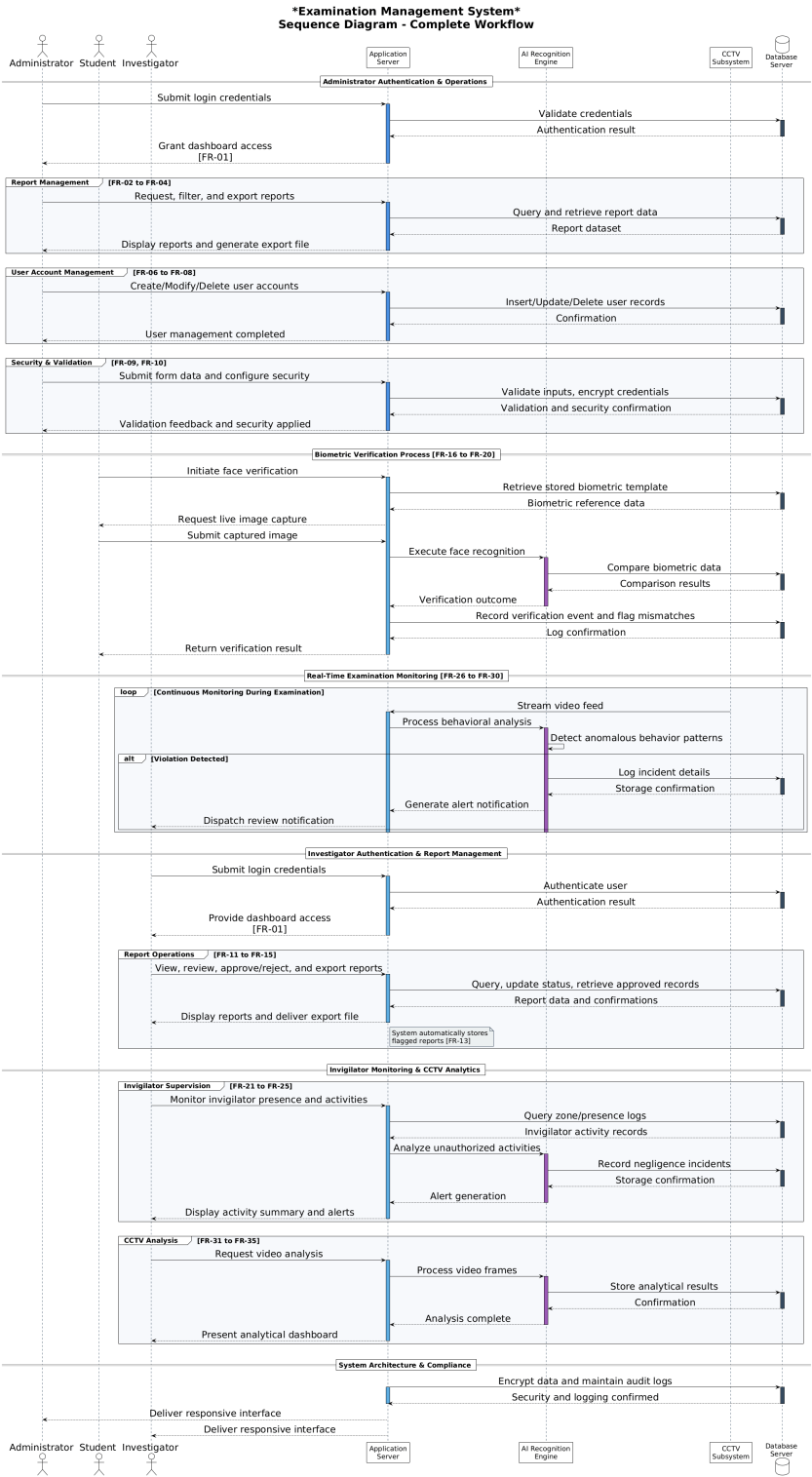


Figure 3.8: System Sequence Diagram for ForeSyte

3.4.4 Sequence Diagrams

This section presents the sequence diagrams for all identified use cases (UC-01 to UC-07). Each diagram illustrates the dynamic interaction between actors, the ForeSyte AI System, and domain entities.

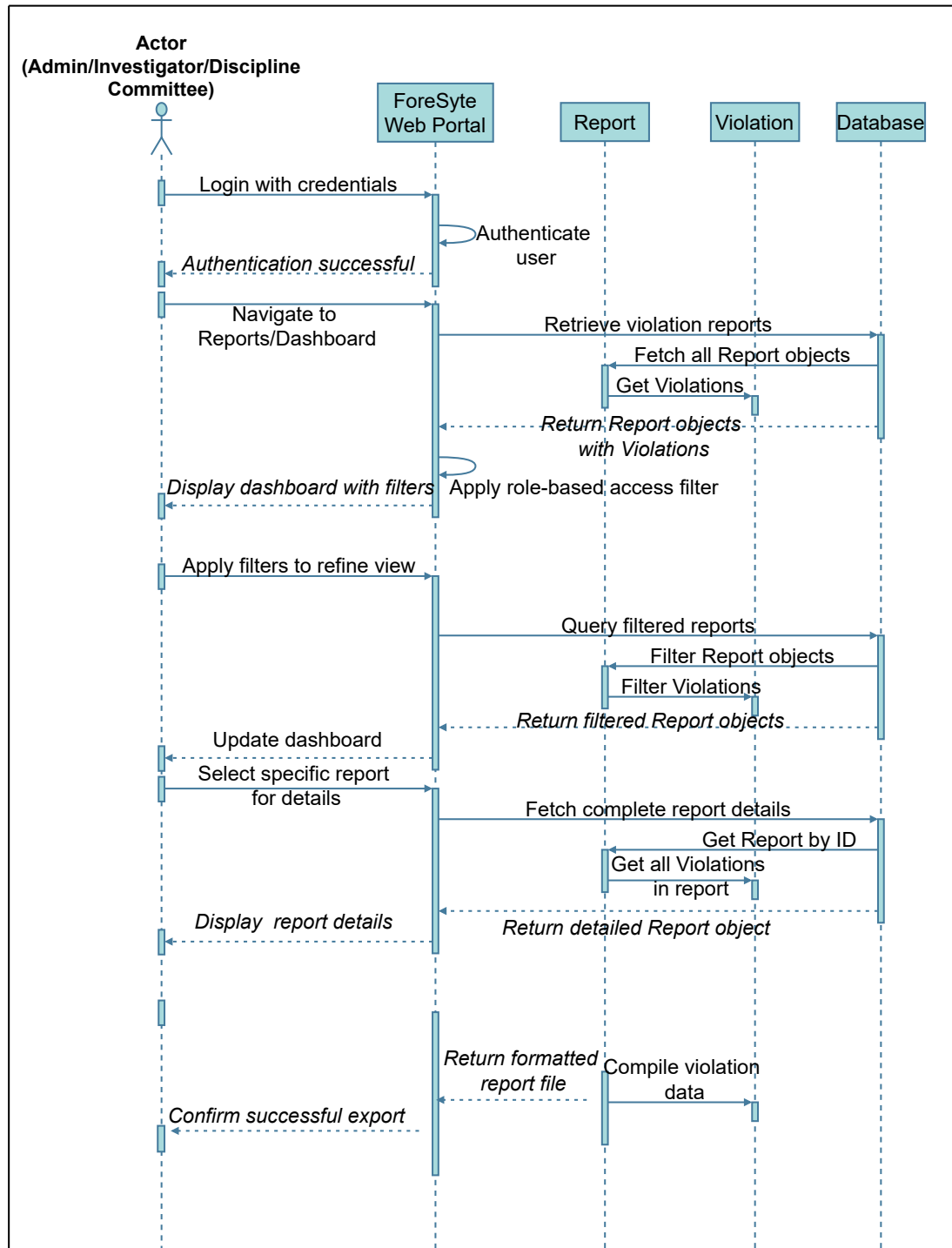


Figure 3.9: Sequence Diagram for UC-01: View and Manage Reports/Dashboard

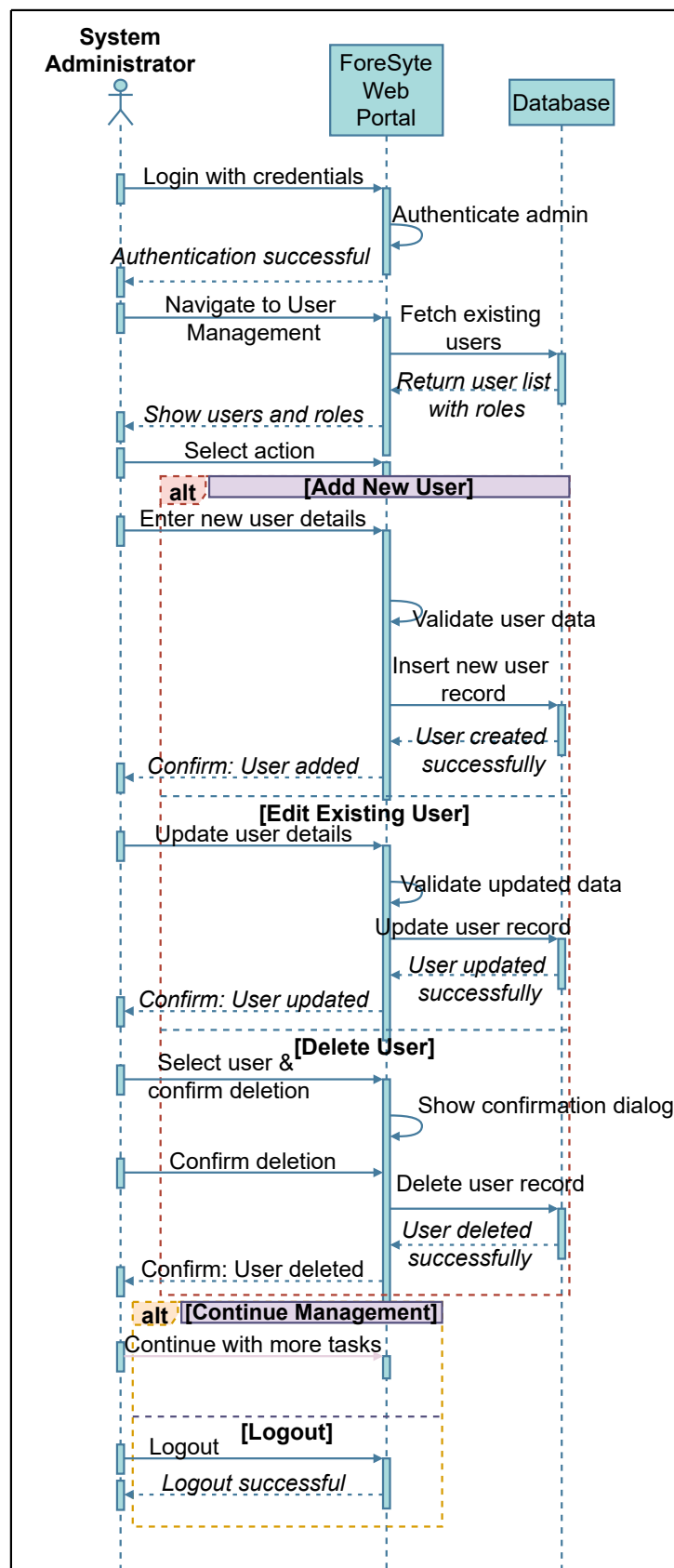


Figure 3.10: Sequence Diagram for UC-02: Manage User Profiles

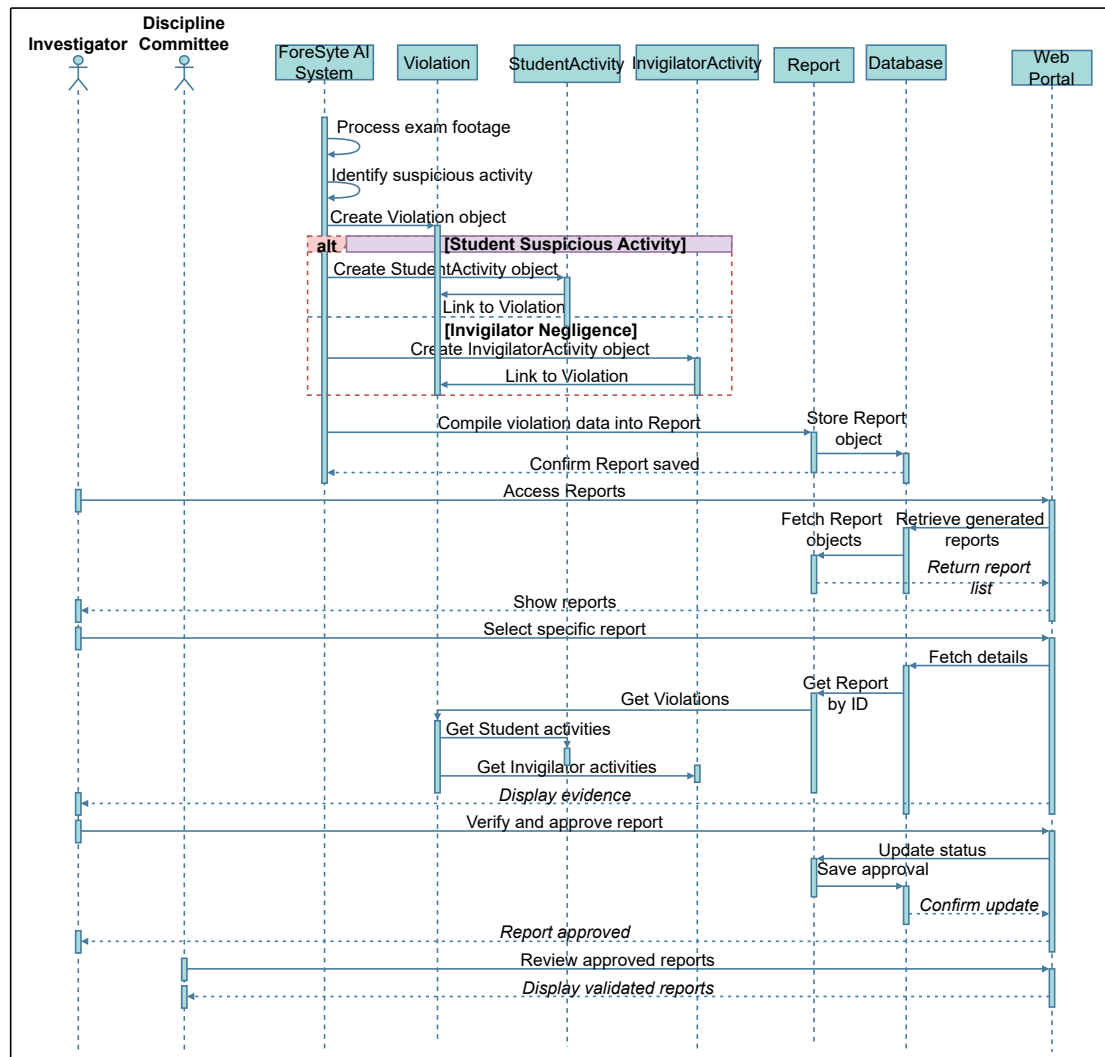


Figure 3.11: Sequence Diagram for UC-03: Generate Detection Reports

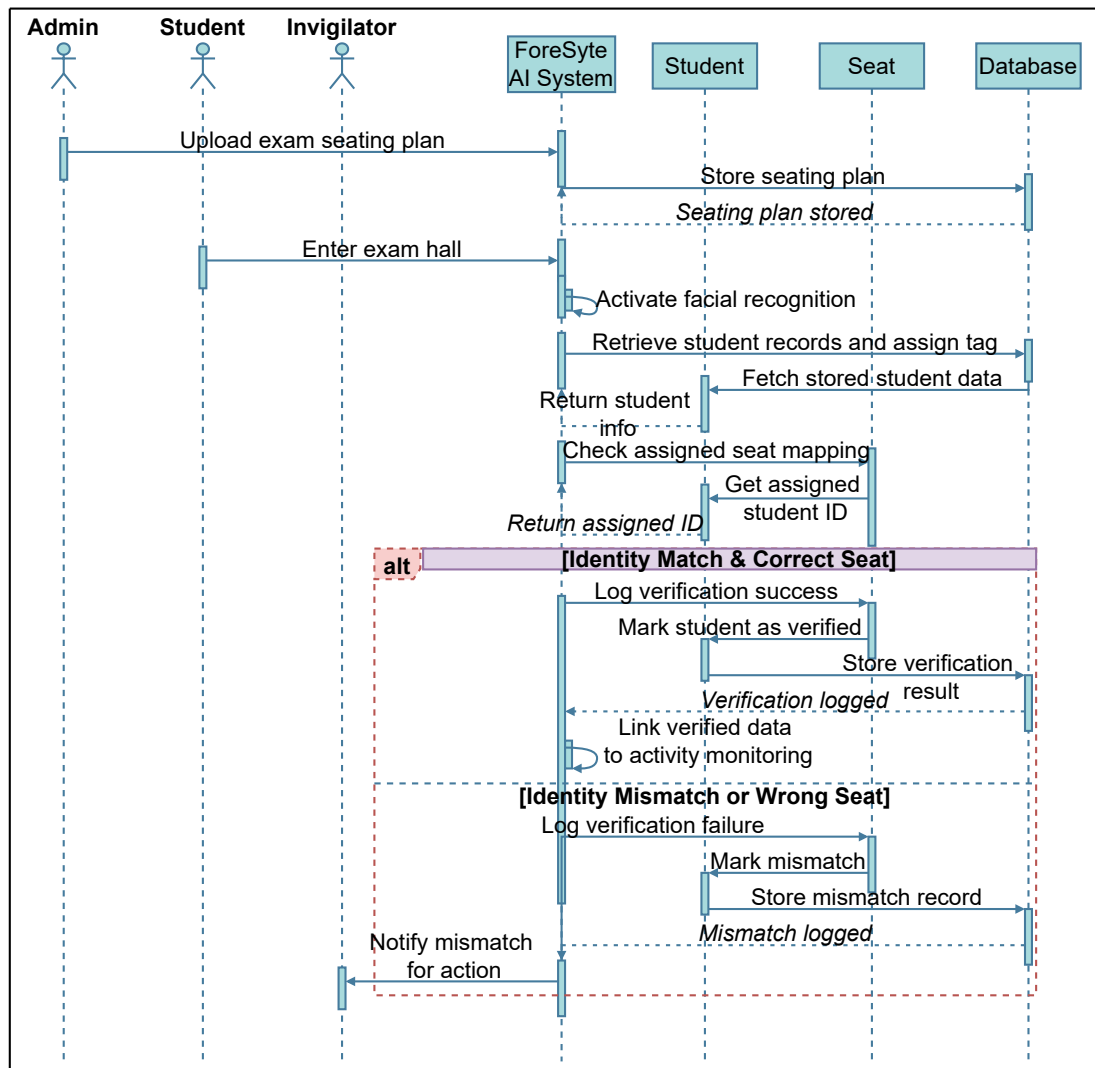


Figure 3.12: Sequence Diagram for UC-04: Verify Student Identity & Seat Mapping

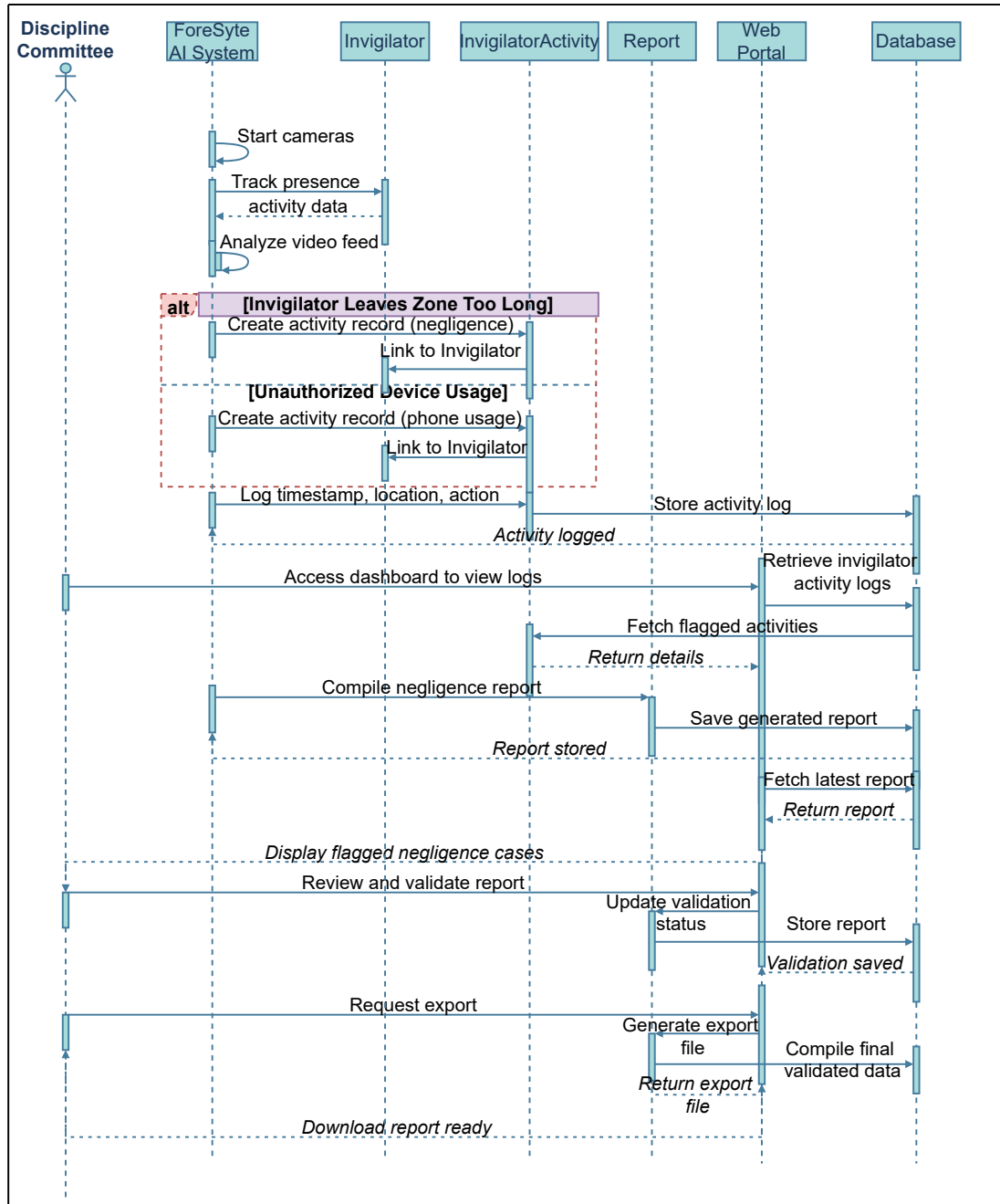


Figure 3.13: Sequence Diagram for UC-05: Monitor Invigilator Activity

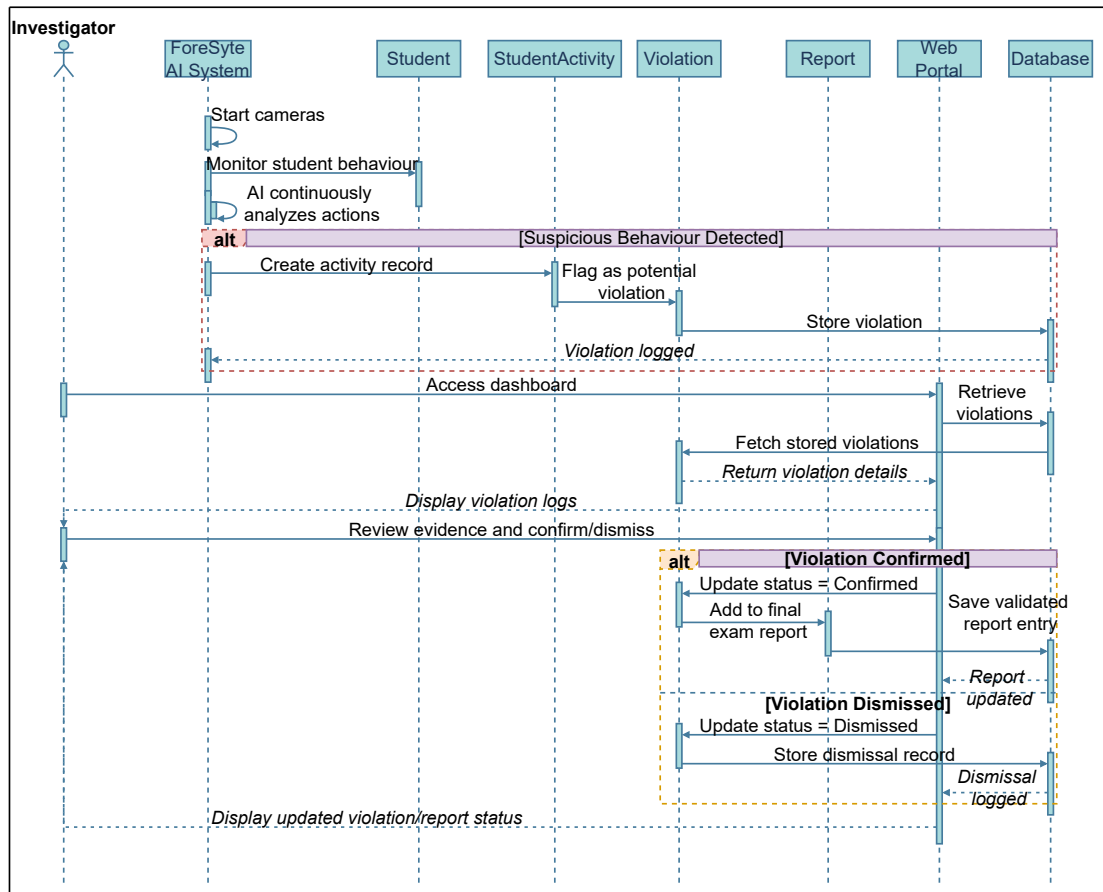


Figure 3.14: Sequence Diagram for UC-06: Monitor Student Behaviour

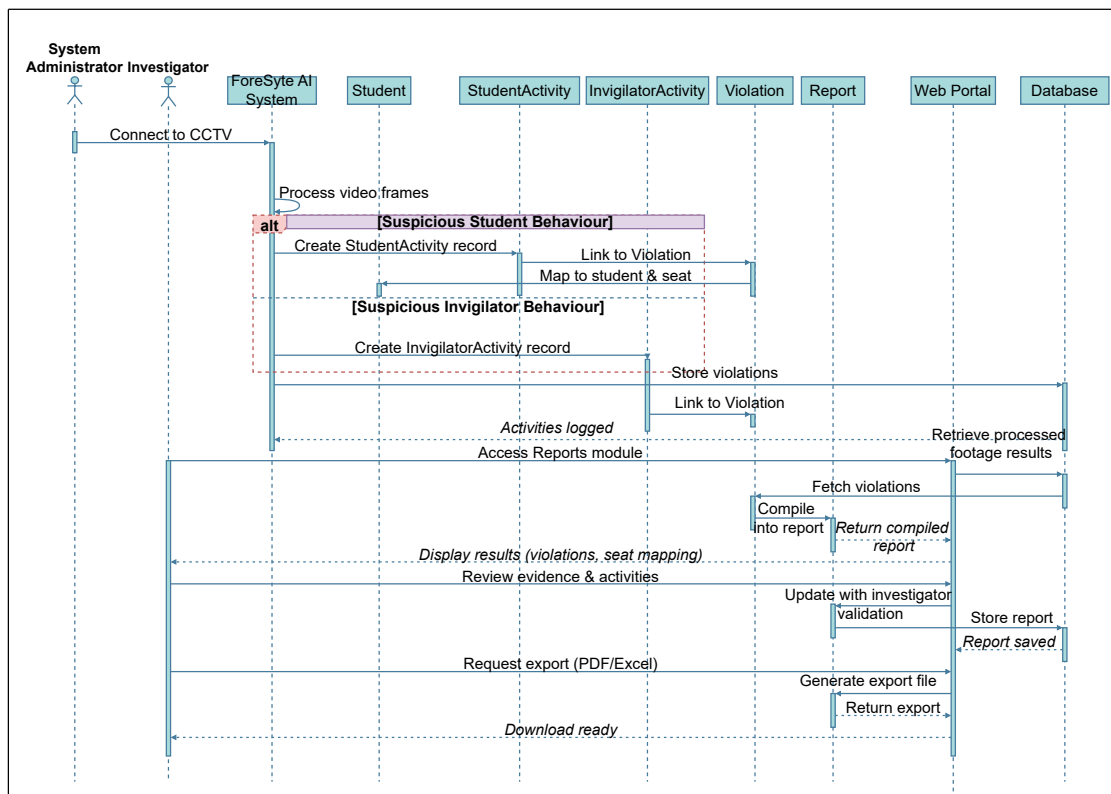


Figure 3.15: Sequence Diagram for UC-07: Process Exam Footage (Live/Recorded)

3.4.5 State Transition Diagram

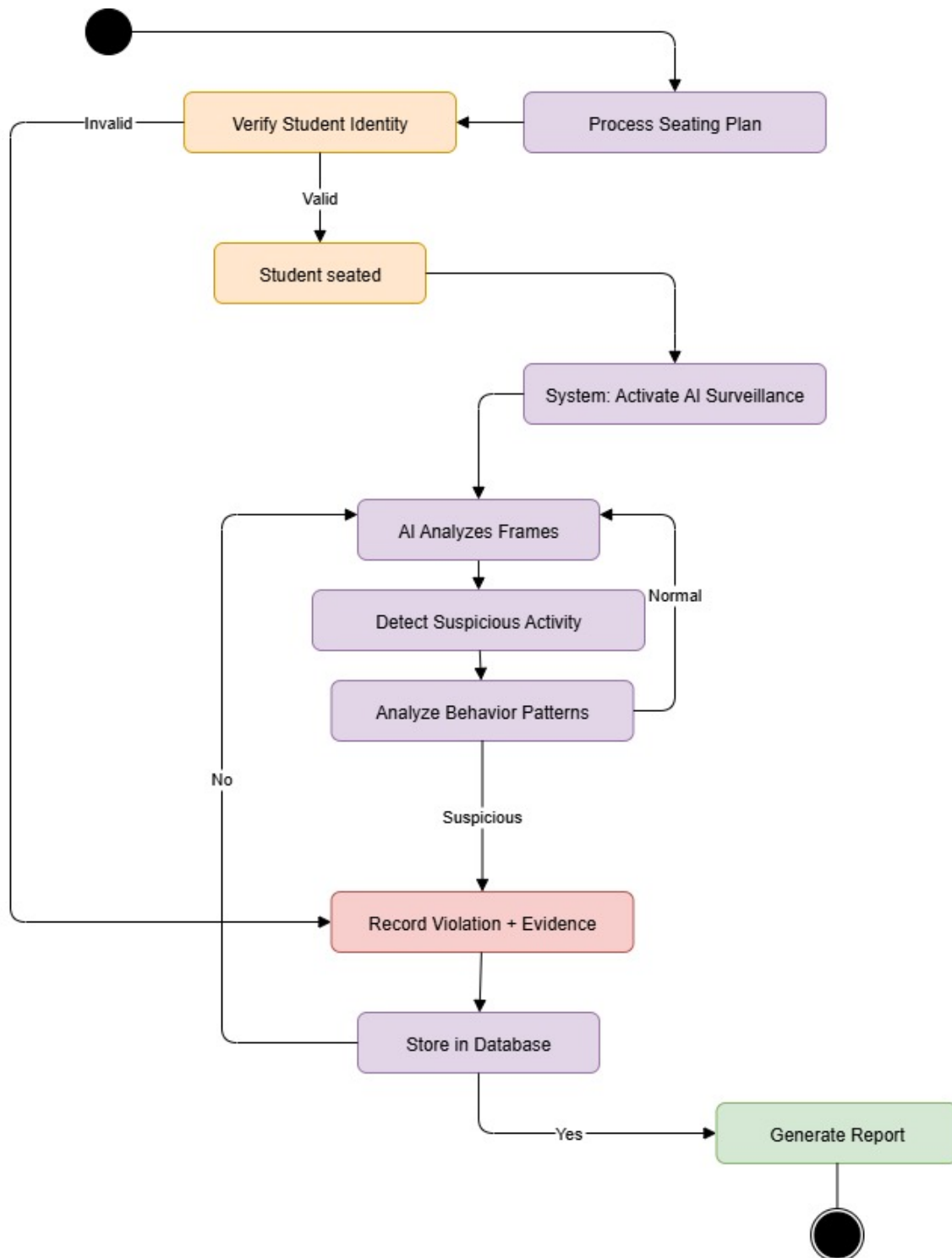


Figure 3.16: State Transition Diagram

This model collectively provide a comprehensive view of the ForeSyte system's architecture, ensuring clear understanding of system components, their interactions, and the

overall system behavior throughout the examination monitoring process.

Chapter 4

Implementation and Testing

This chapter describes the current implementation status of the **ForeSyte** AI Exam Surveillance System and the testing performed up to this iteration. The focus of this phase is on the **Admin** and **Investigator** dashboards and their supporting backend services. Computer-vision based monitoring and the Student Portal are planned components and therefore remain out of scope for this iteration.

4.1 Current Progress

4.1.1 Front-End

The front-end is implemented as a React-based single-page application with TypeScript, Vite, and Tailwind CSS, providing role-specific dashboards for administrators and investigators.

Admin Dashboard

The admin dashboard displays key metrics (active exams, incidents detected, students monitored, average exam duration) fetched from `/dashboard/stats`, along with activity charts, recent incidents, quick actions, and system status indicators. All components are fully integrated with backend APIs.

Investigator Portal

The Investigator Portal provides a dedicated interface with dashboard views showing violation summaries and recent activities, plus a reports page for generating and managing incident and exam reports. Currently uses mock data for validation while backend endpoints are finalized.

4.1.2 Back-End

The backend uses FastAPI with PostgreSQL and MongoDB, implementing JWT-based authentication with RBAC. Key modules include the Dashboard API for aggregating statistics and the Reports API for managing report records.

Dashboard API

The `/dashboard/stats` endpoint computes active exams, incidents detected, students monitored, average exam duration, and trends. The `/dashboard/activity` endpoint provides time-bucketed data, while `/dashboard/recent-incidents` returns the latest flagged activities. All endpoints enforce RBAC policies.

Reports API

The `/reports` router provides CRUD operations for report metadata and endpoints for generating incident and exam reports. Note: actual file generation (PDF/CSV/Excel) is planned for future iterations; current endpoints create database records only.

4.2 External APIs/SDKs

This section describes the third-party APIs/SDKs used in the **ForeSyte** project implementation.

4.2.1 External APIs and SDKs Table

Table 4.1: External APIs and SDKs for ForeSyte

API and version	Description	Purpose of usage	API endpoint/function/class used
Google OAuth 2.0 (via Authlib 1.6.5)	Google authentication and authorization service	User authentication using Google accounts for single sign-on (SSO)	<code>https://accounts.google.com/.well-known/openid-configuration</code> , <code>oauth.register()</code> , <code>google.authorize_redirect()</code> , <code>google.authorize_access_token()</code>
OpenCV (cv2) 4.12.0.88	Computer vision and image processing library	Video stream processing, frame extraction from live CCTV feeds and recorded videos, image analysis for behavior detection	<code>cv2.VideoCapture()</code> , <code>cv2.imread()</code> , <code>cv2.CAP_PROP_FRAME_COUNT</code> , <code>cv2.CAP_PROP_BUFFERSIZE</code>
PDFplumber 0.11.7	PDF text and table extraction library	Parsing seating plan PDFs to extract student information, roll numbers, and exam schedules	<code>pdfplumber.open()</code> , <code>page.extract_tables()</code> , <code>page.extract_text()</code>
Requests 2.32.5	HTTP library for making API calls	Fetching video streams from phone cameras, proxying video streams, testing connection endpoints	<code>requests.get()</code> , <code>requests.exceptions.Timeout</code> , <code>requests.exceptions.ConnectionError</code>
Python-JOSE (jose) 3.5.0	JSON Web Token (JWT) implementation	Creating and validating JWT access tokens for user authentication and authorization	<code>jwt.encode()</code> , <code>jwt.decode()</code> , <code>JWTError</code>
Pillow (PIL) 11.3.0	Python Imaging Library	Image processing and manipulation for video frames and uploaded images	Image processing utilities (used indirectly through OpenCV)
NumPy 2.2.6	Numerical computing library	Array operations and mathematical computations for image/video processing	<code>numpy</code> arrays for image data manipulation

4.2.2 Detailed Usage

Google OAuth 2.0

- **Location:** `src/database/auth.py`
- **Implementation:** Uses Authlib library to integrate with Google's OAuth 2.0 service
- **Endpoints Used:**

- `https://accounts.google.com/.well-known/openid-configuration`
 - OpenID Connect discovery endpoint
- `/auth/google` – Initiates Google OAuth flow
- `/auth/google/callback` – Handles OAuth callback and user authentication

- **Functions:**

- `oauth.register()` – Registers Google OAuth client
- `google.authorize_redirect()` – Redirects user to Google login
- `google.authorize_access_token()` – Exchanges authorization code for access token

OpenCV (cv2)

- **Location:**

- `src/app/video_processing/stream_handler.py`
- `src/app/video_processing/processor.py`
- `src/app/live_phone_feeds/phone_stream_receiver.py`

- **Purpose:**

- Capturing video frames from live streams and recorded videos
- Processing video streams for behavior analysis
- Frame extraction for AI-based detection

- **Key Functions:**

- `cv2.VideoCapture()` – Opens video file or stream URL
- `cap.read()` – Reads frames from video
- `cv2.imread()` – Loads image frames for processing
- `cap.get(cv2.CAP_PROP_FRAME_COUNT)` – Gets total frame count

PDFplumber

- **Location:**

- `src/app/seating_plan/upload_plan.py`
- `import_students_from_pdf.py`

- **Purpose:**

- Extracting student information from seating plan PDFs
- Parsing exam schedules and room assignments
- Extracting roll numbers and student names from PDF tables
- **Key Functions:**
 - `pdfplumber.open()` – Opens PDF file
 - `page.extract_tables()` – Extracts table data from PDF pages
 - `page.extract_text()` – Extracts text content from PDF pages

Requests

- **Location:**
 - `src/database/api/stream_proxy.py`
 - `src/app/live_phone_feeds/phone_stream_receiver.py`
 - `src/app/live_phone_feeds/connection_tester.py`
- **Purpose:**
 - Making HTTP GET requests to fetch video streams
 - Proxying video streams to frontend
 - Testing connectivity to phone camera endpoints
- **Key Functions:**
 - `requests.get()` – Makes HTTP GET requests
 - `requests.exceptions.Timeout` – Handles timeout errors
 - `requests.exceptions.ConnectionError` – Handles connection errors

Python-JOSE

- **Location:** `src/database/auth.py`
- **Purpose:**
 - Encoding JWT tokens for authenticated users
 - Decoding and validating JWT tokens in protected routes
 - Managing user session tokens
- **Key Functions:**
 - `jwt.encode()` – Creates JWT access tokens
 - `jwt.decode()` – Validates and decodes JWT tokens
 - `JWTError` – Exception handling for invalid tokens

Pillow (PIL)

- **Location:** Used indirectly through OpenCV and other image processing operations
- **Purpose:** Image manipulation and processing utilities

NumPy

- **Location:** Used in video processing modules
- **Purpose:** Array operations for image data and mathematical computations in computer vision tasks

4.3 Algorithm Design

This section summarizes the main algorithms and data flows used to support the implemented features in ForeSyte.

4.3.1 Admin Dashboard Aggregation

The Admin Dashboard relies on server-side aggregation logic to compute statistics for different time windows. The high-level algorithm for computing dashboard statistics is:

1. **Determine Time Window:** Given $\text{period} \in \{\text{today}, \text{week}, \text{month}\}$, compute the current window (start_date) and the immediately preceding window (prev_start , prev_end) for trend comparison.
2. **Count Active Exams:** Query Exam where exam_date is greater than or equal to the current date. Perform the same query on the previous window to obtain prev_active_exams .
3. **Count Incidents and Students:**
 - Incidents: count `StudentActivity` records with timestamps within the current window.
 - Previous incidents: count `StudentActivity` records within the previous window.
 - Students monitored: count distinct `Seat.student_id` in rooms and exams that fall inside the current and previous windows.
4. **Compute Average Exam Duration:** For all exams in the current window with non-null start and end times, compute the duration in minutes and average the result.

5. **Compute Trends:** For each metric (incidents, students, exams), compute percentage change between current and previous counts, handling the edge case when the previous value is zero.
6. **Return Structured Response:** Package all calculated values into a `DashboardStats` object and return it to the client.

On the client side, the React dashboard:

- Calls `/dashboard/stats` to populate the metric cards.
- Calls `/dashboard/activity` to render a bar chart comparing incidents and exams per time bucket.
- Calls `/dashboard/recent-incidents` to display the latest suspicious activities with relative time labels.

4.3.2 Report Generation Workflow (Planned Design)

The report generation workflow is designed to be configurable and format-agnostic. **Note:** The following describes the planned design; actual file generation (PDF, CSV, Excel) is not yet implemented in this iteration.

Incident Reports (Planned)

1. The investigator selects one or more incident IDs and chooses an export format (e.g., PDF, CSV, JSON), with options such as including video evidence links.
2. The front-end constructs an `IncidentReportRequest` and sends it to `POST /reports/incidents`.
3. The backend validates permissions, resolves the incident IDs into `StudentActivity` records, ensures there is an associated `Violation`, and generates a unique report filename.
4. A new `Report` record is inserted in the database, and the API returns the report ID, file path, format, and status to the client. **In the current iteration, the actual report file is not generated; only the database record is created.**

Exam Reports (Planned)

1. The investigator selects an exam and desired output format from the Reports Page.
2. The front-end sends an `ExamReportRequest` to `POST /reports/exams/{exam_id}`.

3. The backend verifies the exam, aggregates its related `StudentActivity` records, ensures a linked `Violation` exists, and creates a `Report` entry in the database.
4. The API responds with the report metadata. **The actual report file generation (PDF/CSV/Excel) will be implemented in a future iteration.**

4.3.3 Investigator Violation Review Flow

The Investigator Dashboard is designed to give investigators a structured way to triage violations:

1. Load dashboard summary counters (total violations, pending reviews, resolved today, active exams).
2. Display a list of recent activities with type, severity, and status tags.
3. Allow investigators to toggle the period filter to focus on today, week, or month.
4. Provide a *Review* action for each activity which, in the next iteration, will navigate to a detailed case view and allow updating violation status and adding investigation notes.

4.3.4 Seating Plan Upload and Processing

The seating plan upload algorithm processes PDF files containing exam schedules and extracts room, student, and seat assignment information. The algorithm handles PDF parsing, data extraction, database operations, and visualization generation. The high-level pseudocode for this process is:

```

1  START timer

    READ uploaded PDF bytes
    EXTRACT full text from all pages
    NORMALIZE text (remove extra whitespace)

6
    DEFINE block_pattern to detect each exam block inside PDF
    FIND all exam blocks using regex

    CREATE empty list: matching_exams

11
    FOR each exam block:
        EXTRACT exam date
        EXTRACT exam time slot

```

```

    EXTRACT room number
16 IF failed -> continue

    CHECK if user-specified room_no matches (if provided)
    CHECK if user-specified time_slot matches (if provided)
    IF mismatch -> continue

21 EXTRACT course name + section
    EXTRACT invigilator name

    FIND all students using regex:
26     serial_no, roll_no, name, seat_no

    ADD exam entry to matching_exams list:
        date, time, course, section, room, invigilator,
        student_list, total_students, uploaded_at

31 IF matching_exams is empty:
    RETURN "No seating plan found"

    SELECT first exam -> selected_exam
36 STORE selected_exam in global latest_room_data

    PARSE exam_date, start_time, end_time using parse_date_time()

    -----
41 DATABASE OPERATIONS
    -----

    # --- EXAM ---
    QUERY Exam table:
46     WHERE course == selected_exam.course
        AND exam_date == exam_date
        AND start_time == start_time

    IF exam exists:
51     use existing
    ELSE:
        CREATE new Exam object
        COMMIT to DB

56 # --- ROOM ---
    Split room number into block + room_num
    QUERY Room table:
        WHERE room_number == room_num AND exam_id == exam.exam_id

61 IF room exists:
    use existing

```

```

ELSE:
    CREATE Room with:
        room_number, block, total_seats, exam_id, camera_id
66 COMMIT to DB

# --- STUDENTS + SEATS ---
FOR each student in selected_exam.students:
    roll_number = student.roll_no
71 name = student.name

    QUERY Student table WHERE roll_number matches
    IF not exists:
        CREATE new Student(name, email_from_roll, roll_number)
76 COMMIT

    seat_number = student.seat_no

    QUERY Seat table WHERE (room_id, seat_number)
81 IF exists:
        UPDATE seat.student_id
    ELSE:
        CREATE new Seat(room_id, seat_number, student_id)

86 COMMIT all changes

-----
SAVE JSON EXPORT
-----

91 GENERATE timestamp-based filename
SAVE selected_exam as JSON file in extractions folder

-----
96 VISUALIZATION PROCESS
-----

LOAD CCTV frame image
LOAD seat_map.json (polygon coordinates)
101
CALCULATE max_column in seating plan:
    For each student seat (C#R#)
        Extract column number
        Track the maximum found
106
DEFINE mapping rules based on max_column:
    if max_col == 6 -> map 6 columns to 1,3,5,6,8,10
    if max_col == 5 -> map 5 columns to 1,4,6,8,10
    else -> identity mapping

```

```

111  DEFINE function map_seat_to_seat_map(seat_no):
      Extract C#, R#
      Convert input col -> mapped column
      RETURN seat_map ID like seat_cXrY
116
      CREATE:
          filled_seats = all seat IDs containing students
          student_seat_map = map seat_id -> student

121  FOR each seat_id in seat_map:
      IF seat_id NOT in filled_seats -> skip

      GET polygon points
      DRAW polygon in green

126
      FIND student assigned
      CALCULATE centroid
      DRAW student name + roll number at centroid

131  SAVE annotated image in extractions folder

      -----
      RETURN RESPONSE
      -----

136
      RETURN:
          message
          exam_date
          exam_time
          course
141          room_no
          students_count
          exam_id
          room_id
146          json_file path
          annotated_image path
          processing_time

```

The algorithm processes seating plan PDFs through several stages: (1) **PDF Parsing and Text Extraction**, where the uploaded PDF is read and normalized; (2) **Pattern Matching**, where exam blocks are identified using regex patterns and filtered by user-specified room number and time slot; (3) **Data Extraction**, where course information, invigilator details, and student seat assignments are extracted; (4) **Database Operations**, where Exam, Room, Student, and Seat records are created or updated; (5) **JSON Export**, where the extracted data is saved as a JSON file; and (6) **Visualization**, where the seating plan is

overlaid on a CCTV frame image with student names and roll numbers annotated at their respective seat positions.

4.4 Testing Details

Testing for this iteration concentrates on validating backend APIs, role-based access control, and the correctness of aggregated statistics, along with front-end behaviour under typical and edge-case conditions. The testing approach employs a combination of unit tests for individual components, integration tests for API endpoints, and end-to-end validation of user workflows. Test cases cover functional requirements, error handling, data validation, and user interface interactions across all three portals (Admin, Investigator, and Student). The test suite ensures that role-based access controls are properly enforced, data aggregation logic produces accurate results, and the user interface responds correctly to various input scenarios and edge cases.

4.4.1 Unit Testing

Investigator Portal Testing

The Investigator Portal testing suite validates the core functionality required for investigators to effectively manage violations and generate reports. All test cases passed successfully, confirming that the dashboard correctly displays statistics, violations can be filtered and reviewed, activities are properly categorized, and report generation workflows function as expected. The error handling tests ensure the portal gracefully manages API failures without crashing, maintaining a stable user experience even under adverse conditions.

Table 4.2: Unit Testing for Investigator Portal

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC001	Verify dashboard renders header and sidebar correctly	Valid token exists in sessionStorage	1. Insert token in storage. 2. Mock Stats, Violations, Incidents APIs. 3. Render Investigator Dashboard	Token: "test-token"	Header and sidebar appear on screen	As Expected	Pass
TC002	Dashboard statistics load correctly	API returns valid stats + violations	1. Mock Stats API: active_exams=3. 2. Mock Violations API: 2 violations. 3. Render component	Stats: {active_exams: 3}. 2 violations	UI shows: Active Exams = 3, Total Violations = 2, Pending Reviews ≥ 1	As Expected	Pass
TC003	Dashboard displays recent activities correctly	API returns incident data	1. Mock Incidents API with type='Cheating'. 2. Render dashboard. 3. Verify activity display	Incident: {type: "Cheating", student: "John Doe", severity: "high"}	Recent activities section shows "Cheating" as h3 heading	As Expected	Pass
TC004	Period filter changes dashboard data	Dashboard loaded with initial data	1. Render dashboard. 2. Click "This Week" button. 3. Verify API call	Initial period: "today"	getDashboardStats called with period='week' parameter	As Expected	Pass
TC005	Violations page renders and displays violations list	Valid token and API returns violations	1. Mock getViolations API. 2. Render ViolationsPage. 3. Verify table display	Violation: {student_name: "John Doe", type: "cheating", status: "pending"}	Student name and violation type appear in table row	As Expected	Pass
TC006	Violations can be filtered by search term	Multiple violations loaded	1. Render page with 2 violations. 2. Enter "Jane" in search input. 3. Verify filtered results	Students: "John Doe", "Jane Smith"	Only "Jane Smith" violation visible, "John Doe" hidden	As Expected	Pass
TC007	Violations can be filtered by status	Violations with different statuses loaded	1. Render page with pending and confirmed violations. 2. Select "pending" in status filter. 3. Verify filtered results	Statuses: "pending", "confirmed"	Only pending violation visible, confirmed violation hidden	As Expected	Pass
TC008	Violation details modal opens and confirms violation	Violation exists in system	1. Render ViolationsPage. 2. Click "View" button. 3. Click "Confirm" in modal. 4. Verify API call	Violation ID: "v1"	Modal opens with "Violation Details" heading; confirmViolation API called with ID "v1"	As Expected	Pass

Table 4.3: Unit Testing for Investigator Portal (continued)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC009	Invigilator activities page displays and filters by type	Multiple activity types exist	1. Mock activities API with phone_usage and absent_from_zone. 2. Render page. 3. Select "phone_usage" filter. 4. Verify display	Activities: phone_usage, absent_from_zone	phone_usage appears in table cell (td element) after filtering	As Expected	Pass
TC010	Student activities page displays and filters by search	Multiple student activities loaded	1. Mock API with activities for John and Jane. 2. Render page. 3. Search for "Jane". 4. Verify results	Students: "John Doe", "Jane Smith"	Only Jane's activities visible; phone_usage type shown in table cell	As Expected	Pass
TC011	Reports page opens generate modal and displays reports list	Valid token and reports exist	1. Mock getReports API. 2. Render ReportsPage. 3. Click "Generate New Report" button. 4. Verify modal	Report: {type: "incident", exam: "Calculus I"}	Report displayed; modal opens with "Generate New Report" heading	As Expected	Pass
TC012	All pages handle API errors gracefully	API returns network error	1. Mock API to reject with error. 2. Render any investigator page. 3. Verify error handling	Error: "Network error"	Page renders with header visible; no crash or unhandled exception	As Expected	Pass

Student Portal Testing

The Student Portal testing validates the student-facing features including profile management, disciplinary records viewing, and document uploads. All test cases passed, demonstrating that students can successfully view their violation history, update profile information, change passwords with proper validation, and manage uploaded documents. The portal correctly handles authentication, displays personalized welcome messages, and provides an intuitive interface for students to access their exam-related information and disciplinary records.

Table 4.4: Unit Testing for Student Portal

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC001	Verify dashboard renders header and sidebar correctly	Valid token exists in sessionStorage	1. Insert token in storage. 2. Mock Profile, Activities, Violations APIs. 3. Render Student Dashboard	Token: "test-token", User ID: "student-123"	Header and sidebar appear on screen	As Expected	Pass
TC002	Dashboard displays welcome message with student name	API returns valid profile data	1. Mock getStudentProfile API with name="John Doe". 2. Mock Activities and Violations APIs. 3. Render dashboard	Profile: {name: "John Doe", email: "john@test.com", roll_number: "22I-1234"}	UI shows: "Welcome back, John Doe" message	As Expected	Pass
TC003	Dashboard displays disciplinary cases count correctly	API returns violations data	1. Mock Profile API. 2. Mock getStudentViolations with 2 violations. 3. Render dashboard	Violations: [{violation_id: "v1", status: "pending"}, {violation_id: "v2", status: "confirmed"}]	UI shows: Disciplinary Cases = 2	As Expected	Pass
TC004	Dashboard displays recent activities list	API returns activities data	1. Mock Profile API. 2. Mock getStudentActivities-ByStudentId with head_movement activity. 3. Render dashboard	Activities: [{activity_type: "head_movement", severity: "high", timestamp: "2024-01-01T10:00:00Z"}]	Recent activities section shows "head_movement" activity	As Expected	Pass
TC005	Disciplinary records page displays violations list	Valid token and API returns violations	1. Mock getStudentProfile API. 2. Mock getStudentViolations with cheating violation. 3. Render dcRecords page	Violation: {violation_type: "cheating", severity: 3, status: "pending", timestamp: "2024-01-01T10:00:00Z"}	Violation type "cheating" appears as heading (h3 element)	As Expected	Pass
TC006	Disciplinary records can be expanded to view details	Violations loaded on page	1. Render dcRecords page with violations. 2. Find clickable record card. 3. Click on record. 4. Verify details display	Violation: {violation_type: "cheating", status: "pending"}	Record expands; "Details" text appears on screen	As Expected	Pass
TC007	Profile page allows editing student name	Valid profile data loaded	1. Mock getStudentProfile with name="John Doe". 2. Render profile page. 3. Click "Edit" button. 4. Change name to "Jane Doe". 5. Click save button. 6. Verify API call	Original: "John Doe", Updated: "Jane Doe"	updateStudentProfile API called with name="Jane Doe"; change saved successfully	As Expected	Pass

Table 4.5: Unit Testing for Student Portal (continued)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC008	Profile page cancels editing when cancel button clicked	Profile in edit mode with changes	1. Render profile page. 2. Click "Edit" button. 3. Change name to "Jane Doe". 4. Click cancel button. 5. Verify revert	Changed: "Jane Doe", Original: "John Doe"	Input field disappears; original name "John Doe" displayed; updateStudentProfile not called	As Expected	Pass
TC009	Settings page allows password change with validation	Valid profile and matching passwords	1. Mock getStudentProfile API. 2. Render settings page. 3. Switch to Password tab. 4. Enter current="oldpass123", new="newpass123", confirm="newpass123". 5. Click "Update Password"	Passwords: {current: "oldpass123", new: "newpass123", confirm: "newpass123"}	changePassword API called with current_password="oldpass123" and new_password="newpass123"	As Expected	Pass
TC010	Upload page displays uploaded documents and allows file removal	Documents exist in system	1. Mock getStudentDocuments with id_card.pdf. 2. Render upload page. 3. Find remove button. 4. Click remove. 5. Verify removal	Document: {filename: "id_card.pdf", size: 1024000, status: "verified"}	Document "id_card.pdf" appears initially; after clicking remove button, document no longer visible	As Expected	Pass

Admin Portal Testing

The Admin Portal testing comprehensively validates all administrative functions including dashboard analytics, user management, exam scheduling, incident monitoring, report generation, and system settings. All test cases passed successfully, confirming that administrators can effectively manage the system, view comprehensive analytics, create and manage exams, monitor camera feeds, generate reports, and configure system settings. The authentication and authorization tests ensure that only authorized admin users can access administrative functions, maintaining proper security boundaries throughout the system.

Table 4.6: Unit Testing for Admin Portal

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC001	Verify dashboard renders header and sidebar correctly	Valid token exists in sessionStorage	1. Insert token in storage. 2. Mock Stats, Activity, Incidents APIs. 3. Render Admin Dashboard	Token: "test-token", User type: "admin"	Header and sidebar appear on screen	As Expected	Pass
TC002	Dashboard displays welcome message and statistics	API returns valid stats data	1. Mock getDashboardStats with active_exams=5. 2. Mock other APIs. 3. Render dashboard	Stats: {active_exams: 5, incidents_detected: 10, students_monitored: 150}	UI shows: "Welcome back" message, Active Exams = 5, Incidents = 10	As Expected	Pass
TC003	Dashboard displays recent incidents list	API returns incidents data	1. Mock getRecentIncidents with "Suspicious Movement" incident. 2. Render dashboard. 3. Verify display	Incident: {student_name: "John Doe", type: "Suspicious Movement", severity: "high"}	Recent incidents section shows "John Doe" name	As Expected	Pass
TC004	Analytics page fetches and displays metrics correctly	Valid token and API returns metrics	1. Mock getAnalyticsMetrics with total_incidents=128. 2. Mock other analytics APIs. 3. Render AnalyticsPage	Metrics: {total_incidents: 128, resolution_rate: 87.5, avg_response_time: 4.2}	UI shows: 128 incidents, 87.5% resolution rate	As Expected	Pass
TC005	Analytics page displays incident type distribution	API returns distribution data	1. Mock getIncidentTypeDistribution with types. 2. Render AnalyticsPage. 3. Verify display	Types: [{type: "Suspicious Movement", count: 45}, {type: "Device Detected", count: 32}]	"Suspicious Movement" and "Device Detected" appear on screen	As Expected	Pass
TC006	Exams page renders and displays exams list	Valid token and API returns exams	1. Mock getExams with Calculus I and Data Structures. 2. Render ExamsPage. 3. Verify display	Exams: [{name: "Calculus I", course_code: "MATH101"}, {name: "Data Structures", course_code: "CS201"}]	"Calculus I" and "Data Structures" appear on screen	As Expected	Pass
TC007	Exams page opens create modal when add button clicked	Exams page loaded	1. Render ExamsPage. 2. Click "Create Exam" button. 3. Verify modal display	Initial exams: []	Modal opens with "Create New Exam" heading	As Expected	Pass

Table 4.7: Unit Testing for Admin Portal (continued)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC008	Exams page filters by status correctly	Multiple exams with different statuses loaded	1. Render ExamsPage. 2. Find status filter select. 3. Change to "active". 4. Verify API call	Statuses: "scheduled", "active", "completed"	getExams API called with status='active' parameter	As Expected	Pass
TC009	Login page handles successful authentication	Valid credentials provided	1. Mock fetch with success response. 2. Render LoginPage. 3. Enter email="admin@test.com" and password="password123". 4. Click submit button	Credentials: {email: "admin@test.com", password: "password123"}. Response: {access_token: "test-token", user_type: "admin"}	Login API called; token stored in sessionStorage; navigate to "/dashboard"	As Expected	Pass
TC010	Login page validates input and shows/hides password	Login form rendered	1. Render LoginPage. 2. Verify password input type is "password". 3. Click toggle button. 4. Verify type changes	Initial type: "password"	Password input type changes to "text" when toggle clicked, then back to "password"	As Expected	Pass
TC011	SignUp page validates password requirements	SignUp form rendered	1. Render SignUpPage. 2. Enter password="12345". 3. Enter confirm="12345". 4. Click submit. 5. Verify validation	Passwords: {password: "12345", confirm: "12345"}	Alert shown: "Password must be at least 6 characters long!"	As Expected	Pass
TC012	Users page fetches and displays users list	Valid token and API returns users	1. Mock getUsers with John Doe and Jane Smith. 2. Render UsersPage. 3. Verify display	Users: [{name: "John Doe", email: "john@test.com", user_type: "student"}, {name: "Jane Smith", email: "jane@test.com", user_type: "admin"}]	"John Doe" and "Jane Smith" appear on screen	As Expected	Pass
TC013	Users page opens add user modal when button clicked	Users page loaded	1. Mock getUsers API. 2. Render UsersPage. 3. Click "Add User" button. 4. Verify modal	Initial users: []	Modal opens with "Add New User" heading	As Expected	Pass
TC014	Incidents page renders and displays incidents list	Valid token and API returns incidents	1. Mock getIncidents with "Suspicious Movement" incident. 2. Render Incidents page. 3. Verify display	Incident: {student_name: "John Doe", type: "Suspicious Movement", severity: "high", status: "investigating"}	"John Doe" and "Suspicious Movement" appear on screen	As Expected	Pass

Table 4.8: Unit Testing for Admin Portal (continued)

Test Case ID	Test Objective	Pre-condition	Steps	Test Data	Expected Result	Actual Result	Pass/Fail
TC015	Monitoring page fetches and displays camera feeds	Valid token and API returns feeds	1. Mock getMonitoringFeeds with Room A101 and B205. 2. Render MonitoringPage. 3. Verify display	Feeds: [{room_name: "Room A101", status: "active"}, {room_name: "Room B205", status: "inactive"}]	"Room A101" and "Room B205" appear on screen	As Expected	Pass
TC016	Monitoring page opens phone camera monitor when button clicked	Monitoring page loaded	1. Mock getMonitoringFeeds API. 2. Render MonitoringPage. 3. Click "Start Phone Camera" button. 4. Verify modal	Feeds: []	Phone camera monitor component appears with testid "phone-camera-monitor"	As Expected	Pass
TC017	Reports page renders and displays reports list	Valid token and API returns reports	1. Mock getReportHistory with exam and incident reports. 2. Mock getExams and getIncidents. 3. Render ReportsPage	Reports: [{title: "Exam Report - Calculus I", type: "exam"}, {title: "Incident Report", type: "incident"}]	Report headings "Exam Report - Calculus I" and "Incident Report" appear	As Expected	Pass
TC018	Reports page opens generate modal when button clicked	Reports page loaded	1. Mock all reports APIs. 2. Render ReportsPage. 3. Click "Incident Report" button. 4. Verify modal	Reports: []	Modal opens with "Generate Incident Report" heading	As Expected	Pass
TC019	Seating Plans page fetches and displays plans list	Valid token and API returns plans	1. Mock getSeatingPlans with 2 PDF files. 2. Render SeatingPlansPage. 3. Verify display	Plans: [{filename: "seating_plan_-2024.pdf"}, {filename: "seating_plan_-2025.pdf"}]	"seating_plan_-2024.pdf" and "seating_plan_-2025.pdf" appear on screen	As Expected	Pass
TC020	Settings page displays tabs and allows logout	Valid session exists	1. Render SettingsPage. 2. Verify tabs display. 3. Switch to Security tab. 4. Click Logout button. 5. Verify navigation	Tabs: General, Notifications, Security, System	All tabs appear; logout clears session and navigates to "/login"	As Expected	Pass

Bibliography

- [1] Saravanan Arumugam. Deep learning-based smart invigilation system for enhanced exam integrity. *Proceedings of Engineering and Technology Innovation*, 29:99–115, 2025.
- [2] A. Bhatti and T. Aslam. Smart exam monitoring system using computer vision. *Journal of Emerging Technologies*, 2023.
- [3] A. Buccioli, S. Cicognani, and N. Montinari. Cheating in university exams: the relevance of social factors. *ResearchGate*, 2020.
- [4] M. Khalid and H. Shah. Impact of surveillance systems on academic integrity in higher education. *Journal of Educational Research*, 2021.
- [5] Mohammad M. Masud, Khadim Hayawi, Sujith Samuel Mathew, Temesgen Mikael Abraha, and Mai ElBarachi. Smart online exam proctoring assist for cheating detection. *ResearchGate*, 2025.
- [6] Reuben Moyo, Stanley Ndebvu, Michael Zimba, and Jimmy Mbelwa. A video-based detector for suspicious activity in examination with openpose. *Unknown Journal*, 2023.
- [7] –. Intelligent offline exam monitoring system (iems) for identifying suspicious behavior. *International Journal of Computer Applications*, 2025.

Appendix A

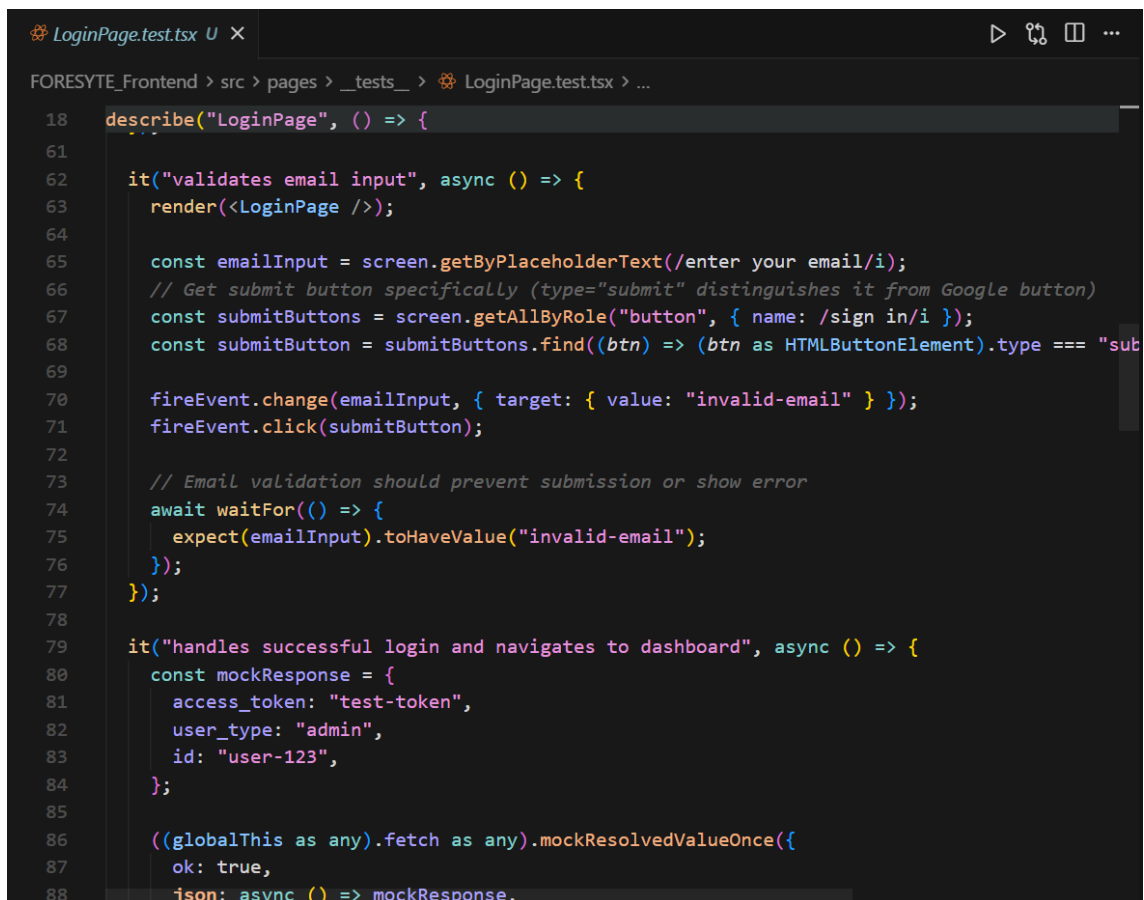
Appendices

A.1 Appendix A

A.1.1 Admin Portal Screenshots

A.1.1.1 Login Page

Figure A.1: Admin Portal Login Page

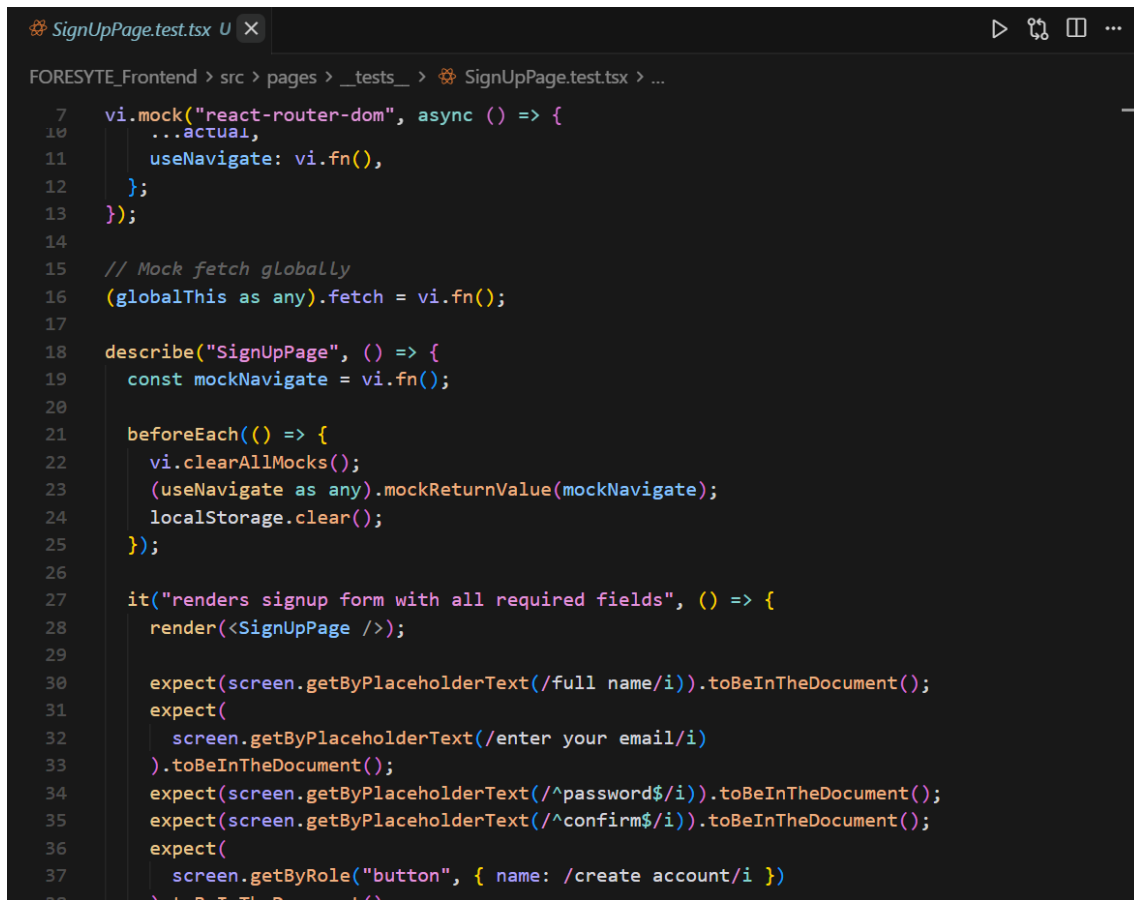


```
LoginPage.test.tsx U X
FORESYTE_Frontend > src > pages > _tests_ > LoginPage.test.tsx > ...

18 describe("LoginPage", () => {
61
62   it("validates email input", async () => {
63     render(<LoginPage />);
64
65     const emailInput = screen.getByPlaceholderText(/enter your email/i);
66     // Get submit button specifically (type="submit" distinguishes it from Google button)
67     const submitButtons = screen.getAllByRole("button", { name: /sign in/i });
68     const submitButton = submitButtons.find((btn) => (btn as HTMLButtonElement).type === "submit");
69
70     fireEvent.change(emailInput, { target: { value: "invalid-email" } });
71     fireEvent.click(submitButton);
72
73     // Email validation should prevent submission or show error
74     await waitFor(() => {
75       expect(emailInput).toHaveValue("invalid-email");
76     });
77   });
78
79   it("handles successful login and navigates to dashboard", async () => {
80     const mockResponse = {
81       access_token: "test-token",
82       user_type: "admin",
83       id: "user-123",
84     };
85
86     ((globalThis as any).fetch as any).mockResolvedValueOnce({
87       ok: true,
88       json: async () => mockResponse,
```

A.1.1.2 Sign Up Page

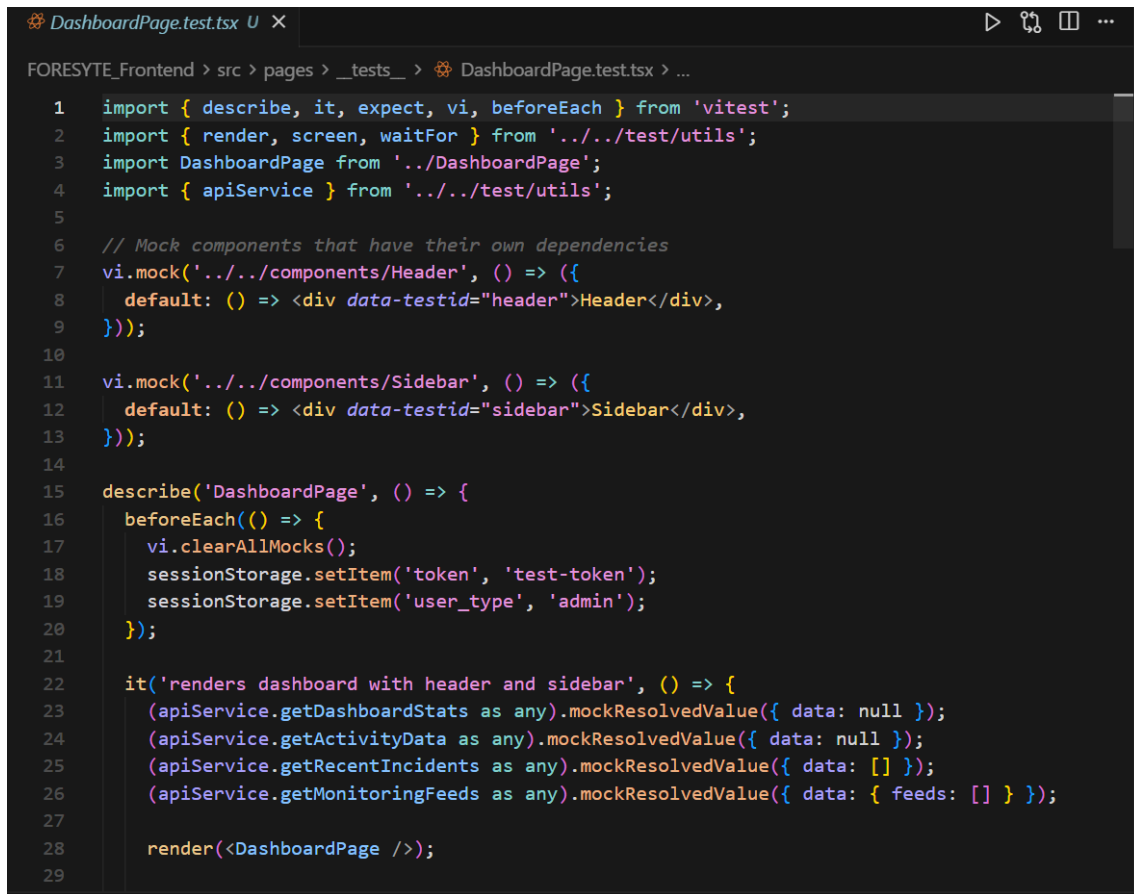
Figure A.2: Admin Portal Sign Up Page



```
7 vi.mock("react-router-dom", async () => {
10   ...actual,
11   useNavigate: vi.fn(),
12 });
13 });
14
15 // Mock fetch globally
16 (globalThis as any).fetch = vi.fn();
17
18 describe("SignUpPage", () => {
19   const mockNavigate = vi.fn();
20
21   beforeEach(() => {
22     vi.clearAllMocks();
23     (useNavigate as any).mockReturnValue(mockNavigate);
24     localStorage.clear();
25   });
26
27   it("renders signup form with all required fields", () => {
28     render(<SignUpPage />);
29
30     expect(screen.getByPlaceholderText(/full name/i)).toBeInTheDocument();
31     expect(
32       screen.getByPlaceholderText(/enter your email/i)
33     ).toBeInTheDocument();
34     expect(screen.getByPlaceholderText(/password$/i)).toBeInTheDocument();
35     expect(screen.getByPlaceholderText(/confirm$/i)).toBeInTheDocument();
36     expect(
37       screen.getByRole("button", { name: /create account/i })
38     ).toBeInTheDocument();
39   });
40 });
```

A.1.1.3 Dashboard

Figure A.3: Admin Portal Dashboard

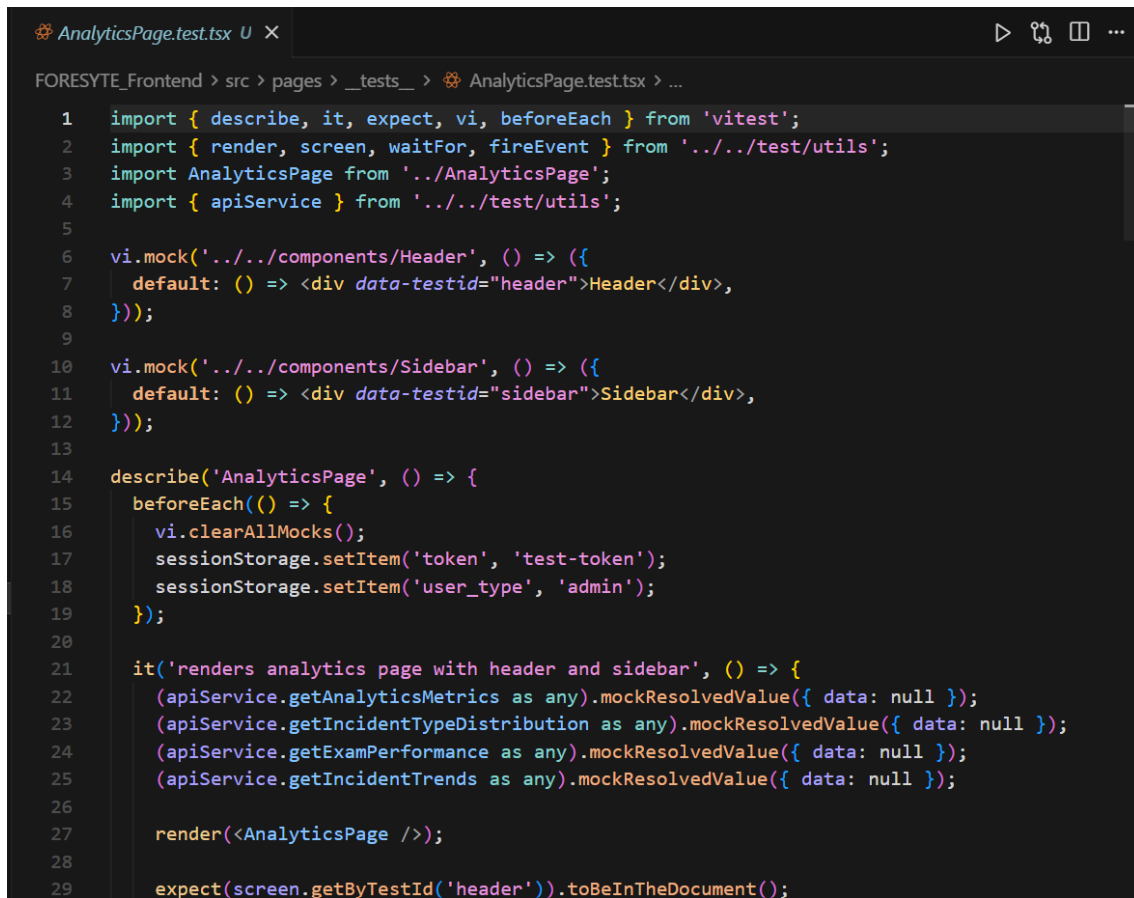


```
DashboardPage.test.tsx X
FORESYTE_Frontend > src > pages > __tests__ > DashboardPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor } from '../test/utils';
3  import DashboardPage from '../DashboardPage';
4  import { apiService } from '../test/utils';
5
6  // Mock components that have their own dependencies
7  vi.mock('../components/Header', () => ({
8    default: () => <div data-testid="header">Header</div>,
9  }));
10
11 vi.mock('../components/Sidebar', () => ({
12   default: () => <div data-testid="sidebar">Sidebar</div>,
13 });
14
15 describe('DashboardPage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.setItem('token', 'test-token');
19     sessionStorage.setItem('user_type', 'admin');
20   });
21
22   it('renders dashboard with header and sidebar', () => {
23     (apiService.getDashboardStats as any).mockResolvedValue({ data: null });
24     (apiService.getActivityData as any).mockResolvedValue({ data: null });
25     (apiService.getRecentIncidents as any).mockResolvedValue({ data: [] });
26     (apiService.getMonitoringFeeds as any).mockResolvedValue({ data: { feeds: [] } });
27
28     render(<DashboardPage />);
29   });
}
```

A.1.1.4 Analytics Page

Figure A.4: Admin Portal Analytics Page

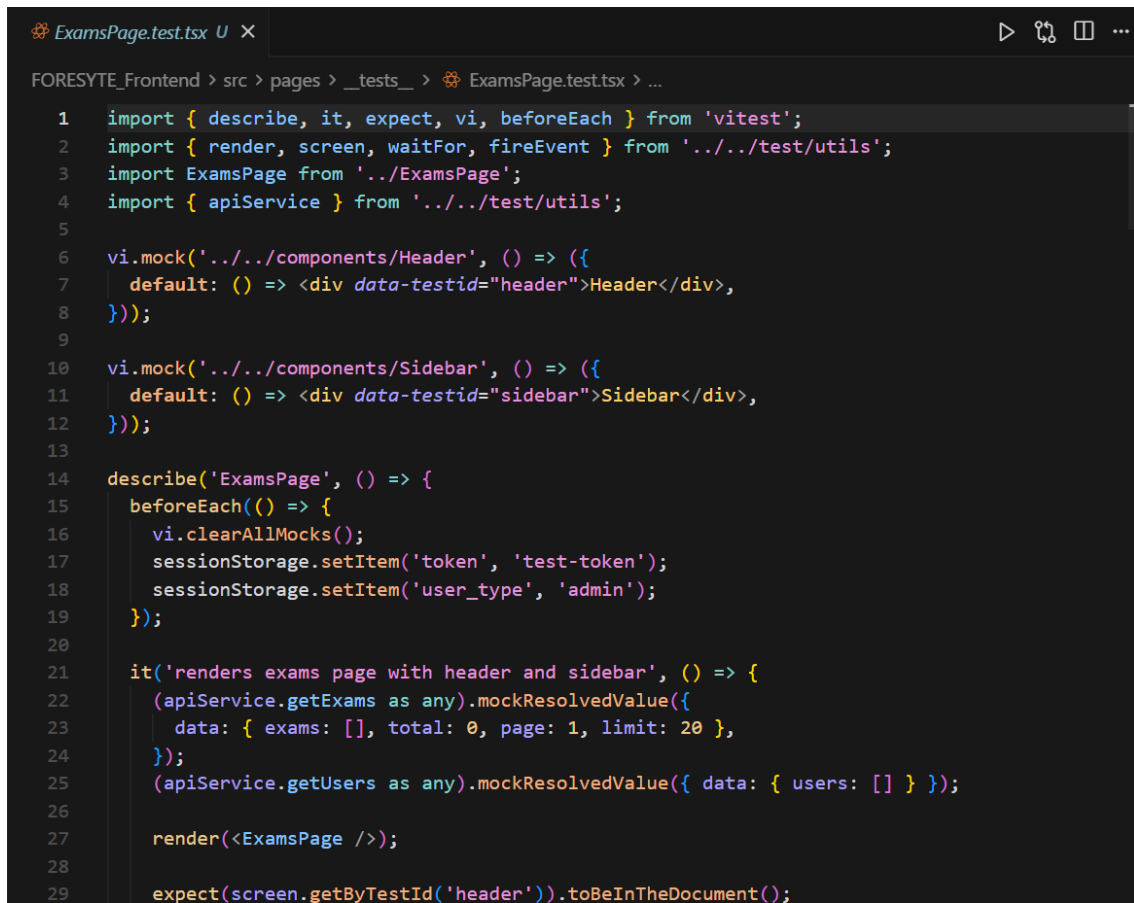


```
AnalyticsPage.test.tsx U X
FORESYTE_Frontend > src > pages > __tests__ > AnalyticsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import AnalyticsPage from '../AnalyticsPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/Header', () => ({
7    default: () => <div data-testid="header">Header</div>,
8  }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 });
13
14 describe('AnalyticsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders analytics page with header and sidebar', () => {
22     (apiService.getAnalyticsMetrics as any).mockResolvedValue({ data: null });
23     (apiService.getIncidentTypeDistribution as any).mockResolvedValue({ data: null });
24     (apiService.getExamPerformance as any).mockResolvedValue({ data: null });
25     (apiService.getIncidentTrends as any).mockResolvedValue({ data: null });
26
27     render(<AnalyticsPage />);
28
29     expect(screen.getByTestId('header')).toBeInTheDocument();
30   });
31 });
```

A.1.1.5 Exams Page

Figure A.5: Admin Portal Exams Management Page

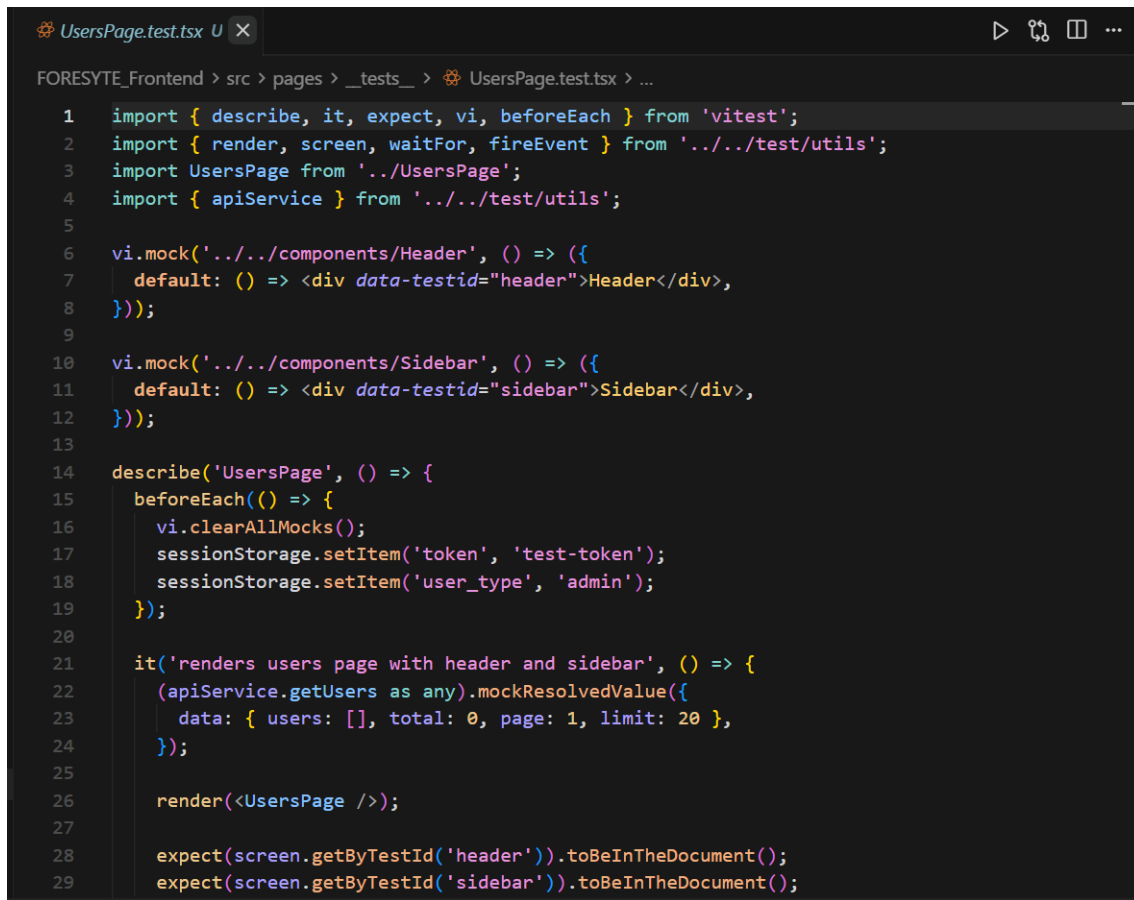


The image shows a code editor window with a file named `ExamsPage.test.tsx`. The breadcrumb navigation indicates the file is located at `FORESYTE_Frontend > src > pages > _tests_ > ExamsPage.test.tsx > ...`. The code is a Jest test file for the `ExamsPage` component. It includes imports for testing utilities from `vitest` and `../test/utills`. It uses `vi.mock` to mock the `Header` and `Sidebar` components. The test suite is defined with `describe('ExamsPage', ...)` and includes a `beforeEach` block to clear mocks and set session storage. The main test is `it('renders exams page with header and sidebar', ...)`, which renders the `ExamsPage` component and expects the header to be in the document.

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../test/utills';
3 import ExamsPage from '../ExamsPage';
4 import { apiService } from '../test/utills';
5
6 vi.mock('../components/Header', () => ({
7   default: () => <div data-testid="header">Header</div>,
8 }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 }));
13
14 describe('ExamsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders exams page with header and sidebar', () => {
22     (apiService.getExams as any).mockResolvedValue({
23       data: { exams: [], total: 0, page: 1, limit: 20 },
24     });
25     (apiService.getUsers as any).mockResolvedValue({ data: { users: [] } });
26
27     render(<ExamsPage />);
28
29     expect(screen.getByTestId('header')).toBeInTheDocument();
```

A.1.1.6 Users Page

Figure A.6: Admin Portal Users Management Page

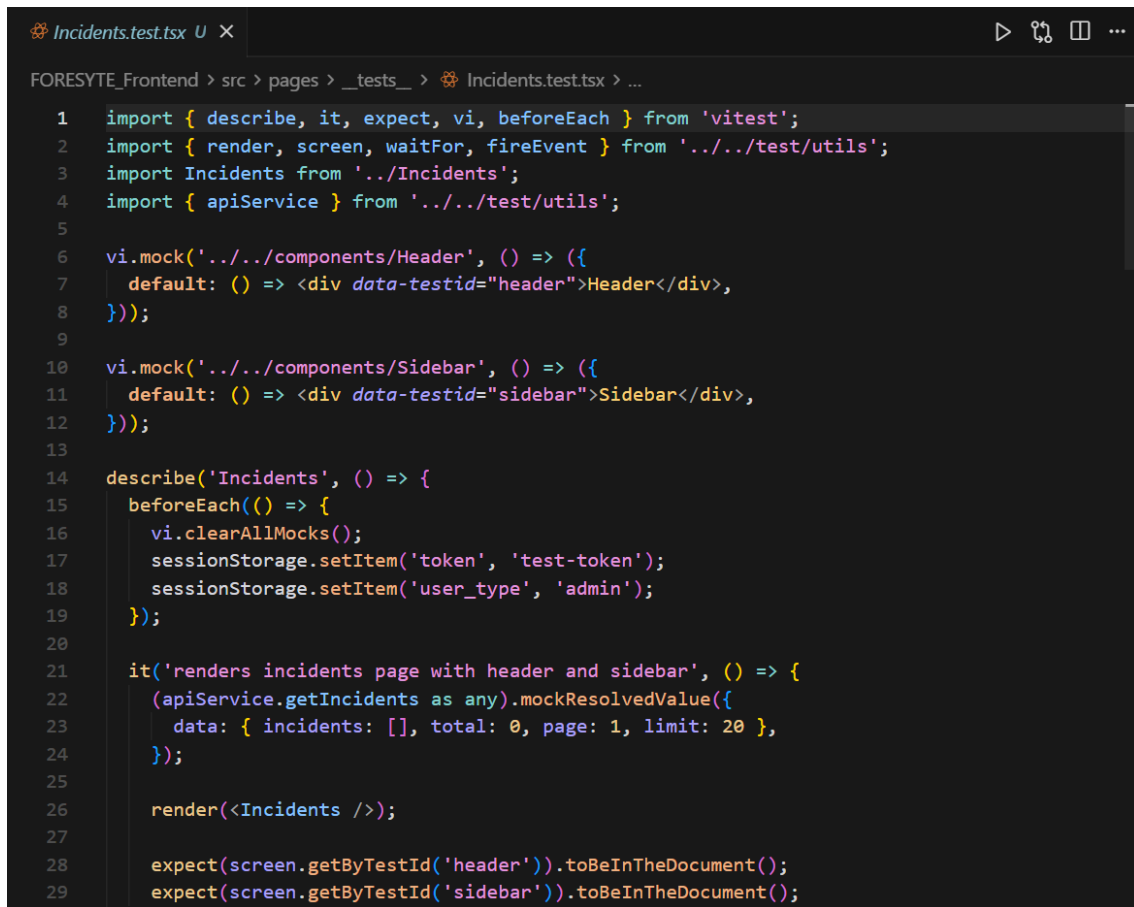


```
UsersPage.test.tsx U X
FORESYTE_Frontend > src > pages > __tests__ > UsersPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import UsersPage from '../UsersPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/Header', () => ({
7    default: () => <div data-testid="header">Header</div>,
8  }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 });
13
14 describe('UsersPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders users page with header and sidebar', () => {
22     (apiService.getUsers as any).mockResolvedValue({
23       data: { users: [], total: 0, page: 1, limit: 20 },
24     });
25
26     render(<UsersPage />);
27
28     expect(screen.getByTestId('header')).toBeInTheDocument();
29     expect(screen.getByTestId('sidebar')).toBeInTheDocument();
30   });
31 });
```

A.1.1.7 Incidents Page

Figure A.7: Admin Portal Incidents Page

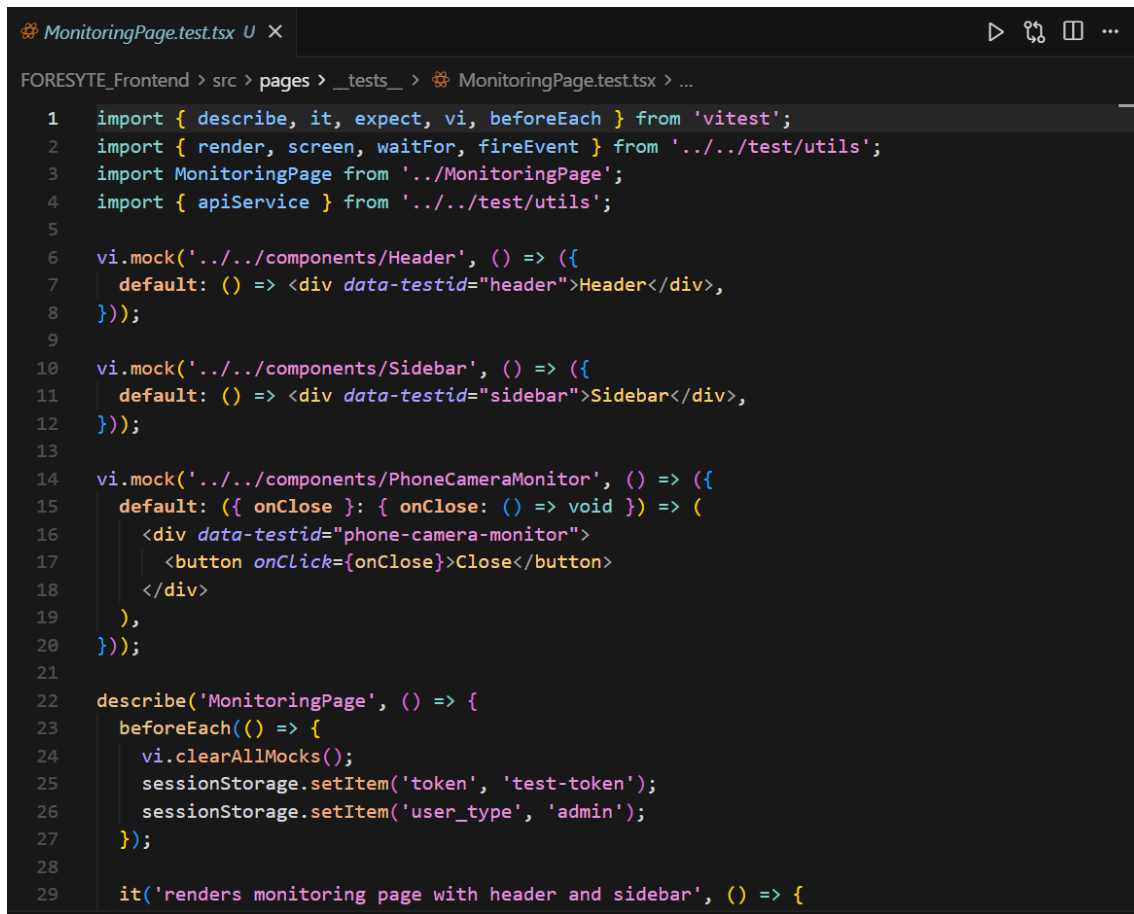


```
Incidents.test.tsx U X
FORESYTE_Frontend > src > pages > _tests_ > Incidents.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import Incidents from '../Incidents';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/Header', () => ({
7    default: () => <div data-testid="header">Header</div>,
8  }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 });
13
14 describe('Incidents', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders incidents page with header and sidebar', () => {
22     (apiService.getIncidents as any).mockResolvedValue({
23       data: { incidents: [], total: 0, page: 1, limit: 20 },
24     });
25
26     render(<Incidents />);
27
28     expect(screen.getByTestId('header')).toBeInTheDocument();
29     expect(screen.getByTestId('sidebar')).toBeInTheDocument();
30   });
31 });
```

A.1.1.8 Monitoring Page

Figure A.8: Admin Portal Monitoring Page

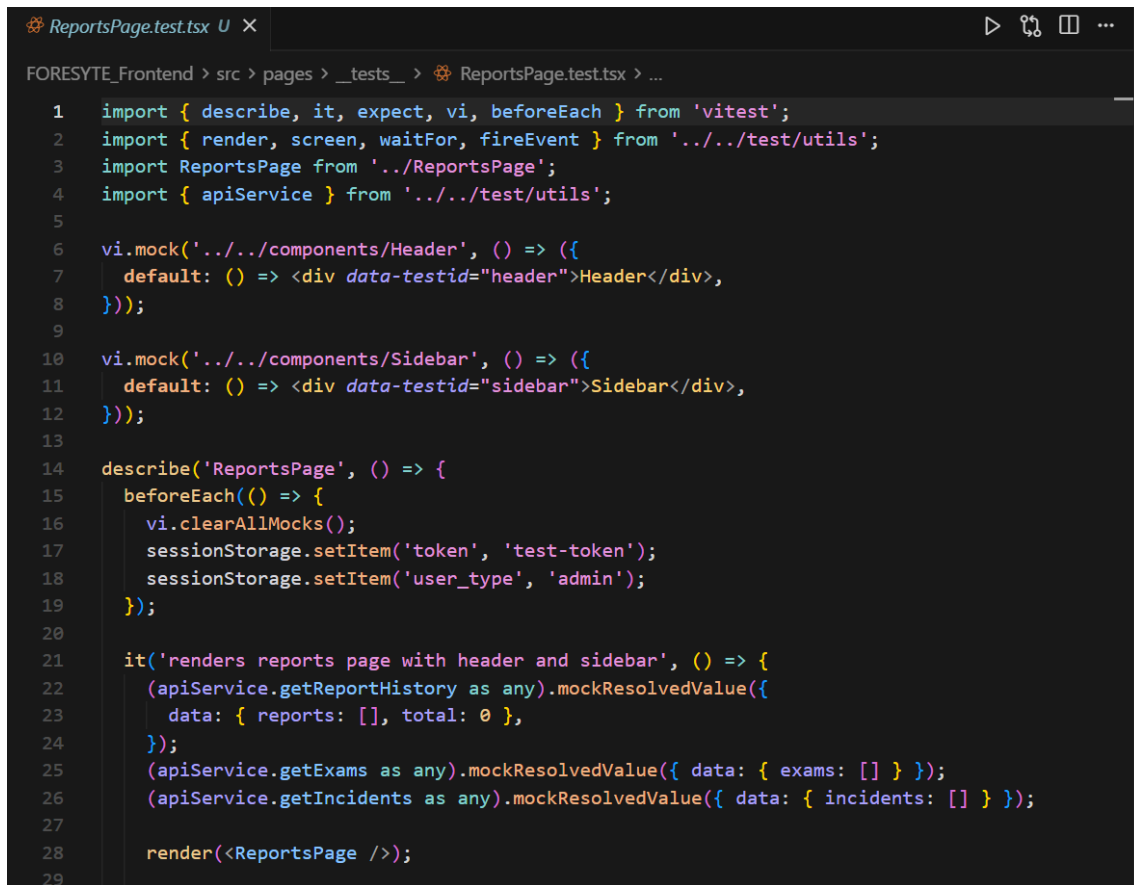


```
MonitoringPage.test.tsx U X
FORESYTE_Frontend > src > pages > _tests_ > MonitoringPage.test.tsx > ...

1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../test/utils';
3 import MonitoringPage from '../MonitoringPage';
4 import { apiService } from '../test/utils';
5
6 vi.mock('../components/Header', () => ({
7   default: () => <div data-testid="header">Header</div>,
8 }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 }));
13
14 vi.mock('../components/PhoneCameraMonitor', () => ({
15   default: ({ onClose }: { onClose: () => void }) => (
16     <div data-testid="phone-camera-monitor">
17       <button onClick={onClose}>Close</button>
18     </div>
19   ),
20 }));
21
22 describe('MonitoringPage', () => {
23   beforeEach(() => {
24     vi.clearAllMocks();
25     sessionStorage.setItem('token', 'test-token');
26     sessionStorage.setItem('user_type', 'admin');
27   });
28
29   it('renders monitoring page with header and sidebar', () => {
```

A.1.1.9 Reports Page

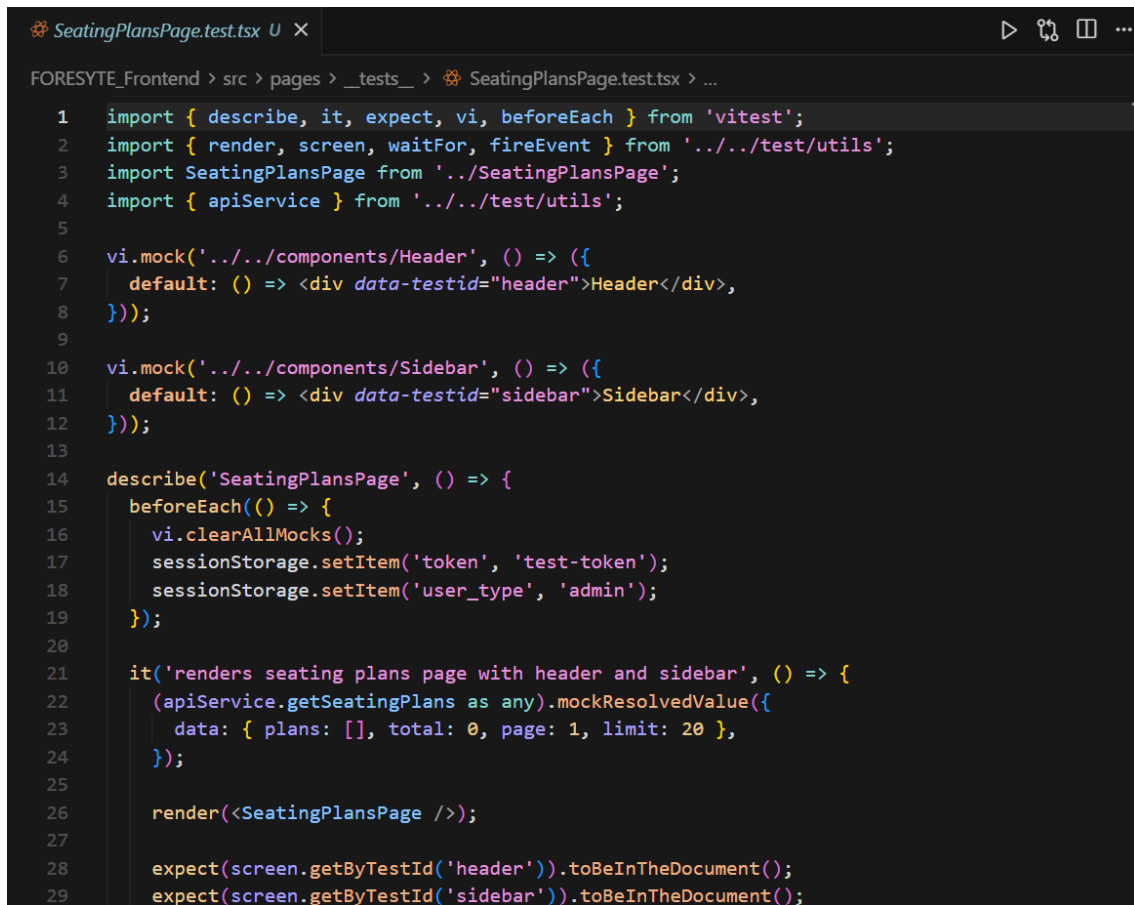
Figure A.9: Admin Portal Reports Page



```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../test/utils';
3 import ReportsPage from '../ReportsPage';
4 import { apiService } from '../test/utils';
5
6 vi.mock('../components/Header', () => ({
7   default: () => <div data-testid="header">Header</div>,
8 }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 }));
13
14 describe('ReportsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders reports page with header and sidebar', () => {
22     (apiService.getReportHistory as any).mockResolvedValue({
23       data: { reports: [], total: 0 },
24     });
25     (apiService.getExams as any).mockResolvedValue({ data: { exams: [] } });
26     (apiService.getIncidents as any).mockResolvedValue({ data: { incidents: [] } });
27
28     render(<ReportsPage />);
29   });
30 }
```

A.1.1.10 Seating Plan Page

Figure A.10: Admin Portal Seating Plan Management Page

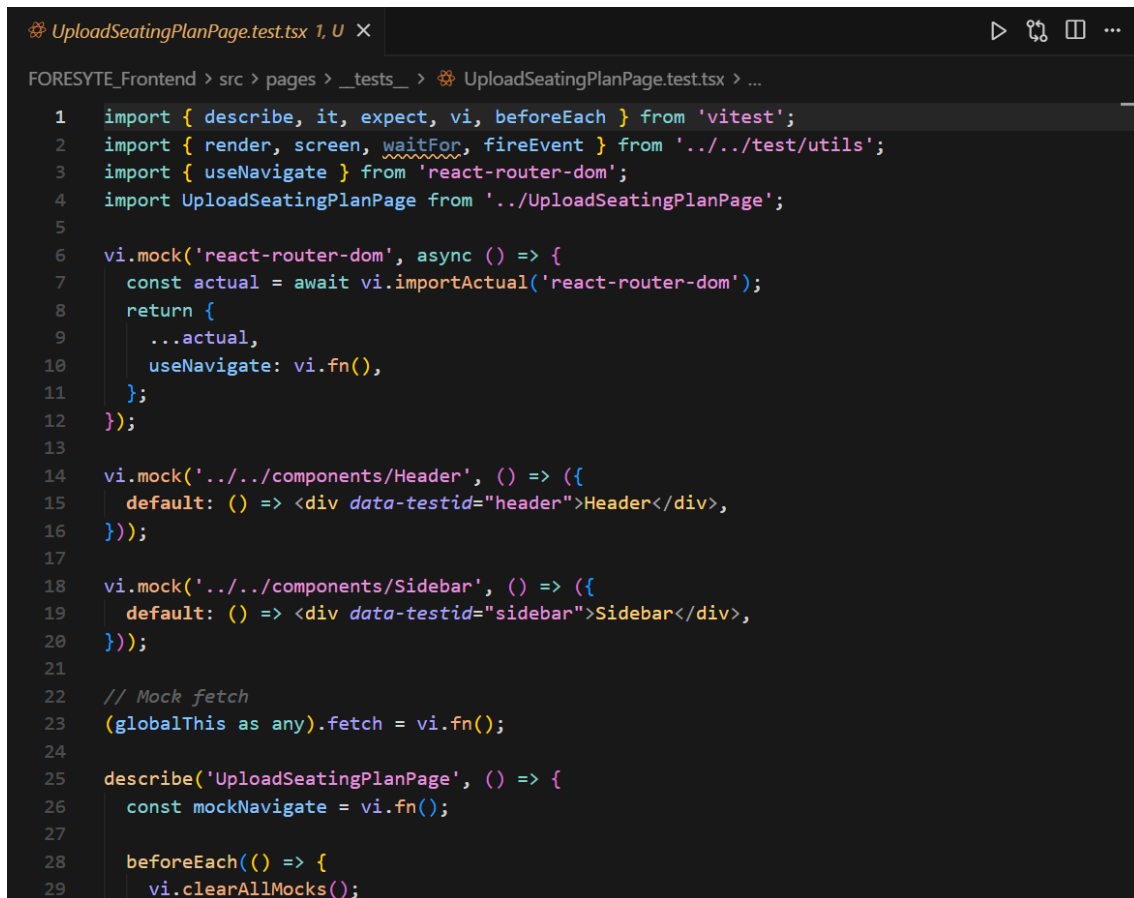


The image shows a code editor window with a file named `SeatingPlansPage.test.tsx`. The editor displays a Vitest test file for the `SeatingPlansPage` component. The code includes imports for testing utilities, mocks for the `Header` and `Sidebar` components, and a test suite for the `SeatingPlansPage` component. The test suite includes a `beforeEach` block to clear mocks and set session storage, and an `it` block to render the page and verify that the header and sidebar are present in the document.

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../test/utils';
3 import SeatingPlansPage from '../SeatingPlansPage';
4 import { apiService } from '../test/utils';
5
6 vi.mock('../components/Header', () => ({
7   default: () => <div data-testid="header">Header</div>,
8 }));
9
10 vi.mock('../components/Sidebar', () => ({
11   default: () => <div data-testid="sidebar">Sidebar</div>,
12 }));
13
14 describe('SeatingPlansPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.setItem('token', 'test-token');
18     sessionStorage.setItem('user_type', 'admin');
19   });
20
21   it('renders seating plans page with header and sidebar', () => {
22     (apiService.getSeatingPlans as any).mockResolvedValue({
23       data: { plans: [], total: 0, page: 1, limit: 20 },
24     });
25
26     render(<SeatingPlansPage />);
27
28     expect(screen.getByTestId('header')).toBeInTheDocument();
29     expect(screen.getByTestId('sidebar')).toBeInTheDocument();
30   });
31 });
```

A.1.1.11 Upload Seating Plan Page

Figure A.11: Admin Portal Upload Seating Plan Page

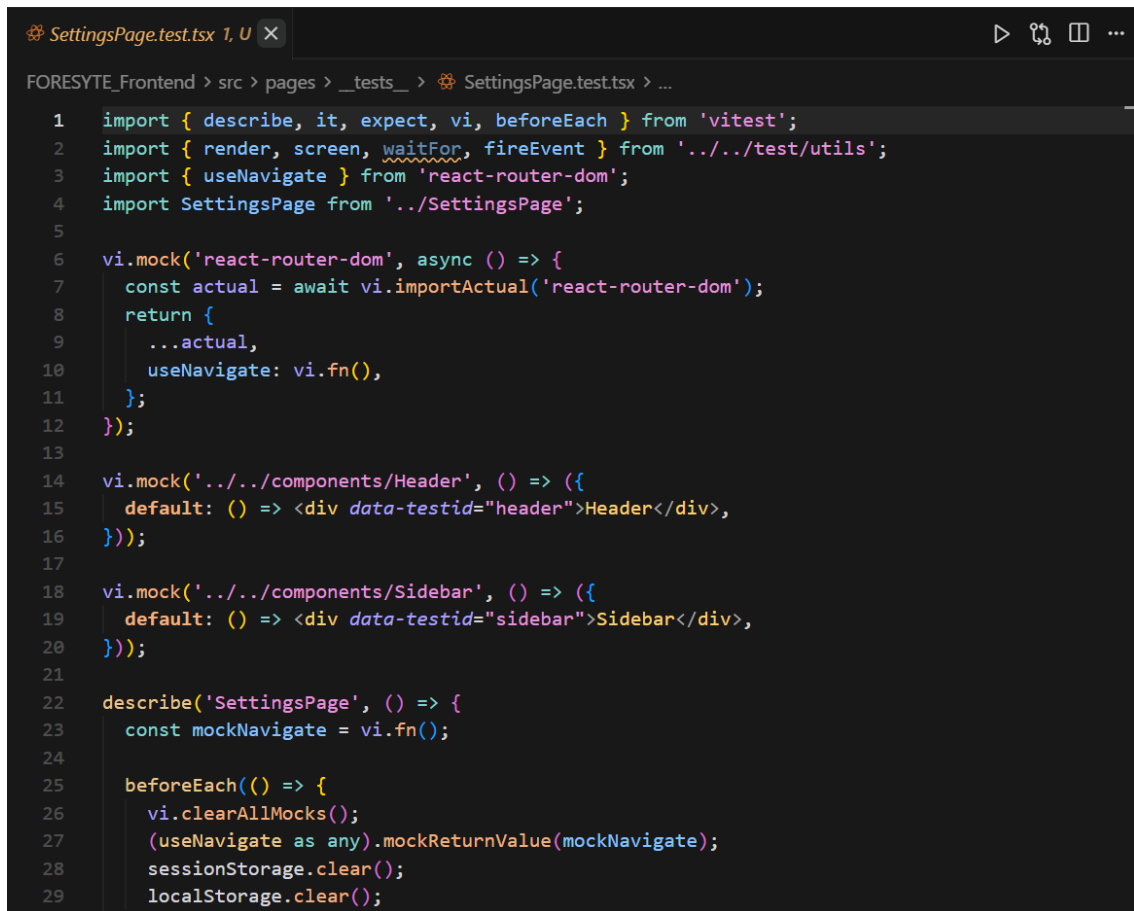


The screenshot shows a code editor with a file named `UploadSeatingPlanPage.test.tsx` open. The editor's breadcrumb navigation shows the path: `FORESYTE_Frontend > src > pages > _tests_ > UploadSeatingPlanPage.test.tsx > ...`. The code is a Jest test file using `vitest` and `vi` for mocking. It imports `describe`, `it`, `expect`, `vi`, and `beforeEach` from `'vitest'`. It also imports `render`, `screen`, `waitFor`, and `fireEvent` from `'../../test/utils'`, and `useNavigate` from `'react-router-dom'`. The test file imports `UploadSeatingPlanPage` from `'../UploadSeatingPlanPage'`. The test suite includes several `vi.mock` calls: one for `'react-router-dom'` to mock `useNavigate`, one for `'../../components/Header'` to mock the `Header` component, and one for `'../../components/Sidebar'` to mock the `Sidebar` component. It also mocks `fetch` on `globalThis`. The test suite is defined with `describe('UploadSeatingPlanPage', () => {` and includes a `beforeEach` block to clear all mocks.

```
1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../../test/utils';
3 import { useNavigate } from 'react-router-dom';
4 import UploadSeatingPlanPage from '../UploadSeatingPlanPage';
5
6 vi.mock('react-router-dom', async () => {
7   const actual = await vi.importActual('react-router-dom');
8   return {
9     ...actual,
10    useNavigate: vi.fn(),
11  };
12 });
13
14 vi.mock('../../components/Header', () => ({
15   default: () => <div data-testid="header">Header</div>,
16 }));
17
18 vi.mock('../../components/Sidebar', () => ({
19   default: () => <div data-testid="sidebar">Sidebar</div>,
20 }));
21
22 // Mock fetch
23 (globalThis as any).fetch = vi.fn();
24
25 describe('UploadSeatingPlanPage', () => {
26   const mockNavigate = vi.fn();
27
28   beforeEach(() => {
29     vi.clearAllMocks();
```

A.1.1.12 Settings Page

Figure A.12: Admin Portal Settings Page



```
SettingsPage.test.tsx 1, U X
FORESYTE_Frontend > src > pages > __tests__ > SettingsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utlis';
3  import { useNavigate } from 'react-router-dom';
4  import SettingsPage from '../SettingsPage';
5
6  vi.mock('react-router-dom', async () => {
7    const actual = await vi.importActual('react-router-dom');
8    return {
9      ...actual,
10     useNavigate: vi.fn(),
11   };
12 });
13
14 vi.mock('../components/Header', () => ({
15   default: () => <div data-testid="header">Header</div>,
16 }));
17
18 vi.mock('../components/Sidebar', () => ({
19   default: () => <div data-testid="sidebar">Sidebar</div>,
20 }));
21
22 describe('SettingsPage', () => {
23   const mockNavigate = vi.fn();
24
25   beforeEach(() => {
26     vi.clearAllMocks();
27     (useNavigate as any).mockReturnValue(mockNavigate);
28     sessionStorage.clear();
29     localStorage.clear();
30   });
31
32   it('renders the SettingsPage component', () => {
33     // Test logic
34   });
35
36   it('calls useNavigate when a button is clicked', () => {
37     // Test logic
38   });
39
40   it('clears session and local storage on page load', () => {
41     // Test logic
42   });
43
44   it('handles navigation correctly', () => {
45     // Test logic
46   });
47
48   it('renders the correct header and sidebar', () => {
49     // Test logic
50   });
51
52   it('handles form submissions', () => {
53     // Test logic
54   });
55
56   it('handles errors gracefully', () => {
57     // Test logic
58   });
59
60   it('updates the settings state', () => {
61     // Test logic
62   });
63
64   it('handles user authentication', () => {
65     // Test logic
66   });
67
68   it('renders the correct footer', () => {
69     // Test logic
70   });
71
72   it('handles user logout', () => {
73     // Test logic
74   });
75
76   it('handles user login', () => {
77     // Test logic
78   });
79
80   it('handles user registration', () => {
81     // Test logic
82   });
83
84   it('handles user password reset', () => {
85     // Test logic
86   });
87
88   it('handles user profile update', () => {
89     // Test logic
90   });
91
92   it('handles user settings update', () => {
93     // Test logic
94   });
95
96   it('handles user preferences update', () => {
97     // Test logic
98   });
99
100  it('handles user notifications', () => {
101    // Test logic
102  });
103
104  it('handles user alerts', () => {
105    // Test logic
106  });
107
108  it('handles user messages', () => {
109    // Test logic
110  });
111
112  it('handles user comments', () => {
113    // Test logic
114  });
115
116  it('handles user reviews', () => {
117    // Test logic
118  });
119
120  it('handles user ratings', () => {
121    // Test logic
122  });
123
124  it('handles user feedback', () => {
125    // Test logic
126  });
127
128  it('handles user suggestions', () => {
129    // Test logic
130  });
131
132  it('handles user reports', () => {
133    // Test logic
134  });
135
136  it('handles user complaints', () => {
137    // Test logic
138  });
139
140  it('handles user inquiries', () => {
141    // Test logic
142  });
143
144  it('handles user questions', () => {
145    // Test logic
146  });
147
148  it('handles user answers', () => {
149    // Test logic
150  });
151
152  it('handles user comments', () => {
153    // Test logic
154  });
155
156  it('handles user reviews', () => {
157    // Test logic
158  });
159
160  it('handles user ratings', () => {
161    // Test logic
162  });
163
164  it('handles user feedback', () => {
165    // Test logic
166  });
167
168  it('handles user suggestions', () => {
169    // Test logic
170  });
171
172  it('handles user reports', () => {
173    // Test logic
174  });
175
176  it('handles user complaints', () => {
177    // Test logic
178  });
179
180  it('handles user inquiries', () => {
181    // Test logic
182  });
183
184  it('handles user questions', () => {
185    // Test logic
186  });
187
188  it('handles user answers', () => {
189    // Test logic
190  });
191
192  it('handles user comments', () => {
193    // Test logic
194  });
195
196  it('handles user reviews', () => {
197    // Test logic
198  });
199
200  it('handles user ratings', () => {
201    // Test logic
202  });
203
204  it('handles user feedback', () => {
205    // Test logic
206  });
207
208  it('handles user suggestions', () => {
209    // Test logic
210  });
211
212  it('handles user reports', () => {
213    // Test logic
214  });
215
216  it('handles user complaints', () => {
217    // Test logic
218  });
219
220  it('handles user inquiries', () => {
221    // Test logic
222  });
223
224  it('handles user questions', () => {
225    // Test logic
226  });
227
228  it('handles user answers', () => {
229    // Test logic
230  });
231
232  it('handles user comments', () => {
233    // Test logic
234  });
235
236  it('handles user reviews', () => {
237    // Test logic
238  });
239
240  it('handles user ratings', () => {
241    // Test logic
242  });
243
244  it('handles user feedback', () => {
245    // Test logic
246  });
247
248  it('handles user suggestions', () => {
249    // Test logic
250  });
251
252  it('handles user reports', () => {
253    // Test logic
254  });
255
256  it('handles user complaints', () => {
257    // Test logic
258  });
259
260  it('handles user inquiries', () => {
261    // Test logic
262  });
263
264  it('handles user questions', () => {
265    // Test logic
266  });
267
268  it('handles user answers', () => {
269    // Test logic
270  });
271
272  it('handles user comments', () => {
273    // Test logic
274  });
275
276  it('handles user reviews', () => {
277    // Test logic
278  });
279
280  it('handles user ratings', () => {
281    // Test logic
282  });
283
284  it('handles user feedback', () => {
285    // Test logic
286  });
287
288  it('handles user suggestions', () => {
289    // Test logic
290  });
291
292  it('handles user reports', () => {
293    // Test logic
294  });
295
296  it('handles user complaints', () => {
297    // Test logic
298  });
299
300  it('handles user inquiries', () => {
301    // Test logic
302  });
303
304  it('handles user questions', () => {
305    // Test logic
306  });
307
308  it('handles user answers', () => {
309    // Test logic
310  });
311
312  it('handles user comments', () => {
313    // Test logic
314  });
315
316  it('handles user reviews', () => {
317    // Test logic
318  });
319
320  it('handles user ratings', () => {
321    // Test logic
322  });
323
324  it('handles user feedback', () => {
325    // Test logic
326  });
327
328  it('handles user suggestions', () => {
329    // Test logic
330  });
331
332  it('handles user reports', () => {
333    // Test logic
334  });
335
336  it('handles user complaints', () => {
337    // Test logic
338  });
339
340  it('handles user inquiries', () => {
341    // Test logic
342  });
343
344  it('handles user questions', () => {
345    // Test logic
346  });
347
348  it('handles user answers', () => {
349    // Test logic
350  });
351
352  it('handles user comments', () => {
353    // Test logic
354  });
355
356  it('handles user reviews', () => {
357    // Test logic
358  });
359
360  it('handles user ratings', () => {
361    // Test logic
362  });
363
364  it('handles user feedback', () => {
365    // Test logic
366  });
367
368  it('handles user suggestions', () => {
369    // Test logic
370  });
371
372  it('handles user reports', () => {
373    // Test logic
374  });
375
376  it('handles user complaints', () => {
377    // Test logic
378  });
379
380  it('handles user inquiries', () => {
381    // Test logic
382  });
383
384  it('handles user questions', () => {
385    // Test logic
386  });
387
388  it('handles user answers', () => {
389    // Test logic
390  });
391
392  it('handles user comments', () => {
393    // Test logic
394  });
395
396  it('handles user reviews', () => {
397    // Test logic
398  });
399
400  it('handles user ratings', () => {
401    // Test logic
402  });
403
404  it('handles user feedback', () => {
405    // Test logic
406  });
407
408  it('handles user suggestions', () => {
409    // Test logic
410  });
411
412  it('handles user reports', () => {
413    // Test logic
414  });
415
416  it('handles user complaints', () => {
417    // Test logic
418  });
419
420  it('handles user inquiries', () => {
421    // Test logic
422  });
423
424  it('handles user questions', () => {
425    // Test logic
426  });
427
428  it('handles user answers', () => {
429    // Test logic
430  });
431
432  it('handles user comments', () => {
433    // Test logic
434  });
435
436  it('handles user reviews', () => {
437    // Test logic
438  });
439
440  it('handles user ratings', () => {
441    // Test logic
442  });
443
444  it('handles user feedback', () => {
445    // Test logic
446  });
447
448  it('handles user suggestions', () => {
449    // Test logic
450  });
451
452  it('handles user reports', () => {
453    // Test logic
454  });
455
456  it('handles user complaints', () => {
457    // Test logic
458  });
459
460  it('handles user inquiries', () => {
461    // Test logic
462  });
463
464  it('handles user questions', () => {
465    // Test logic
466  });
467
468  it('handles user answers', () => {
469    // Test logic
470  });
471
472  it('handles user comments', () => {
473    // Test logic
474  });
475
476  it('handles user reviews', () => {
477    // Test logic
478  });
479
480  it('handles user ratings', () => {
481    // Test logic
482  });
483
484  it('handles user feedback', () => {
485    // Test logic
486  });
487
488  it('handles user suggestions', () => {
489    // Test logic
490  });
491
492  it('handles user reports', () => {
493    // Test logic
494  });
495
496  it('handles user complaints', () => {
497    // Test logic
498  });
499
500  it('handles user inquiries', () => {
501    // Test logic
502  });
503
504  it('handles user questions', () => {
505    // Test logic
506  });
507
508  it('handles user answers', () => {
509    // Test logic
510  });
511
512  it('handles user comments', () => {
513    // Test logic
514  });
515
516  it('handles user reviews', () => {
517    // Test logic
518  });
519
520  it('handles user ratings', () => {
521    // Test logic
522  });
523
524  it('handles user feedback', () => {
525    // Test logic
526  });
527
528  it('handles user suggestions', () => {
529    // Test logic
530  });
531
532  it('handles user reports', () => {
533    // Test logic
534  });
535
536  it('handles user complaints', () => {
537    // Test logic
538  });
539
540  it('handles user inquiries', () => {
541    // Test logic
542  });
543
544  it('handles user questions', () => {
545    // Test logic
546  });
547
548  it('handles user answers', () => {
549    // Test logic
550  });
551
552  it('handles user comments', () => {
553    // Test logic
554  });
555
556  it('handles user reviews', () => {
557    // Test logic
558  });
559
560  it('handles user ratings', () => {
561    // Test logic
562  });
563
564  it('handles user feedback', () => {
565    // Test logic
566  });
567
568  it('handles user suggestions', () => {
569    // Test logic
570  });
571
572  it('handles user reports', () => {
573    // Test logic
574  });
575
576  it('handles user complaints', () => {
577    // Test logic
578  });
579
580  it('handles user inquiries', () => {
581    // Test logic
582  });
583
584  it('handles user questions', () => {
585    // Test logic
586  });
587
588  it('handles user answers', () => {
589    // Test logic
590  });
591
592  it('handles user comments', () => {
593    // Test logic
594  });
595
596  it('handles user reviews', () => {
597    // Test logic
598  });
599
600  it('handles user ratings', () => {
601    // Test logic
602  });
603
604  it('handles user feedback', () => {
605    // Test logic
606  });
607
608  it('handles user suggestions', () => {
609    // Test logic
610  });
611
612  it('handles user reports', () => {
613    // Test logic
614  });
615
616  it('handles user complaints', () => {
617    // Test logic
618  });
619
620  it('handles user inquiries', () => {
621    // Test logic
622  });
623
624  it('handles user questions', () => {
625    // Test logic
626  });
627
628  it('handles user answers', () => {
629    // Test logic
630  });
631
632  it('handles user comments', () => {
633    // Test logic
634  });
635
636  it('handles user reviews', () => {
637    // Test logic
638  });
639
640  it('handles user ratings', () => {
641    // Test logic
642  });
643
644  it('handles user feedback', () => {
645    // Test logic
646  });
647
648  it('handles user suggestions', () => {
649    // Test logic
650  });
651
652  it('handles user reports', () => {
653    // Test logic
654  });
655
656  it('handles user complaints', () => {
657    // Test logic
658  });
659
660  it('handles user inquiries', () => {
661    // Test logic
662  });
663
664  it('handles user questions', () => {
665    // Test logic
666  });
667
668  it('handles user answers', () => {
669    // Test logic
670  });
671
672  it('handles user comments', () => {
673    // Test logic
674  });
675
676  it('handles user reviews', () => {
677    // Test logic
678  });
679
680  it('handles user ratings', () => {
681    // Test logic
682  });
683
684  it('handles user feedback', () => {
685    // Test logic
686  });
687
688  it('handles user suggestions', () => {
689    // Test logic
690  });
691
692  it('handles user reports', () => {
693    // Test logic
694  });
695
696  it('handles user complaints', () => {
697    // Test logic
698  });
699
700  it('handles user inquiries', () => {
701    // Test logic
702  });
703
704  it('handles user questions', () => {
705    // Test logic
706  });
707
708  it('handles user answers', () => {
709    // Test logic
710  });
711
712  it('handles user comments', () => {
713    // Test logic
714  });
715
716  it('handles user reviews', () => {
717    // Test logic
718  });
719
720  it('handles user ratings', () => {
721    // Test logic
722  });
723
724  it('handles user feedback', () => {
725    // Test logic
726  });
727
728  it('handles user suggestions', () => {
729    // Test logic
730  });
731
732  it('handles user reports', () => {
733    // Test logic
734  });
735
736  it('handles user complaints', () => {
737    // Test logic
738  });
739
740  it('handles user inquiries', () => {
741    // Test logic
742  });
743
744  it('handles user questions', () => {
745    // Test logic
746  });
747
748  it('handles user answers', () => {
749    // Test logic
750  });
751
752  it('handles user comments', () => {
753    // Test logic
754  });
755
756  it('handles user reviews', () => {
757    // Test logic
758  });
759
760  it('handles user ratings', () => {
761    // Test logic
762  });
763
764  it('handles user feedback', () => {
765    // Test logic
766  });
767
768  it('handles user suggestions', () => {
769    // Test logic
770  });
771
772  it('handles user reports', () => {
773    // Test logic
774  });
775
776  it('handles user complaints', () => {
777    // Test logic
778  });
779
780  it('handles user inquiries', () => {
781    // Test logic
782  });
783
784  it('handles user questions', () => {
785    // Test logic
786  });
787
788  it('handles user answers', () => {
789    // Test logic
790  });
791
792  it('handles user comments', () => {
793    // Test logic
794  });
795
796  it('handles user reviews', () => {
797    // Test logic
798  });
799
800  it('handles user ratings', () => {
801    // Test logic
802  });
803
804  it('handles user feedback', () => {
805    // Test logic
806  });
807
808  it('handles user suggestions', () => {
809    // Test logic
810  });
811
812  it('handles user reports', () => {
813    // Test logic
814  });
815
816  it('handles user complaints', () => {
817    // Test logic
818  });
819
820  it('handles user inquiries', () => {
821    // Test logic
822  });
823
824  it('handles user questions', () => {
825    // Test logic
826  });
827
828  it('handles user answers', () => {
829    // Test logic
830  });
831
832  it('handles user comments', () => {
833    // Test logic
834  });
835
836  it('handles user reviews', () => {
837    // Test logic
838  });
839
840  it('handles user ratings', () => {
841    // Test logic
842  });
843
844  it('handles user feedback', () => {
845    // Test logic
846  });
847
848  it('handles user suggestions', () => {
849    // Test logic
850  });
851
852  it('handles user reports', () => {
853    // Test logic
854  });
855
856  it('handles user complaints', () => {
857    // Test logic
858  });
859
860  it('handles user inquiries', () => {
861    // Test logic
862  });
863
864  it('handles user questions', () => {
865    // Test logic
866  });
867
868  it('handles user answers', () => {
869    // Test logic
870  });
871
872  it('handles user comments', () => {
873    // Test logic
874  });
875
876  it('handles user reviews', () => {
877    // Test logic
878  });
879
880  it('handles user ratings', () => {
881    // Test logic
882  });
883
884  it('handles user feedback', () => {
885    // Test logic
886  });
887
888  it('handles user suggestions', () => {
889    // Test logic
890  });
891
892  it('handles user reports', () => {
893    // Test logic
894  });
895
896  it('handles user complaints', () => {
897    // Test logic
898  });
899
900  it('handles user inquiries', () => {
901    // Test logic
902  });
903
904  it('handles user questions', () => {
905    // Test logic
906  });
907
908  it('handles user answers', () => {
909    // Test logic
910  });
911
912  it('handles user comments', () => {
913    // Test logic
914  });
915
916  it('handles user reviews', () => {
917    // Test logic
918  });
919
920  it('handles user ratings', () => {
921    // Test logic
922  });
923
924  it('handles user feedback', () => {
925    // Test logic
926  });
927
928  it('handles user suggestions', () => {
929    // Test logic
930  });
931
932  it('handles user reports', () => {
933    // Test logic
934  });
935
936  it('handles user complaints', () => {
937    // Test logic
938  });
939
940  it('handles user inquiries', () => {
941    // Test logic
942  });
943
944  it('handles user questions', () => {
945    // Test logic
946  });
947
948  it('handles user answers', () => {
949    // Test logic
950  });
951
952  it('handles user comments', () => {
953    // Test logic
954  });
955
956  it('handles user reviews', () => {
957    // Test logic
958  });
959
960  it('handles user ratings', () => {
961    // Test logic
962  });
963
964  it('handles user feedback', () => {
965    // Test logic
966  });
967
968  it('handles user suggestions', () => {
969    // Test logic
970  });
971
972  it('handles user reports', () => {
973    // Test logic
974  });
975
976  it('handles user complaints', () => {
977    // Test logic
978  });
979
980  it('handles user inquiries', () => {
981    // Test logic
982  });
983
984  it('handles user questions', () => {
985    // Test logic
986  });
987
988  it('handles user answers', () => {
989    // Test logic
990  });
991
992  it('handles user comments', () => {
993    // Test logic
994  });
995
996  it('handles user reviews', () => {
997    // Test logic
998  });
999
1000  it('handles user ratings', () => {
1001    // Test logic
1002  });
1003
1004  it('handles user feedback', () => {
1005    // Test logic
1006  });
1007
1008  it('handles user suggestions', () => {
1009    // Test logic
1010  });
1011
1012  it('handles user reports', () => {
1013    // Test logic
1014  });
1015
1016  it('handles user complaints', () => {
1017    // Test logic
1018  });
1019
1020  it('handles user inquiries', () => {
1021    // Test logic
1022  });
1023
1024  it('handles user questions', () => {
1025    // Test logic
1026  });
1027
1028  it('handles user answers', () => {
1029    // Test logic
1030  });
1031
1032  it('handles user comments', () => {
1033    // Test logic
1034  });
1035
1036  it('handles user reviews', () => {
1037    // Test logic
1038  });
1039
1040  it('handles user ratings', () => {
1041    // Test logic
1042  });
1043
1044  it('handles user feedback', () => {
1045    // Test logic
1046  });
1047
1048  it('handles user suggestions', () => {
1049    // Test logic
1050  });
1051
1052  it('handles user reports', () => {
1053    // Test logic
1054  });
1055
1056  it('handles user complaints', () => {
1057    // Test logic
1058  });
1059
1060  it('handles user inquiries', () => {
1061    // Test logic
1062  });
1063
1064  it('handles user questions', () => {
1065    // Test logic
1066  });
1067
1068  it('handles user answers', () => {
1069    // Test logic
1070  });
1071
1072  it('handles user comments', () => {
1073    // Test logic
1074  });
1075
1076  it('handles user reviews', () => {
1077    // Test logic
1078  });
1079
1080  it('handles user ratings', () => {
1081    // Test logic
1082  });
1083
1084  it('handles user feedback', () => {
1085    // Test logic
1086  });
1087
1088  it('handles user suggestions', () => {
1089    // Test logic
1090  });
1091
1092  it('handles user reports', () => {
1093    // Test logic
1094  });
1095
1096  it('handles user complaints', () => {
1097    // Test logic
1098  });
1099
1100  it('handles user inquiries', () => {
1101    // Test logic
1102  });
1103
1104  it('handles user questions', () => {
1105    // Test logic
1106  });
1107
1108  it('handles user answers', () => {
1109    // Test logic
1110  });
1111
1112  it('handles user comments', () => {
1113    // Test logic
1114  });
1115
1116  it('handles user reviews', () => {
1117    // Test logic
1118  });
1119
1120  it('handles user ratings', () => {
1121    // Test logic
1122  });
1123
1124  it('handles user feedback', () => {
1125    // Test logic
1126  });
1127
1128  it('handles user suggestions', () => {
1129    // Test logic
1130  });
1131
1132  it('handles user reports', () => {
1133    // Test logic
1134  });
1135
1136  it('handles user complaints', () => {
1137    // Test logic
1138  });
1139
1140  it('handles user inquiries', () => {
1141    // Test logic
1142  });
1143
1144  it('handles user questions', () => {
1145    // Test logic
1146  });
1147
1148  it('handles user answers', () => {
1149    // Test logic
1150  });
1151
1152  it('handles user comments', () => {
1153    // Test logic
1154  });
1155
1156  it('handles user reviews', () => {
1157    // Test logic
1158  });
1159
1160  it('handles user ratings', () => {
1161    // Test logic
1162  });
1163
1164  it('handles user feedback', () => {
1165    // Test logic
1166  });
1167
1168  it('handles user suggestions', () => {
1169    // Test logic
1170  });
1171
1172  it('handles user reports', () => {
1173    // Test logic
1174  });
1175
1176  it('handles user complaints', () => {
1177    // Test logic
1178  });
1179
1180  it('handles user inquiries', () => {
1181    // Test logic
1182  });
1183
1184  it('handles user questions', () => {
1185    // Test logic
1186  });
1187
1188  it('handles user answers', () => {
1189    // Test logic
1190  });
1191
1192  it('handles user comments', () => {
1193    // Test logic
1194  });
1195
1196  it('handles user reviews', () => {
1197    // Test logic
1198  });
1199
1200  it('handles user ratings', () => {
1201    // Test logic
1202  });
1203
1204  it('handles user feedback', () => {
1205    // Test logic
1206  });
1207
1208  it('handles user suggestions', () => {
1209    // Test logic
1210  });
1211
1212  it('handles user reports', () => {
1213    // Test logic
1214  });
1215
1216  it('handles user complaints', () => {
1217    // Test logic
1218  });
1219
1220  it('handles user inquiries', () => {
1221    // Test logic
1222  });
1223
1224  it('handles user questions', () => {
1225    // Test logic
1226  });
1227
1228  it('handles user answers', () => {
1229    // Test logic
1230  });
1231
1232  it('handles user comments', () => {
1233    // Test logic
1234  });
1235
1236  it('handles user reviews', () => {
1237    // Test logic
1238  });
1239
1240  it('handles user ratings', () => {
1241    // Test logic
1242  });
1243
1244  it('handles user feedback', () => {
1245    // Test logic
1246  });
1247
1248  it('handles user suggestions', () => {
1249    // Test logic
1250  });
1251
1252  it('handles user reports', () => {
1253    // Test logic
1254  });
1255
1256  it('handles user complaints', () => {
1257    // Test logic
1258  });
1259
1260  it('handles user inquiries', () => {
1261    // Test logic
1262  });
1263
1264  it('handles user questions', () => {
1265    // Test logic
1266  });
1267
1268  it('handles user answers', () => {
1269    // Test logic
1270  });
1271
1272  it('handles user comments', () => {
1273    // Test logic
1274  });
1275
1276  it('handles user reviews', () => {
1277    // Test logic
1278  });
1279
1280  it('handles user ratings', () => {
1281    // Test logic
1282  });
1283
1284  it('handles user feedback', () => {
1285    // Test logic
1286  });
1287
1288  it('handles user suggestions', () => {
1289    // Test logic
1290  });
1291
1292  it('handles user reports', () => {
1293    // Test logic
1294  });
1295
1296  it('handles user complaints', () => {
1297    // Test logic
1298  });
1299
1300  it('handles user inquiries', () => {
1301    // Test logic
1302  });
1303
1304  it('handles user questions', () => {
1305    // Test logic
1306  });
1307
1308  it('handles user answers', () => {
1309    // Test logic
1310  });
1311
1312  it('handles user comments', () => {
1313    // Test logic
1314  });
1315
1316  it('handles user reviews', () => {
1317    // Test logic
1318  });
1319
1320  it('handles user ratings', () => {
1321    // Test logic
1322  });
1323
1324  it('handles user feedback', () => {
1325    // Test logic
1326  });
1327
1328  it('handles user suggestions', () => {
1329    // Test logic
1330  });
1331
1332  it('handles user reports', () => {
1333    // Test logic
1334  });
1335
1336  it('handles user complaints', () => {
1337    // Test logic
1338  });
1339
1340  it('handles user inquiries', () => {
1341    // Test logic
1342  });
1343
1344  it('handles user questions', () => {
1345    // Test logic
1346  });
1347
1348  it('handles user answers', () => {
1349    // Test logic
1350  });
1351
1352  it('handles user comments', () => {
1353    // Test logic
1354  });
1355
1356  it('handles user reviews', () => {
1357    // Test logic
1358  });
1359
1360  it('handles user ratings', () => {
1361    // Test logic
1362  });
1363
1364  it('handles user feedback', () => {
1365    // Test logic
1366  });
1367
1368  it('handles user suggestions', () => {
1369    // Test logic
1370  });
1371
1372  it('handles user reports', () => {
1373    // Test logic
1374  });
1375
1376  it('handles user complaints', () => {
1377    // Test logic
1378  });
1379
1380  it('handles user inquiries', () => {

```

A.1.1.13 Test Results

Figure A.13: Admin Portal Unit Test Results

```
This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

✓ src/pages/__tests__/AnalyticsPage.test.tsx (6) 486ms
✓ src/pages/__tests__/DashboardPage.test.tsx (6) 584ms
✓ src/pages/__tests__/ExamsPage.test.tsx (6) 597ms
✓ src/pages/__tests__/Incidents.test.tsx (5) 365ms
✓ src/pages/__tests__/LoginPage.test.tsx (7) 956ms
✓ src/pages/__tests__/MonitoringPage.test.tsx (5) 555ms
✓ src/pages/__tests__/ReportsPage.test.tsx (4) 611ms
✓ src/pages/__tests__/SeatingPlansPage.test.tsx (4) 571ms
✓ src/pages/__tests__/SettingsPage.test.tsx (7) 747ms
✓ src/pages/__tests__/SignUpPage.test.tsx (8) 1363ms
✓ src/pages/__tests__/UploadSeatingPlanPage.test.tsx (5) 1339ms
✓ src/pages/__tests__/UsersPage.test.tsx (8) 895ms

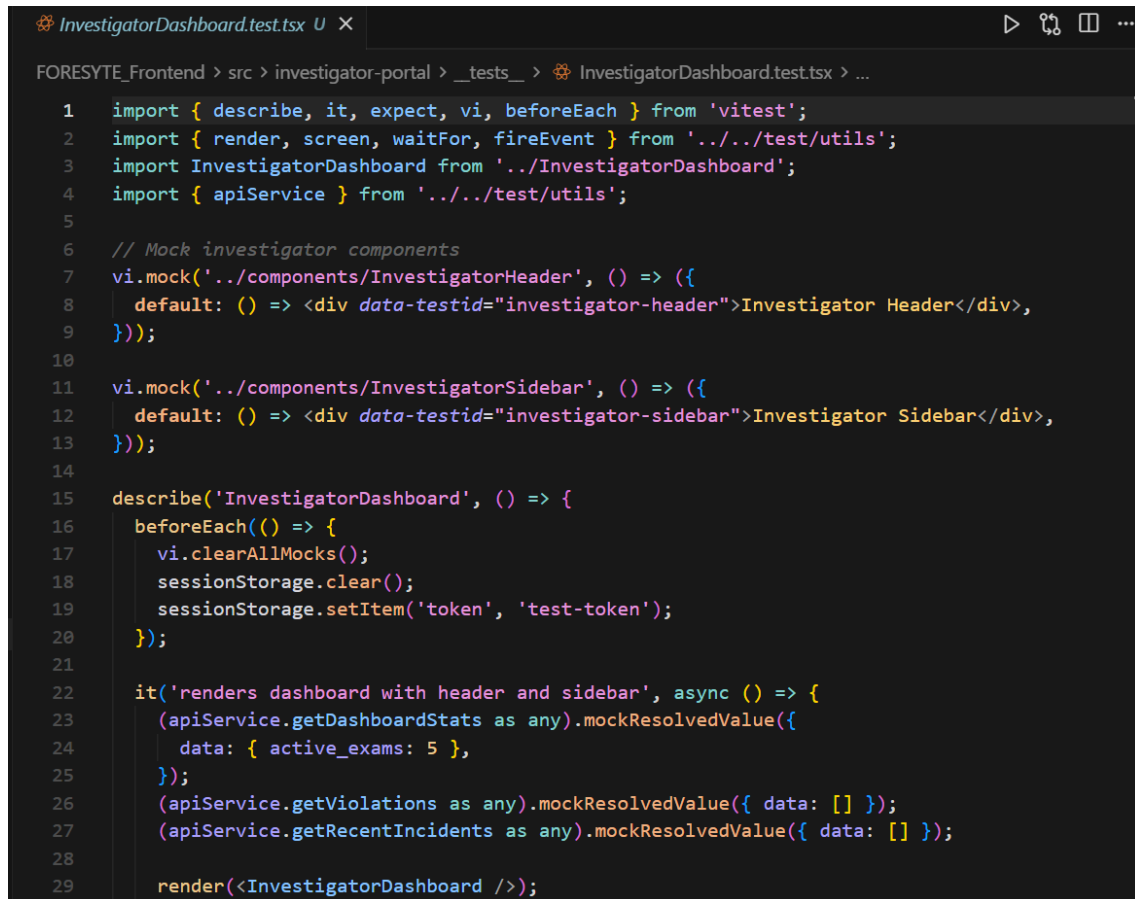
Test Files 12 passed (12)
Tests 71 passed (71)
Start at 04:33:35
Duration 9.82s (transform 1.17s, setup 4.18s, collect 4.44s, tests 9.07s, environment 11.99s, prepare 2.47s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

A.1.2 Investigator Portal Screenshots

A.1.2.1 Investigator Dashboard

Figure A.14: Investigator Portal Dashboard

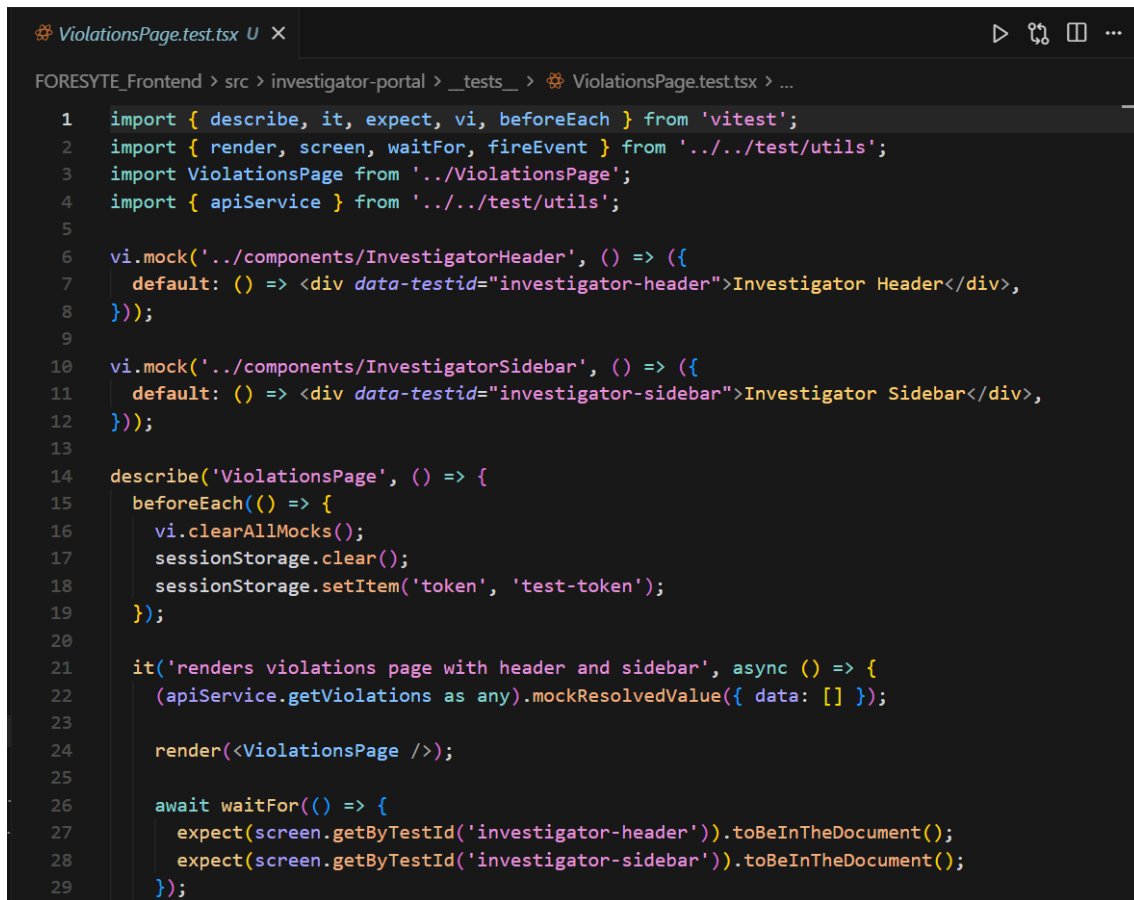


```
InvestigatorDashboard.test.tsx X
FORESYTE_Frontend > src > investigator-portal > __tests__ > InvestigatorDashboard.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import InvestigatorDashboard from '../InvestigatorDashboard';
4  import { apiService } from '../test/utils';
5
6  // Mock investigator components
7  vi.mock('../components/InvestigatorHeader', () => ({
8    default: () => <div data-testid="investigator-header">Investigator Header</div>,
9  }));
10
11 vi.mock('../components/InvestigatorSidebar', () => ({
12   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
13 });
14
15 describe('InvestigatorDashboard', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20   });
21
22   it('renders dashboard with header and sidebar', async () => {
23     (apiService.getDashboardStats as any).mockResolvedValue({
24       data: { active_exams: 5 },
25     });
26     (apiService.getViolations as any).mockResolvedValue({ data: [] });
27     (apiService.getRecentIncidents as any).mockResolvedValue({ data: [] });
28
29     render(<InvestigatorDashboard />);
```

A.1.2.2 Violations Page

Figure A.15: Investigator Portal Violations Management Page

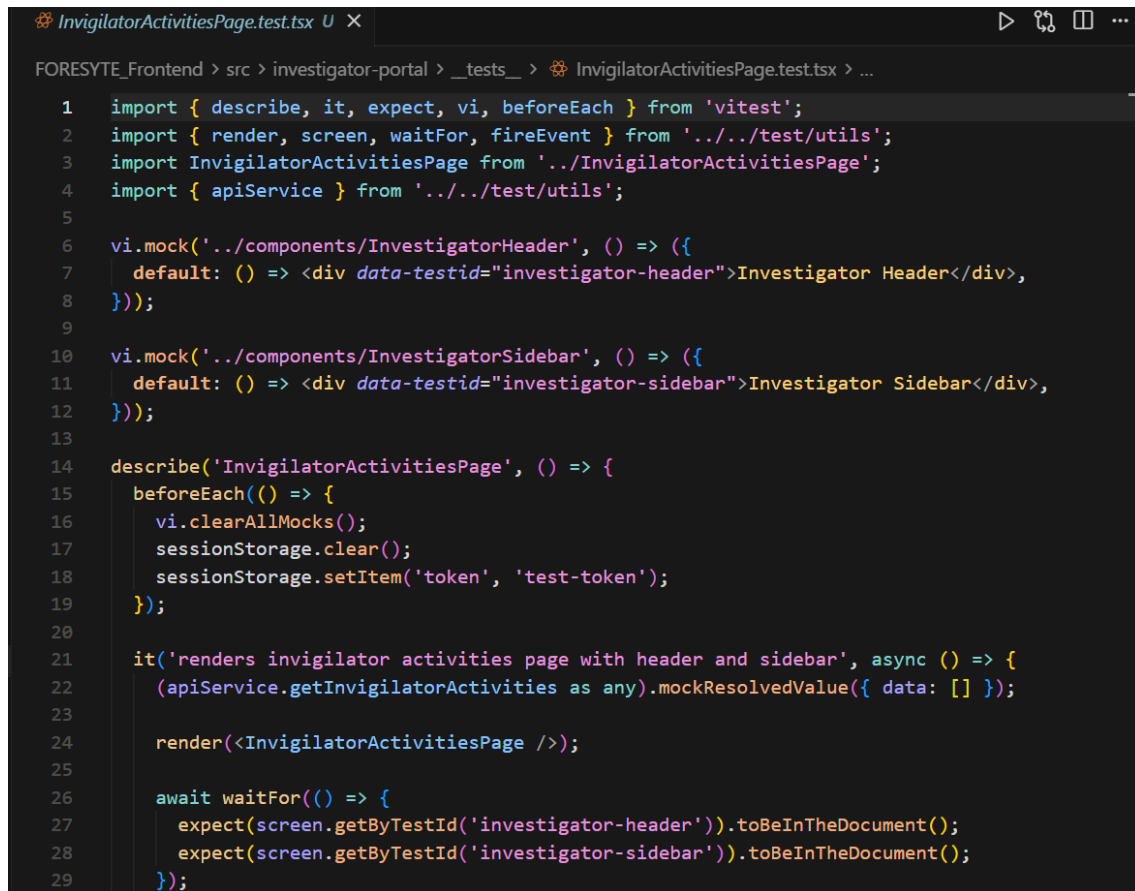


```
ViolationsPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > ViolationsPage.test.tsx > ...

1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor, fireEvent } from '../test/utils';
3 import ViolationsPage from '../ViolationsPage';
4 import { apiService } from '../test/utils';
5
6 vi.mock('../components/InvestigatorHeader', () => ({
7   default: () => <div data-testid="investigator-header">Investigator Header</div>,
8 }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 }));
13
14 describe('ViolationsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders violations page with header and sidebar', async () => {
22     (apiService.getViolations as any).mockResolvedValue({ data: [] });
23
24     render(<ViolationsPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.3 Invigilator Activities Page

Figure A.16: Investigator Portal Invigilator Activities Page

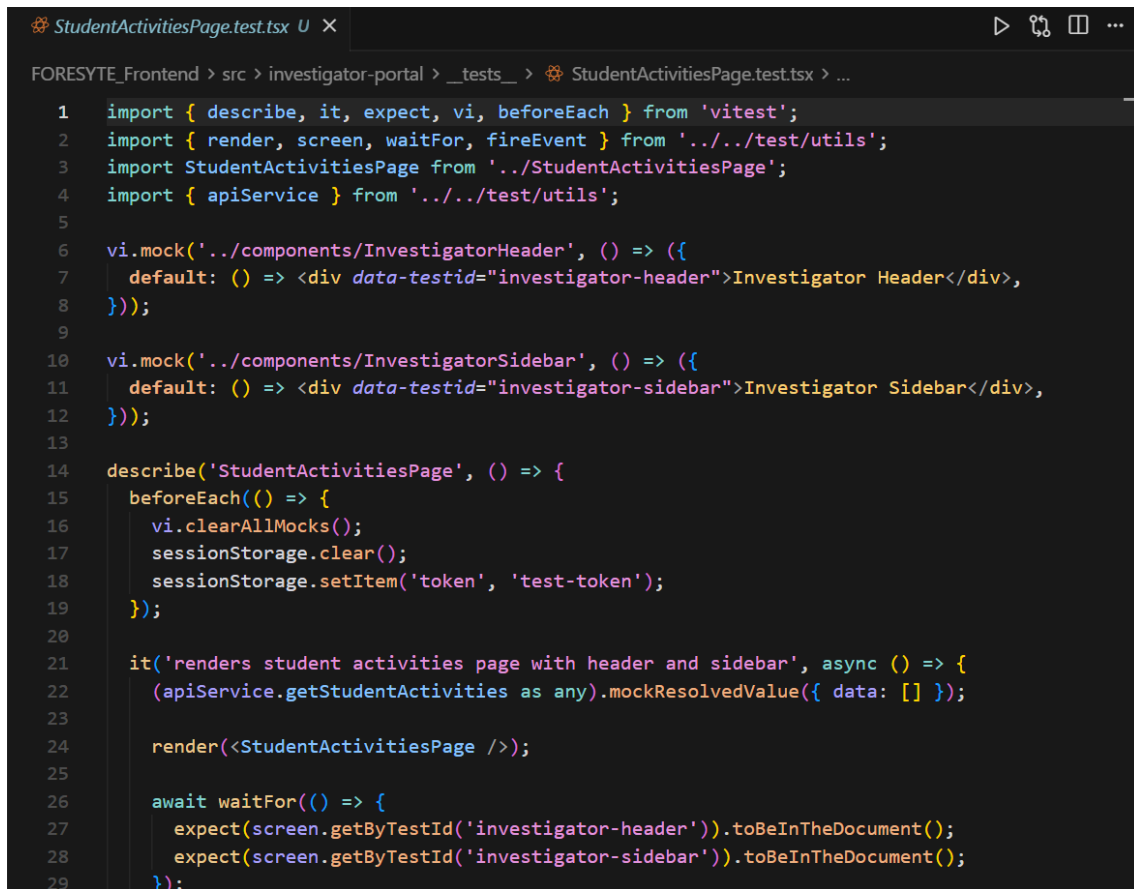


```
InvigilatorActivitiesPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > InvigilatorActivitiesPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import InvigilatorActivitiesPage from '../InvigilatorActivitiesPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('InvigilatorActivitiesPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders invigilator activities page with header and sidebar', async () => {
22     (apiService.getInvigilatorActivities as any).mockResolvedValue({ data: [] });
23
24     render(<InvigilatorActivitiesPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.4 Student Activities Page

Figure A.17: Investigator Portal Student Activities Page



```
StudentActivitiesPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > StudentActivitiesPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import StudentActivitiesPage from '../StudentActivitiesPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('StudentActivitiesPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders student activities page with header and sidebar', async () => {
22     (apiService.getStudentActivities as any).mockResolvedValue({ data: [] });
23
24     render(<StudentActivitiesPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.5 Reports Page

Figure A.18: Investigator Portal Reports Page

```
ReportsPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > _tests_ > ReportsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import ReportsPage from '../ReportsPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('ReportsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders reports page with header and sidebar', async () => {
22     (apiService.getReports as any).mockResolvedValue({ data: { reports: [] } });
23
24     render(<ReportsPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.6 Live Monitoring Page

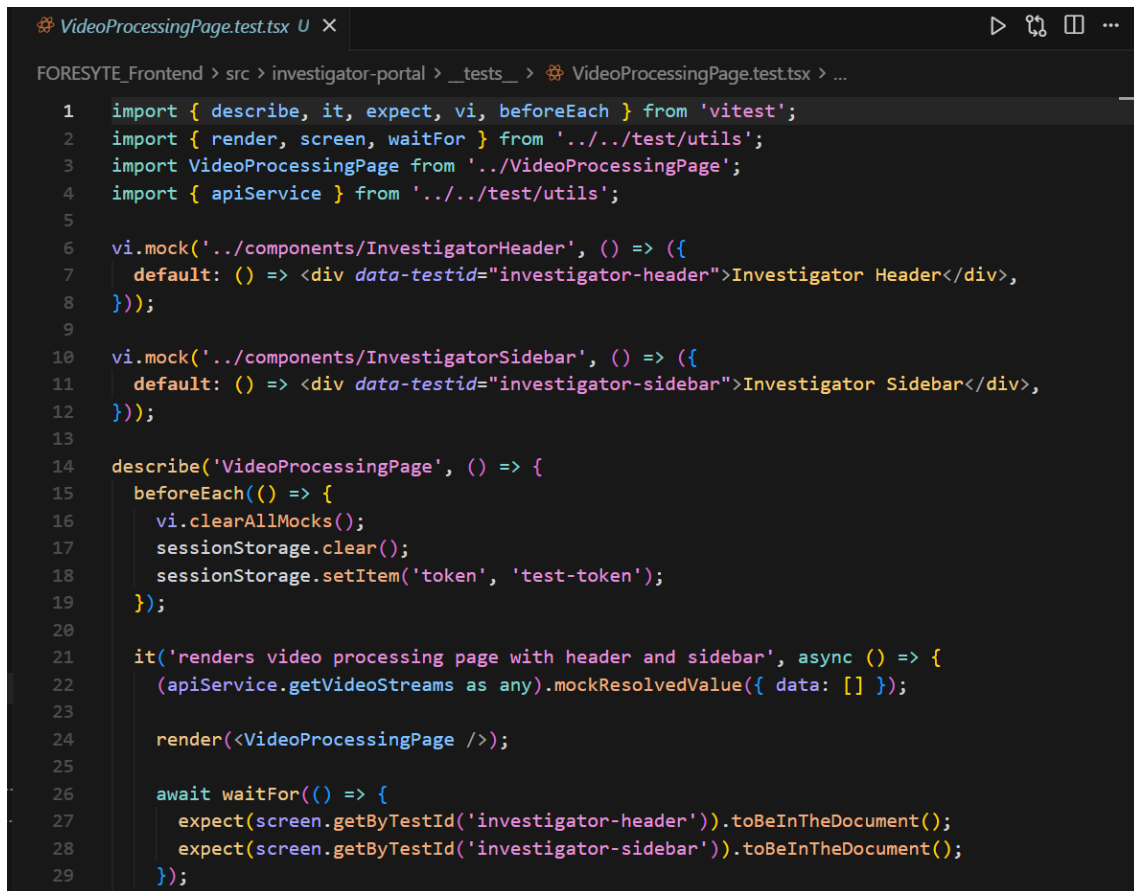
Figure A.19: Investigator Portal Live Monitoring Page

```
LiveMonitoringPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > LiveMonitoringPage.test.tsx > ...

1 import { describe, it, expect, vi, beforeEach } from 'vitest';
2 import { render, screen, waitFor } from '../test/utils';
3 import LiveMonitoringPage from '../LiveMonitoringPage';
4 import { apiService } from '../test/utils';
5
6 vi.mock('../components/InvestigatorHeader', () => ({
7   default: () => <div data-testid="investigator-header">Investigator Header</div>,
8 }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 }));
13
14 describe('LiveMonitoringPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders live monitoring page with header and sidebar', async () => {
22     (apiService.getMonitoringFeeds as any).mockResolvedValue({ data: [] });
23
24     render(<LiveMonitoringPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.7 Video Processing Page

Figure A.20: Investigator Portal Video Processing Page

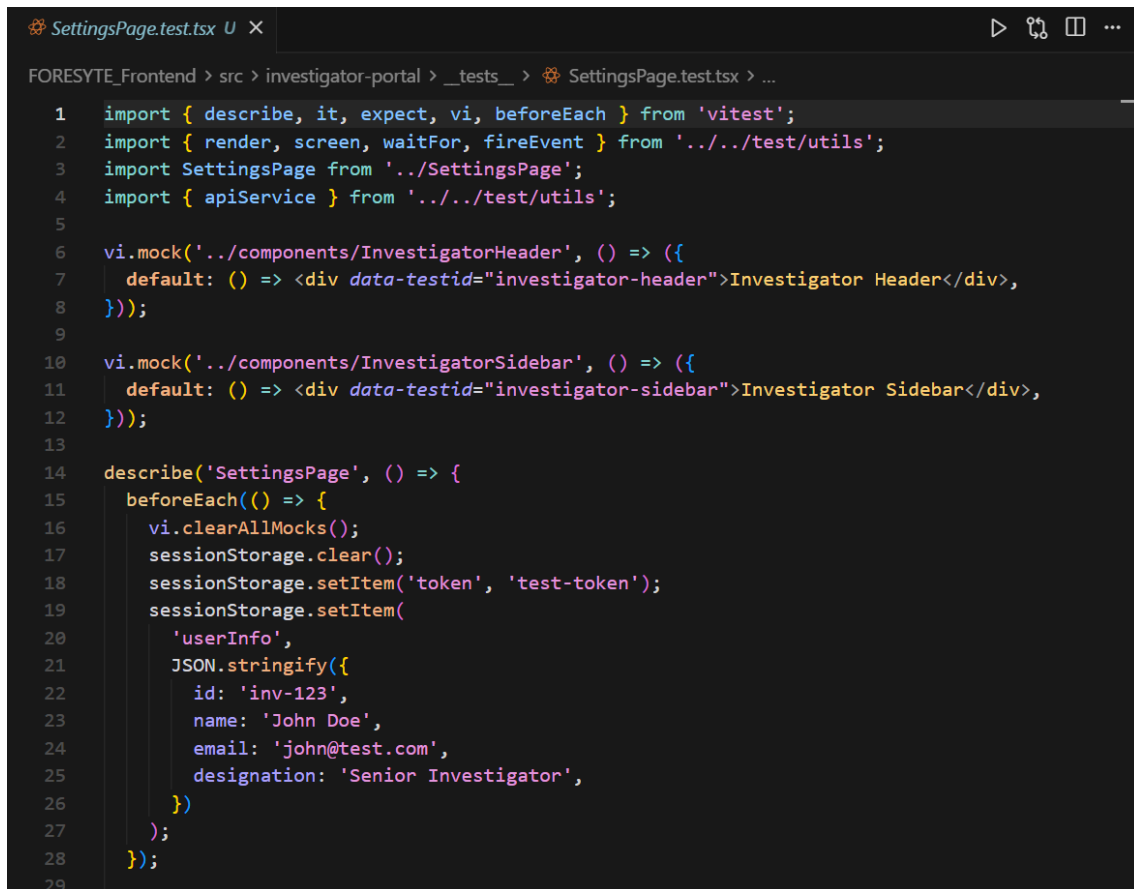


```
VideoProcessingPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > VideoProcessingPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor } from '../test/utils';
3  import VideoProcessingPage from '../VideoProcessingPage';
4  import { apiService } from '../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('VideoProcessingPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19   });
20
21   it('renders video processing page with header and sidebar', async () => {
22     (apiService.getVideoStreams as any).mockResolvedValue({ data: [] });
23
24     render(<VideoProcessingPage />);
25
26     await waitFor(() => {
27       expect(screen.getByTestId('investigator-header')).toBeInTheDocument();
28       expect(screen.getByTestId('investigator-sidebar')).toBeInTheDocument();
29     });
30   });
31 });
```

A.1.2.8 Settings Page

Figure A.21: Investigator Portal Settings Page

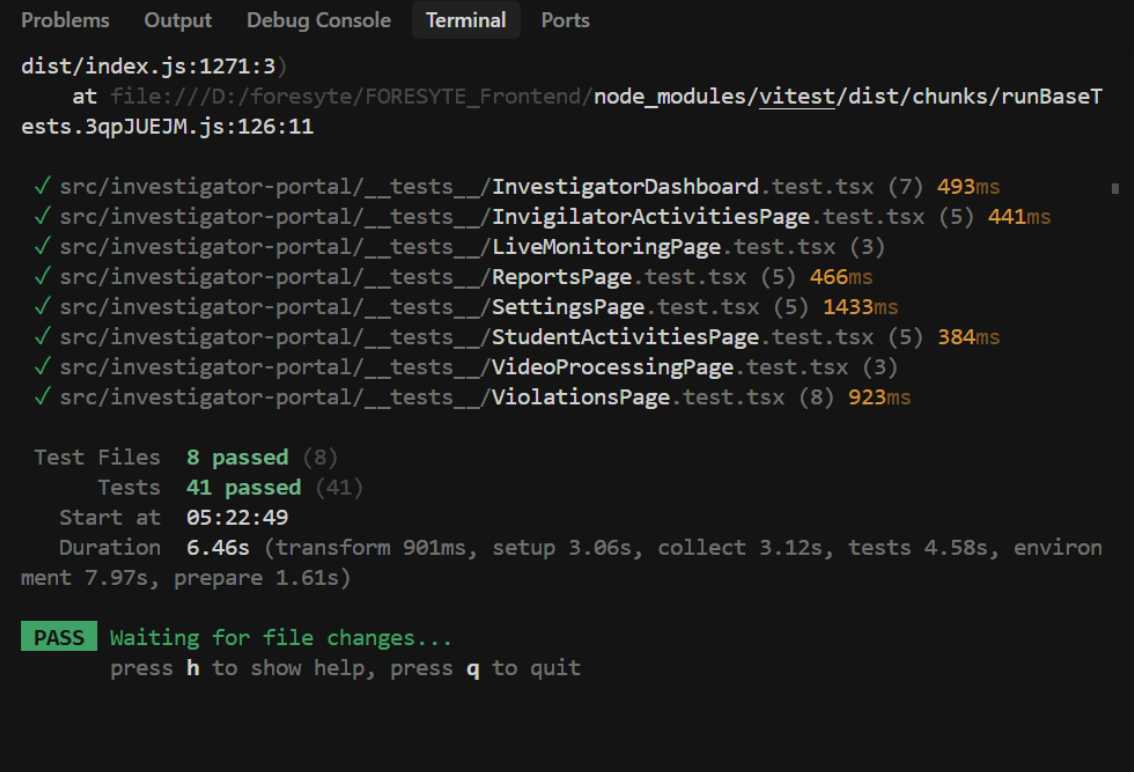


```
SettingsPage.test.tsx U X
FORESYTE_Frontend > src > investigator-portal > __tests__ > SettingsPage.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../../test/utils';
3  import SettingsPage from '../SettingsPage';
4  import { apiService } from '../../test/utils';
5
6  vi.mock('../components/InvestigatorHeader', () => ({
7    default: () => <div data-testid="investigator-header">Investigator Header</div>,
8  }));
9
10 vi.mock('../components/InvestigatorSidebar', () => ({
11   default: () => <div data-testid="investigator-sidebar">Investigator Sidebar</div>,
12 });
13
14 describe('SettingsPage', () => {
15   beforeEach(() => {
16     vi.clearAllMocks();
17     sessionStorage.clear();
18     sessionStorage.setItem('token', 'test-token');
19     sessionStorage.setItem(
20       'userInfo',
21       JSON.stringify({
22         id: 'inv-123',
23         name: 'John Doe',
24         email: 'john@test.com',
25         designation: 'Senior Investigator',
26       })
27     );
28   });
29 }
```

A.1.2.9 Test Results

Figure A.22: Investigator Portal Unit Test Results



```
Problems Output Debug Console Terminal Ports
dist/index.js:1271:3)
  at file:///D:/foresyte/FORESYTE_Frontend/node_modules/vitest/dist/chunks/runBaseT
ests.3qpJUEJM.js:126:11

✓ src/investigator-portal/__tests__/InvestigatorDashboard.test.tsx (7) 493ms
✓ src/investigator-portal/__tests__/InvigilatorActivitiesPage.test.tsx (5) 441ms
✓ src/investigator-portal/__tests__/LiveMonitoringPage.test.tsx (3)
✓ src/investigator-portal/__tests__/ReportsPage.test.tsx (5) 466ms
✓ src/investigator-portal/__tests__/SettingsPage.test.tsx (5) 1433ms
✓ src/investigator-portal/__tests__/StudentActivitiesPage.test.tsx (5) 384ms
✓ src/investigator-portal/__tests__/VideoProcessingPage.test.tsx (3)
✓ src/investigator-portal/__tests__/ViolationsPage.test.tsx (8) 923ms

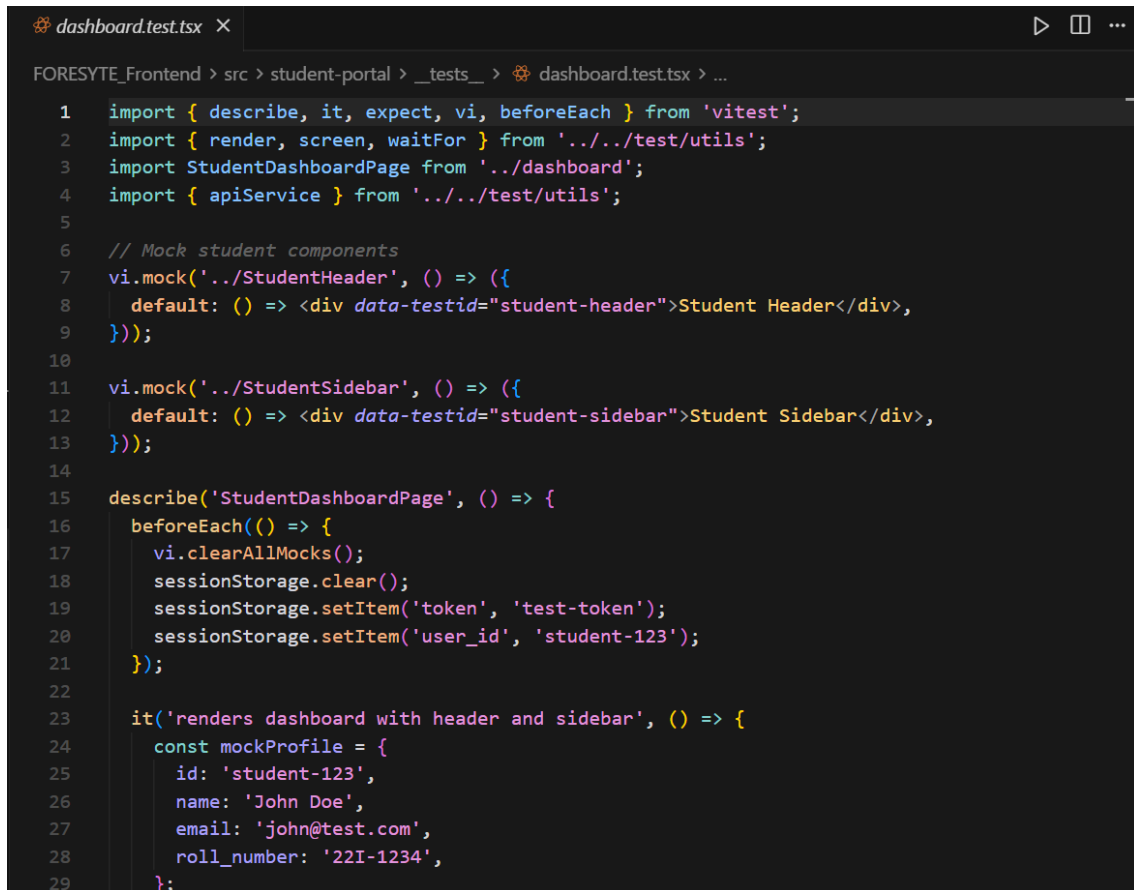
Test Files 8 passed (8)
Tests 41 passed (41)
Start at 05:22:49
Duration 6.46s (transform 901ms, setup 3.06s, collect 3.12s, tests 4.58s, environ
ment 7.97s, prepare 1.61s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

A.1.3 Student Portal Screenshots

A.1.3.1 Student Dashboard

Figure A.23: Student Portal Dashboard



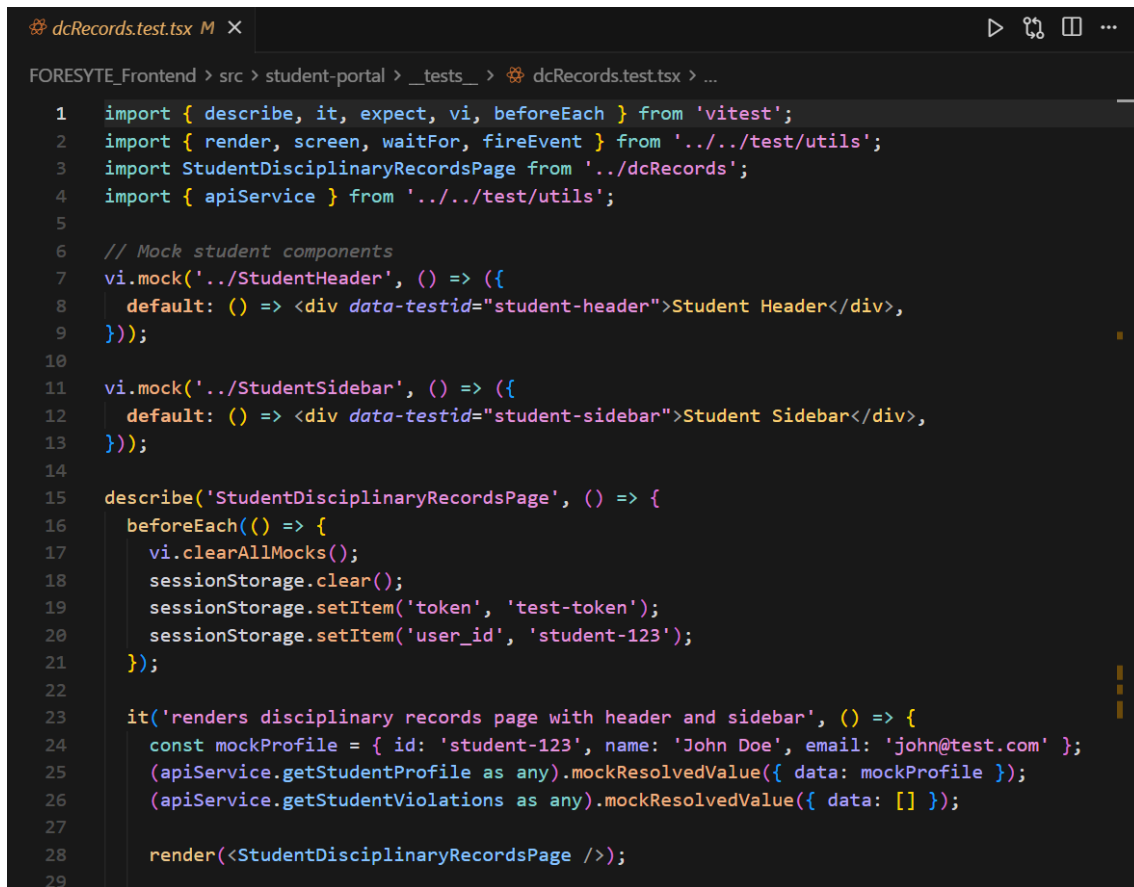
```
dashboard.test.tsx X
FORESYTE_Frontend > src > student-portal > _tests_ > dashboard.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor } from '../test/utils';
3  import StudentDashboardPage from '../dashboard';
4  import { apiService } from '../test/utils';
5
6  // Mock student components
7  vi.mock('../StudentHeader', () => ({
8    default: () => <div data-testid="student-header">Student Header</div>,
9  }));
10
11 vi.mock('../StudentSidebar', () => ({
12   default: () => <div data-testid="student-sidebar">Student Sidebar</div>,
13 });
14
15 describe('StudentDashboardPage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20     sessionStorage.setItem('user_id', 'student-123');
21   });
22
23   it('renders dashboard with header and sidebar', () => {
24     const mockProfile = {
25       id: 'student-123',
26       name: 'John Doe',
27       email: 'john@test.com',
28       roll_number: '22I-1234',
29     };

```

A.1.3.2 Disciplinary Records Page

Figure A.24: Student Portal Disciplinary Records Page

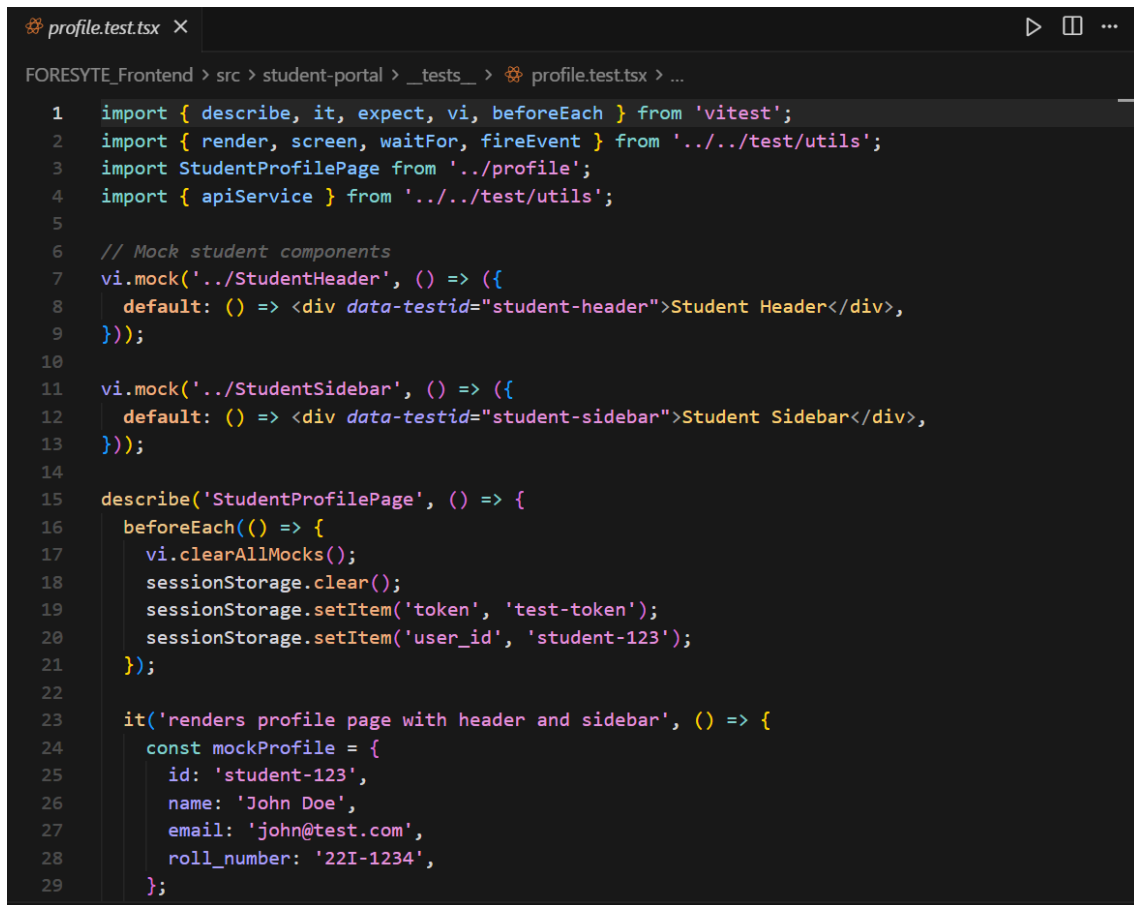


```
dcRecords.test.tsx M X
FORESYTE_Frontend > src > student-portal > __tests__ > dcRecords.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import StudentDisciplinaryRecordsPage from '../dcRecords';
4  import { apiService } from '../test/utils';
5
6  // Mock student components
7  vi.mock('../StudentHeader', () => ({
8    default: () => <div data-testid="student-header">Student Header</div>,
9  }));
10
11 vi.mock('../StudentSidebar', () => ({
12   default: () => <div data-testid="student-sidebar">Student Sidebar</div>,
13 });
14
15 describe('StudentDisciplinaryRecordsPage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20     sessionStorage.setItem('user_id', 'student-123');
21   });
22
23   it('renders disciplinary records page with header and sidebar', () => {
24     const mockProfile = { id: 'student-123', name: 'John Doe', email: 'john@test.com' };
25     (apiService.getStudentProfile as any).mockResolvedValue({ data: mockProfile });
26     (apiService.getStudentViolations as any).mockResolvedValue({ data: [] });
27
28     render(<StudentDisciplinaryRecordsPage />);
29   });
}
```

A.1.3.3 Profile Page

Figure A.25: Student Portal Profile Page

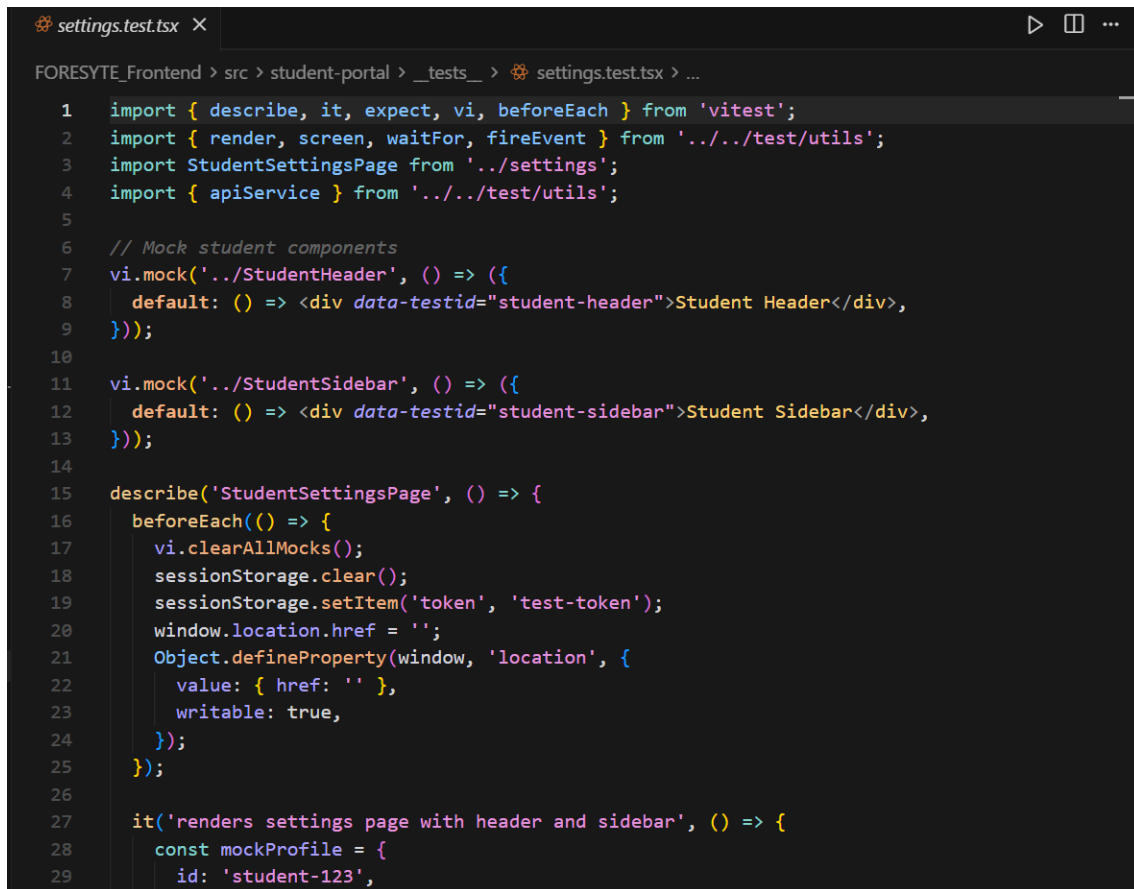


```
profile.test.tsx X
FORESYTE_Frontend > src > student-portal > __tests__ > profile.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import StudentProfilePage from '../profile';
4  import { apiService } from '../test/utils';
5
6  // Mock student components
7  vi.mock('../StudentHeader', () => ({
8    default: () => <div data-testid="student-header">Student Header</div>,
9  }));
10
11 vi.mock('../StudentSidebar', () => ({
12   default: () => <div data-testid="student-sidebar">Student Sidebar</div>,
13 });
14
15 describe('StudentProfilePage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20     sessionStorage.setItem('user_id', 'student-123');
21   });
22
23   it('renders profile page with header and sidebar', () => {
24     const mockProfile = {
25       id: 'student-123',
26       name: 'John Doe',
27       email: 'john@test.com',
28       roll_number: '22I-1234',
29     };
30   });
31 });
```

A.1.3.4 Settings Page

Figure A.26: Student Portal Settings Page



```
settings.test.tsx X
FORESYTE_Frontend > src > student-portal > _tests_ > settings.test.tsx > ...

1  import { describe, it, expect, vi, beforeEach } from 'vitest';
2  import { render, screen, waitFor, fireEvent } from '../test/utils';
3  import StudentSettingsPage from '../settings';
4  import { apiService } from '../test/utils';
5
6  // Mock student components
7  vi.mock('../StudentHeader', () => ({
8    default: () => <div data-testid="student-header">Student Header</div>,
9  }));
10
11 vi.mock('../StudentSidebar', () => ({
12   default: () => <div data-testid="student-sidebar">Student Sidebar</div>,
13 });
14
15 describe('StudentSettingsPage', () => {
16   beforeEach(() => {
17     vi.clearAllMocks();
18     sessionStorage.clear();
19     sessionStorage.setItem('token', 'test-token');
20     window.location.href = '';
21     Object.defineProperty(window, 'location', {
22       value: { href: '' },
23       writable: true,
24     });
25   });
26
27   it('renders settings page with header and sidebar', () => {
28     const mockProfile = {
29       id: 'student-123',
```

A.1.3.5 Test Results

Figure A.27: Student Portal Unit Test Results

```
✓ src/student-portal/__tests__/dashboard.test.tsx (9) 467ms
✓ src/student-portal/__tests__/dcRecords.test.tsx (9) 949ms
✓ src/student-portal/__tests__/profile.test.tsx (7) 1021ms
✓ src/student-portal/__tests__/settings.test.tsx (11) 1617ms
✓ src/student-portal/__tests__/uploadImage.test.tsx (9) 317ms

Test Files 5 passed (5)
  Tests 45 passed (45)
  Start at 05:01:47
  Duration 5.36s (transform 787ms, setup 1.66s, collect 2.11s, tests 4.37s, environ-
ment 5.15s, prepare 1.00s)

PASS Waiting for file changes...
  press h to show help, press q to quit
```